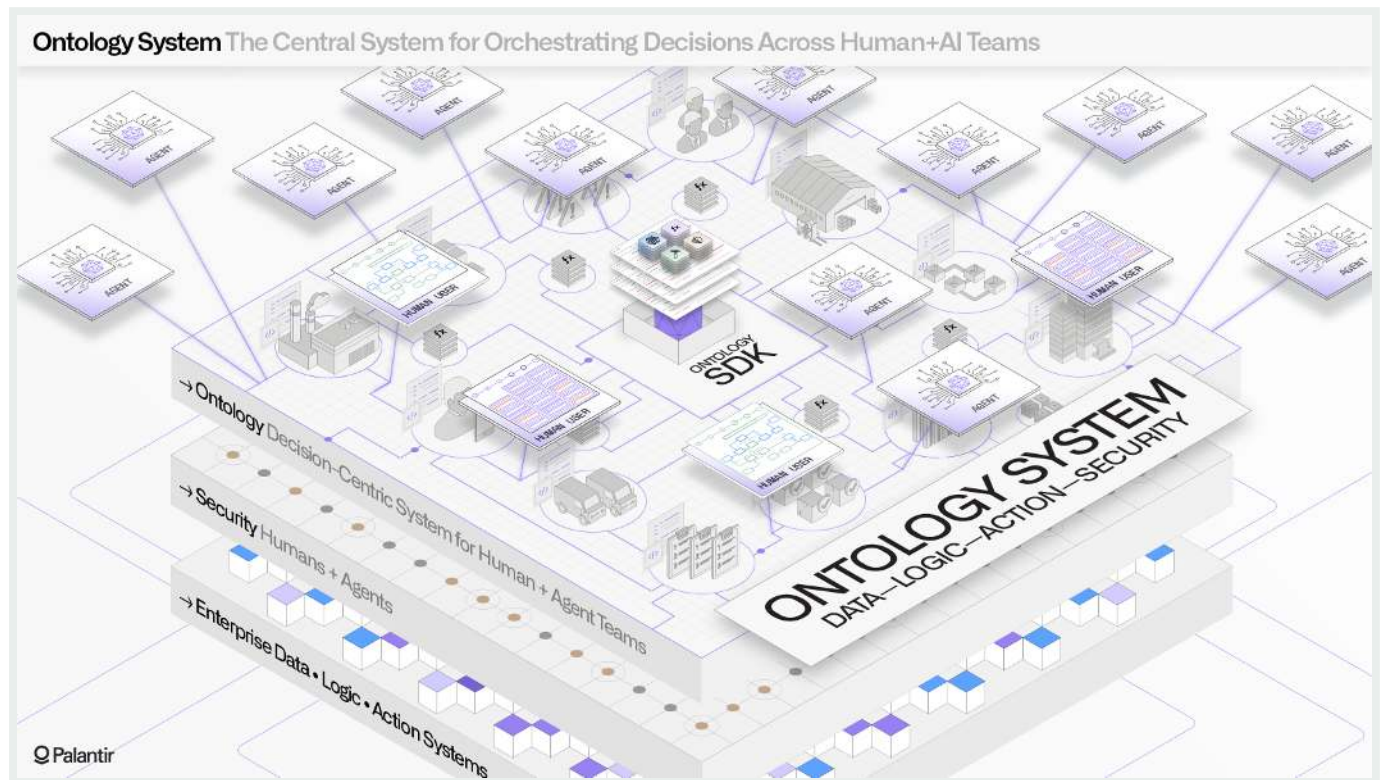


Table of Contents

1. Overview • Ontology • Palantir
2. Overview • Ontology • Palantir
3. Why create an Ontology? • Palantir
4. Models in the Ontology • Palantir
5. Core concepts • Palantir
6. Ontology-aware applications • Palantir
7. Ontology design: Best practices • Palantir
8. Ontology design: Structural guidance • Palantir
9. Ontology design: Anti-patterns • Palantir
10. Semantic search • Overview • Palantir
11. Semantic search • Document processing • Palantir
12. Semantic search • Process multimodal and embedding models • Palantir
13. Semantic search • Ontology augmented generation • Palantir
14. Semantic search • Use Palantir-provided models to create a semantic search workflow • Palantir
15. Semantic search • Use custom models to create a semantic search workflow • Palantir
16. Search syntax • Palantir
17. Derived properties • Palantir
18. Overview • Ontology • Palantir
19. Temporary scenarios • Palantir
20. Store scenario metadata as objects • Palantir
21. Merge scenarios • Palantir
22. Use scenarios with OSDK • Palantir

Ontology building / Overview



Ontology building

The Palantir **Ontology** is an operational layer for the organization. The Ontology sits on top of the digital assets integrated into the Palantir platform ([datasets](#), [virtual tables](#), and [models](#)) and connects them to their real-world counterparts, ranging from physical assets like plants, equipment, and products to concepts like customer orders or financial transactions. In many settings, the Ontology serves as a digital twin of the organization, containing both the semantic elements (objects, properties, links) and kinetic elements (actions, functions, dynamic security) needed to enable use cases of all types.

link types

[API Reference ↗](#)



 semantics of your organization happens by mapping existing
 [objects, properties, and links](#) in the Ontology. Far beyond data cataloging or schema design
 [Send feedback](#)

solutions, the Ontology allows you to define a robust foundation for end-user workflows, including rich metadata for all fields and complete with granular security and governance for all changes.

Learn about creating the semantic elements of the Ontology: [object types](#) and [link types](#).

Action types and functions

The kinetics of the organization—enabling change while complying with organizational controls and governance—are defined in the Ontology using **action types** and **functions**. Action types enable you to capture data from operators in your organization or orchestrate decision-making processes that connect to your existing systems, while functions provide a way to author and evolve business logic with arbitrary complexity.

Learn about creating the kinetic elements of the Ontology: [action types](#) and [functions](#).

Interfaces

An **interface** is an Ontology type that describes the shape of an object type and its capabilities. Interfaces provide object type polymorphism, allowing for consistent modeling of and interaction with object types that share a common shape.

Learn more about [interfaces](#).

Powering decision-making

The goal of investing in the Ontology is to facilitate better decision-making in an organization at scale. To achieve this, the Ontology is deeply integrated into Palantir's user-facing analytical and operational tools: users can create reusable Object Views, search for objects of interest in Object Explorer, perform complex analyses in Quiver, build high-quality applications in Workshop, and more.

[API Reference ↗](#) [to leverage the Ontology in user-facing applications.](#)

Send feedback

Palantir Learning portal

Now that you know the theory, get started on building your first Ontology with our course on learn.palantir.com ↗.

NEXT

Why create an Ontology? →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

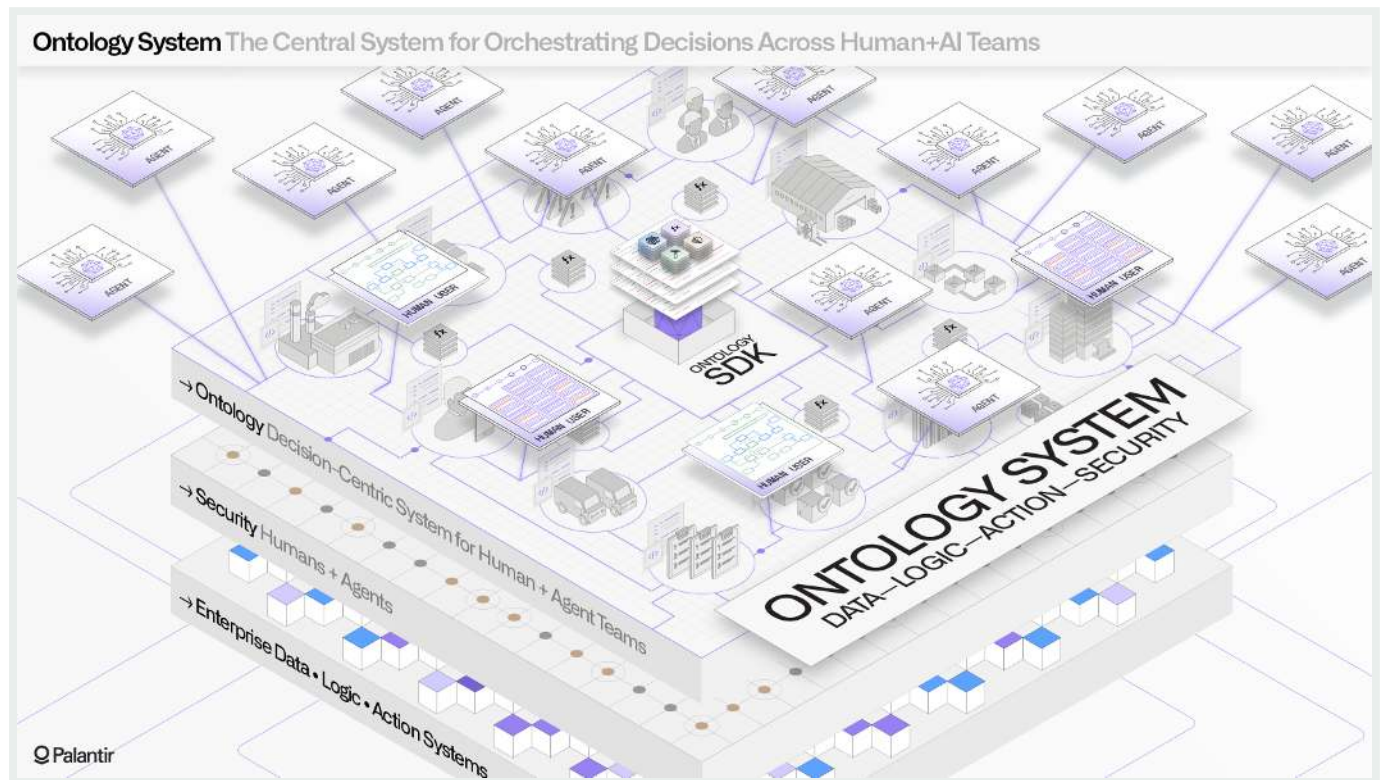
[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Overview



Ontology building

The Palantir **Ontology** is an operational layer for the organization. The Ontology sits on top of the digital assets integrated into the Palantir platform ([datasets](#), [virtual tables](#), and [models](#)) and connects them to their real-world counterparts, ranging from physical assets like plants, equipment, and products to concepts like customer orders or financial transactions. In many settings, the Ontology serves as a digital twin of the organization, containing both the semantic elements (objects, properties, links) and kinetic elements (actions, functions, dynamic security) needed to enable use cases of all types.

link types

[API Reference ↗](#)



 semantics of your organization happens by mapping existing
 [objects, properties, and links](#) in the Ontology. Far beyond data cataloging or schema design
 [Send feedback](#)

solutions, the Ontology allows you to define a robust foundation for end-user workflows, including rich metadata for all fields and complete with granular security and governance for all changes.

Learn about creating the semantic elements of the Ontology: [object types](#) and [link types](#).

Action types and functions

The kinetics of the organization—enabling change while complying with organizational controls and governance—are defined in the Ontology using **action types** and **functions**. Action types enable you to capture data from operators in your organization or orchestrate decision-making processes that connect to your existing systems, while functions provide a way to author and evolve business logic with arbitrary complexity.

Learn about creating the kinetic elements of the Ontology: [action types](#) and [functions](#).

Interfaces

An **interface** is an Ontology type that describes the shape of an object type and its capabilities. Interfaces provide object type polymorphism, allowing for consistent modeling of and interaction with object types that share a common shape.

Learn more about [interfaces](#).

Powering decision-making

The goal of investing in the Ontology is to facilitate better decision-making in an organization at scale. To achieve this, the Ontology is deeply integrated into Palantir's user-facing analytical and operational tools: users can create reusable Object Views, search for objects of interest in Object Explorer, perform complex analyses in Quiver, build high-quality applications in Workshop, and more.

[API Reference ↗](#) [to leverage the Ontology in user-facing applications.](#)

Send feedback

Palantir Learning portal

Now that you know the theory, get started on building your first Ontology with our course on learn.palantir.com ↗.

NEXT

Why create an Ontology? →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Why create an Ontology?

Why create an Ontology?

The Palantir platform powers real-time, human-agent decision-making in the most critical commercial and government contexts around the world. The Ontology is the central system that enables customers to safely, securely, and effectively leverage AI in their enterprises and drive operational transformation.

The Ontology represents the *decisions* in an enterprise, not simply the data. With the Ontology, organizations can make the best possible decisions, often in real time, based on constantly changing internal and external conditions. Traditional data architectures do not capture the reasoning that goes into decision-making or the actions that follow, and therefore limit learning and the incorporation of AI. Conventional analytics architectures do not contextualize computation in lived reality, and remain disconnected from operations. In contrast, the decision-centric Ontology connects humans and agents directly to operations to face and overcome an organization's toughest challenges.

Understanding the value of the Ontology

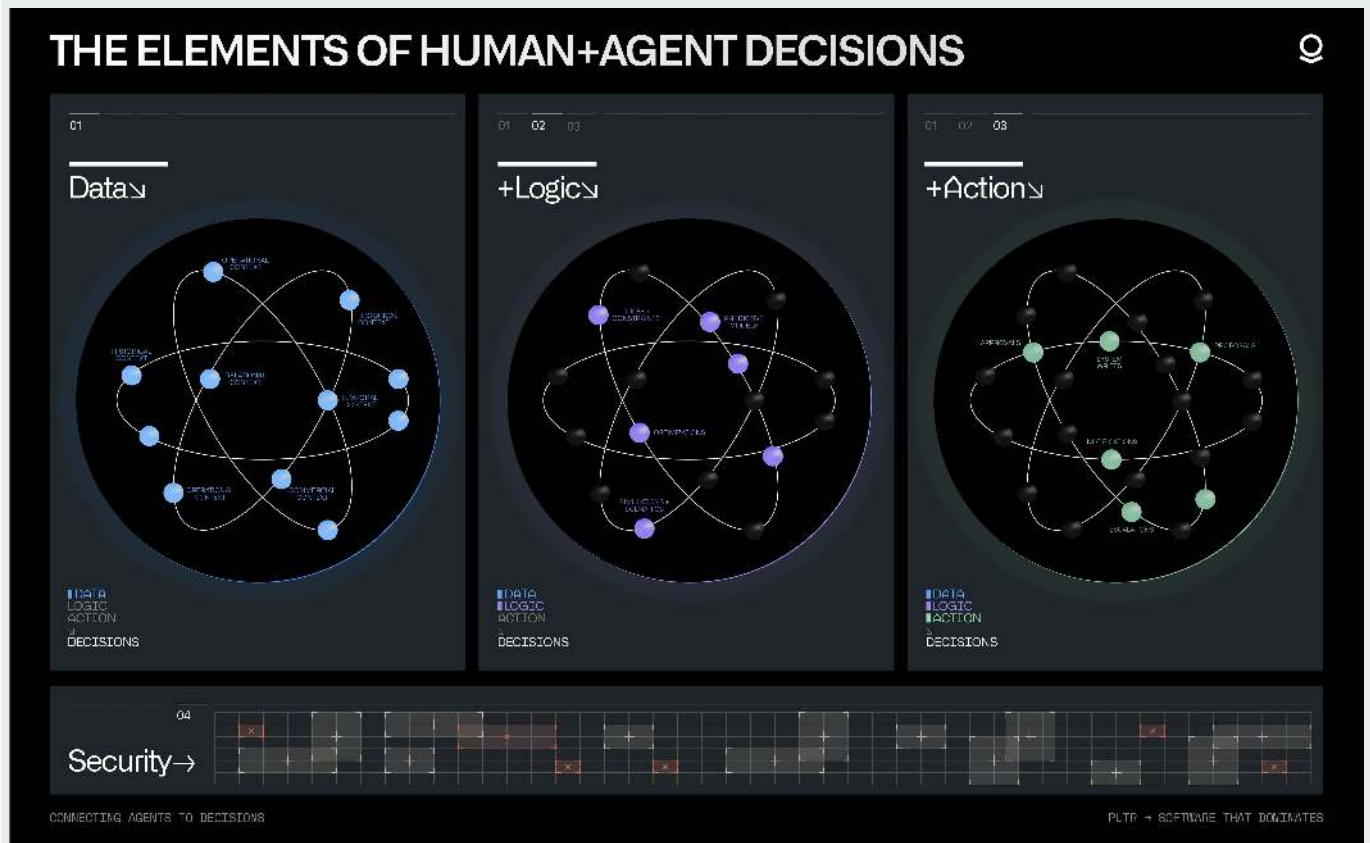
Palantir models each operational decision as comprising four components:

- **Data:** The information leveraged to make the decision.
- **Logic:** The heuristics and computational processes that evaluate a decision.
- **Action:** The orchestration and execution of the chosen decision.
- **Security:** The assurance that the decision complies with operational policies.

API Reference ↗



Send feedback



The Ontology integrates these four elements into a scalable, dynamic, collaborative resource which enables decision-making, informed by the ever-changing conditions and needs of your organization.

Data

The Ontology includes not only the many sources of enterprise data — structured data, streaming and edge sources, unstructured repositories, imagery data, and more — but also the data generated by end users and agents as decisions are being made. This "decision data" contains the context surrounding a given decision, the different options evaluated, and the downstream implications of the committed choice. Integrating the full range of enterprise data alongside decision data requires a different architecture than a classical database management solution optimized for reporting and analytics.

The Ontology integrates this data into a full-scale, full-fidelity semantic representation of the enterprise. The wide range of operational data sources (such as ERPs, MES, and WMS) can be contextualized alongside data streams from IoT and edge systems, the unstructured data repositories, geospatial data stores, and more. The Ontology unites these data sources in the form of objects, properties in real-time, and are designed to be embedded directly into decision-making environments.

[API Reference ↗](#)

[Send feedback](#)

The Ontology safely captures the decision data produced by operational users as they carry out daily work, whether in supply chains, hospital systems, customer service centers, or elsewhere. This includes decisions made at the edge, captured through the lightweight [Embedded Ontology](#). The end-to-end "decision lineage" of when a given decision was made, atop which version of enterprise data, and through which application, is automatically captured and securely accessible to both human developers and agents. Together, these data resources can power AI-driven learning at scale and continuously refine short-term and long-term agentic memory.



Logic

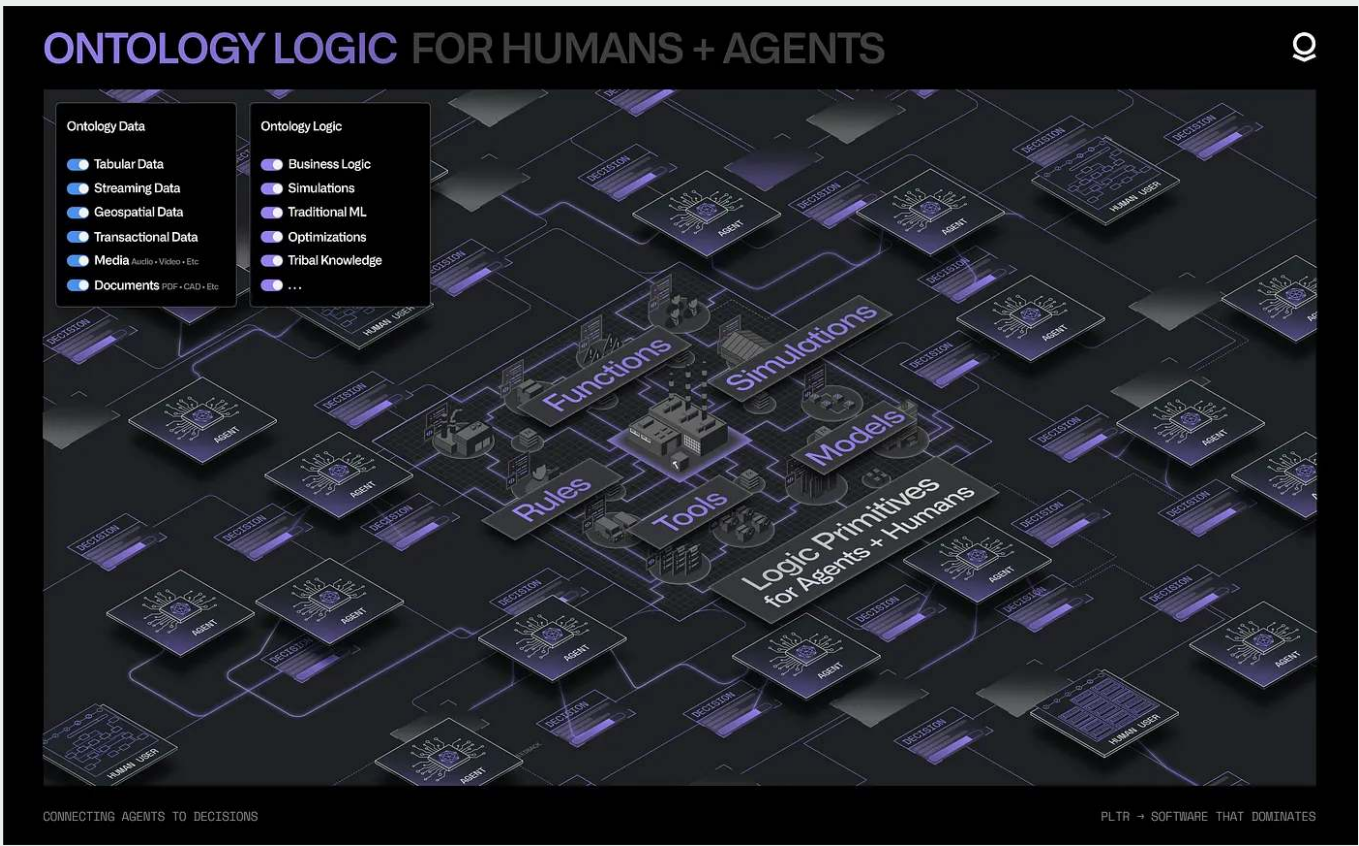
The data stored in the Ontology is complemented by the reasoning, or logic, that determines when and how to make a given decision. Examples of decision logic include a simple piece of business logic within a core business system, a forecast model maintained using a cloud data science workbench, or an optimization model that uses several data sources to produce an operational plan.

With [API Reference](#) agentic orchestration, it is critical that AI-driven reasoning can leverage these logical assets in the same way that humans have historically. [Send feedback](#)

algorithms, and conventional statistical processes can serve as operational tools which complement the non-deterministic reasoning of LLMs and multi-modal models.

The Ontology enables the full set of logic assets — the calculations and processes that dictate *how* decisions are made — to be connected and contextualized for both human and agents. This includes business logic related to customer interactions, often found in CRMs and ERPs; the modeling logic that drives conventional machine learning, which is spread across data science environments; and the planning, optimization, and simulation algorithms that are often associated with domain-specific tools.

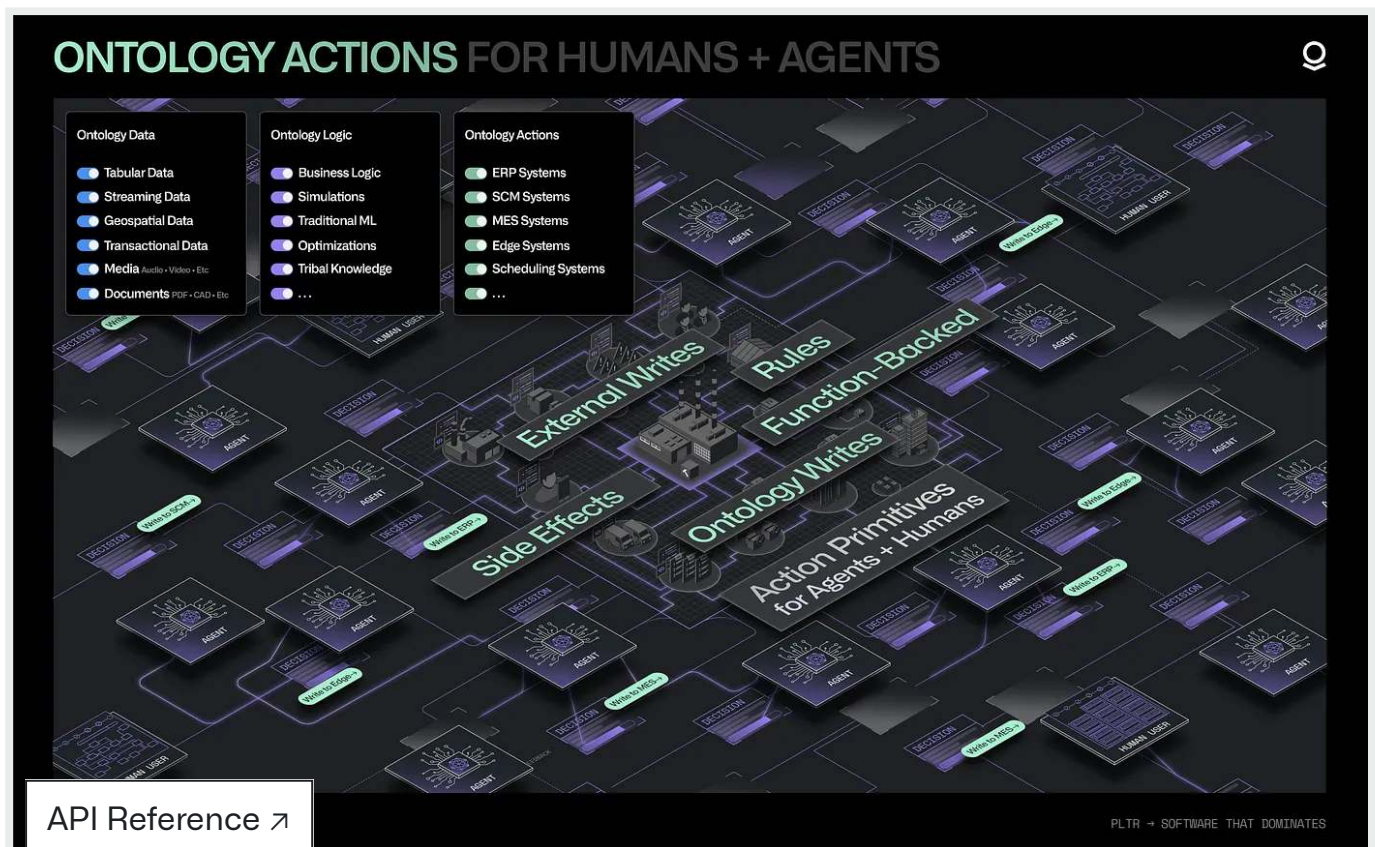
The Ontology’s flexible "logic binding" paradigm provides a consistent interface for constructing workflows that incorporate and combine heterogeneous logic assets from different environments (such as on-premises data centers, enterprise cloud environments, SaaS environments, or the Palantir platform itself). This enables the introduction of agent-driven reasoning into decision-making contexts with diverse sets of logic, which were previously the exclusive domain of human users.



With both information (the data) and reasoning (the logic) incorporated into a shared representation, the next piece is the execution and orchestration of the decision itself (the action). Closing the action loop as decisions are made in real-time is what distinguishes an operational system from an analytical system.

The Ontology natively models actions within a cohesive, decision-centric model of the enterprise. If the data elements in the Ontology are “the nouns” of the enterprise (the semantic, real-world objects and links), then the actions can be considered “the verbs” (the kinetic, real-world execution). With every Ontology-driven workflow, the nouns and the verbs are brought together into complete sentences through human- and/or AI-driven reasoning, which incorporates various pieces of logic.

Uniting data within a semantic model and combining it with the logic required to evaluate decisions is valuable, but ultimately limited unless the executed decisions can be synchronized with operational systems in a way that *compounds*, with each decision informing the next in a shared lineage. The Ontology enables human and agent actions to be safely staged as scenarios, governed with the same granular access controls as data and logic primitives, and securely written back to every enterprise substrate (transactional systems, edge devices, custom applications, and so on).



Security

In an operational setting, human-agent interaction requires rigorous security and governance capabilities that can go beyond conventional role-driven policies on buckets of data. Palantir provides a security architecture that combines:

- Marking-, purpose-, and role-based policies;
- Dynamic lineage that flows across data, logic, action, and application artifacts; and
- A full suite of integrated change and release management tools that apply across both human-driven and agentic workflows.

Granular policies can constrain both agentic and human access to sensitive or context-dependent information across the Ontology. These policies are dynamically computed at runtime for every interaction, combining row- and column-level restrictions that have been applied to underlying datasets, attributes of particular user groups (including those that flow via SSO), security markings that propagate across underlying data pipelines, and more.

Tool usage is dynamically enforced through the same security architecture that governs data access and all forms of memory. This ensures, at minimum, that any tool invocations are dependent on access to the underlying objects, properties, and links in the Ontology. Tools can also contain runtime validations that are dependent on granular submission criteria.

Every agentic or human action depends on precise authorization grants that explicitly dictate the set of allowable operations, safeguarding against unexpected invocations (such as querying data that exists across organizational boundaries, or tools that connect to unspecified external systems) and other forms of privilege escalation.

As detailed telemetry is generated by agents, the security and transmission of the logs is a critical last-mile concern. Palantir enables administrators to control how logging is accessible across specific projects, workflows, and agents. Data markings and other active security primitives govern log access, in the same manner that they govern access to the underlying data, logic, and action primitives.

The Ontology brings together data, logic, action, and security into a decision-centric model of the enterprise, which can be jointly leveraged by both humans and agents. From data ingestion to application building to end user workflows, the platform's modular architecture enables human users and agents to query, reason, and act on data. [API Reference ↗](#)

operational foundation.

Send feedback



Example of an operational workflow

This section provides a notional example of how the Ontology can enable human-agent workflows in an organization.

Background

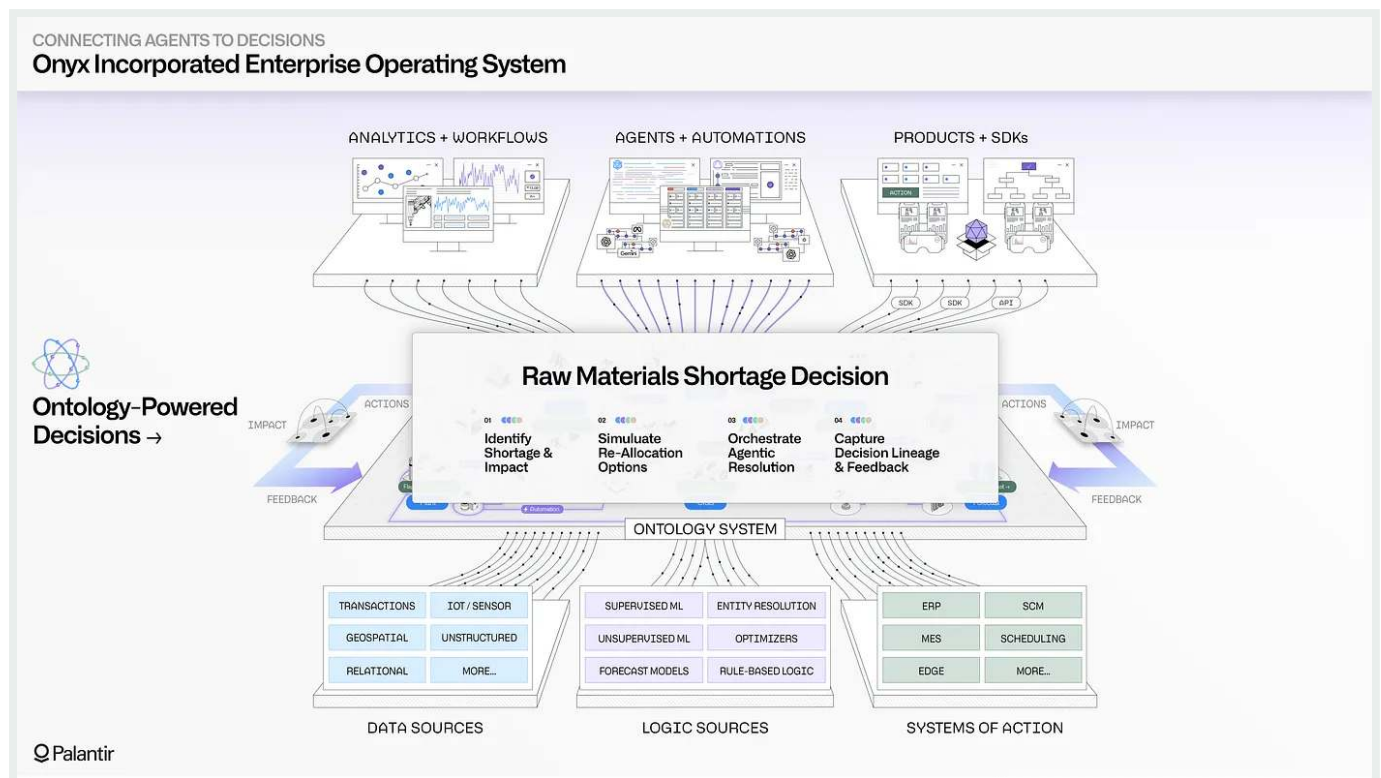
In this scenario, Onyx Incorporated, a fictional manufacturer of medical equipment, produces a range of finished goods, from syringes to surgical masks, each of which requires moving a precise set of materials through an associated manufacturing process. A diverse set of teams manages everything from supplier relations, to warehouse operations, to production of the finished goods, to distribution to end customers; decisions are interdependent, and constantly adapting to changing circumstances.

Imagine that Onyx is faced with an unexpected disruption with one of their major suppliers, where they raw materials needed to produce surgical masks. Given the tight production timelines across Onyx's manufacturing plants and the escalating demand from customers for surgical masks, this disruption could create serious issues with outstanding customer orders. To respond, Onyx's operational teams have decided to use

API Reference ↗

Send feedback

Palantir's [AI FDE](#) to connect a wide array of data sources, logic assets, and systems of action into their enterprise ontology.



Gaining visibility into the problem

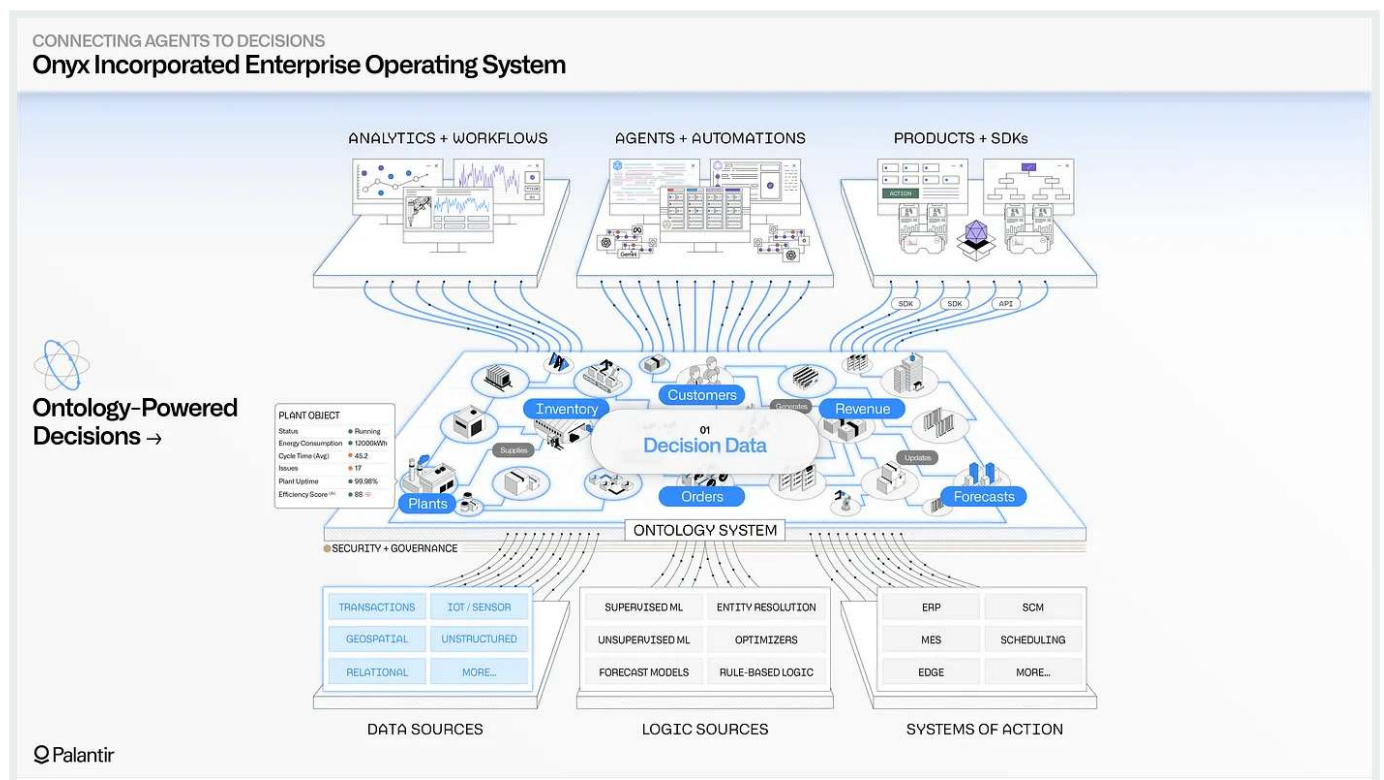
Onyx will start by assessing the immediate impact of the supplier shortage, and will then employ AI to assess possible reallocation strategies across production lines, before finally translating their decisions into a set of connected actions that will simultaneously update warehouse processes, production schedules, and fulfillment routes.

Onyx's ontology provides real-time, end-to-end visibility into the operations happening across each interdependent part of the business. This enables both leadership and on-the-ground teams to quickly understand the supplier disruption. Vital data systems related to supplier management, warehouse operations, production activity within plants, distribution center processing, and customer fulfillment are all synthesized into semantic objects and links, which reflect the language of the business. Using the Palantir platform, an operations leader can rapidly pinpoint the surgical mask production at risk due to the raw material shortage, and through the connections in their ontology, navigate to every outstanding

c [API Reference ↗](#) is now also at risk. The Ontology's granular security model ensures that more sensitive data elements (such as financial metrics) are automatically hidden by default, as the response widens to include more teams across the en

Send feedback

While operational users can easily navigate the Ontology through [Workshop](#)- and [SDK](#)-driven applications, the inclusion of agentic capabilities is a force multiplier for Onyx Incorporated. Agents leveraging both open-source and proprietary LLMs can navigate across supplier information, stock levels, real-time production metrics, shipping manifests, and customer feedback all contained within the organization's ontology. Importantly, all agentic activity is controlled with the same security policies that govern human usage, ensuring that Onyx engineers always have precise control over what the LLMs can query, recommend, and act upon. Each constructed and deployed agent can be treated like a new team member that is gradually granted a wider purview as Onyx team members gain confidence in its performance.



Building simulations and designing solutions

Situational awareness is only the tip of the ontological iceberg. Onyx needs to rapidly identify *solutions* to deal with the supplier disruption, and explore the tradeoffs inherent with each possible decision.

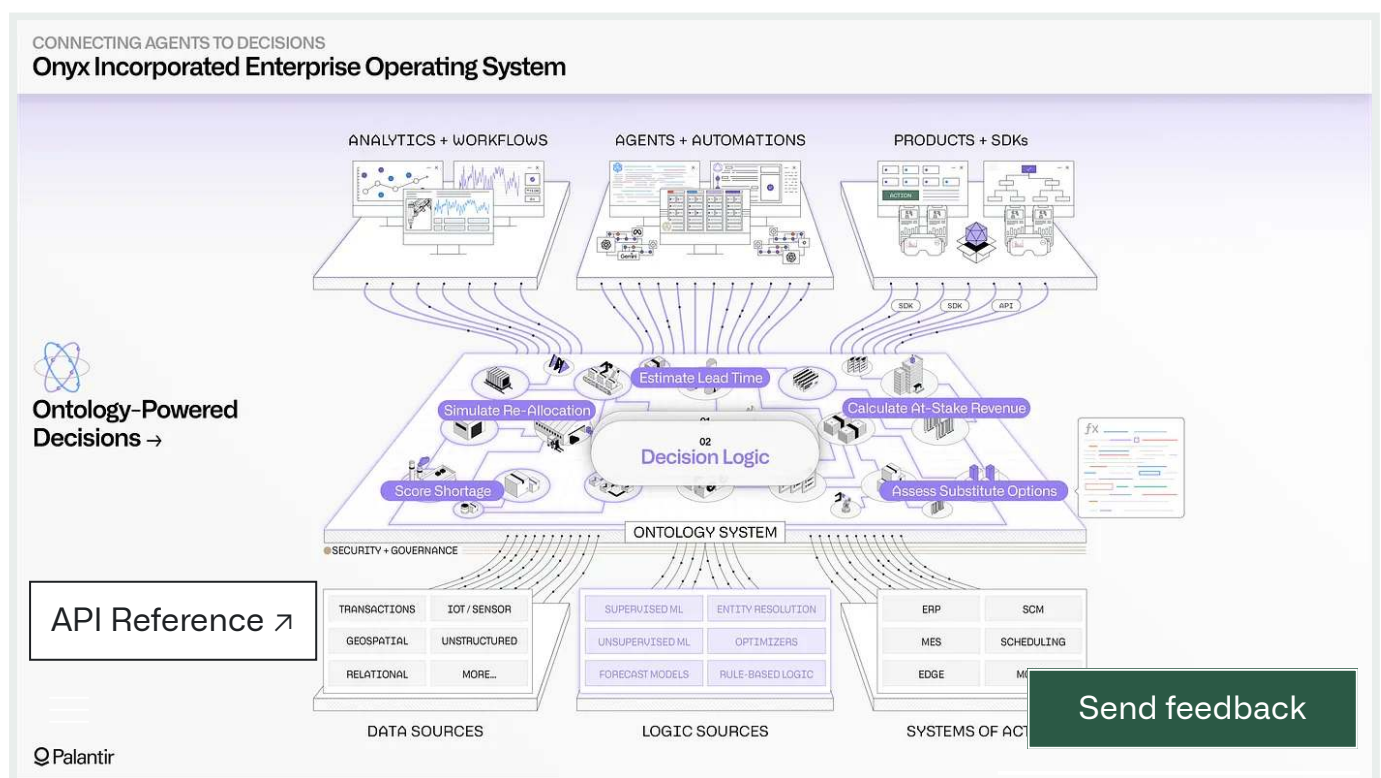
Since the diverse set of forecast models, allocation models, production optimizers, and [API Reference](#) have been connected into Onyx's ontology (alongside the aforementioned data sources), Onyx supply chain analysts can quickly run a battery of simulations that detail the consequences of the different possible material substitutions. The nature of the Ontology is key at this stage, since substituting raw materials will potentially

Send feedback

have downstream implications for the other products (like syringes and gloves) being produced from the same materials. As the simulations are run, the simulated outputs are staged as ontology scenarios, which safely package the proposed changes into a sandboxed subset of the Ontology — enabling teams to safely explore and analyze the implications of the decision before committing to it.

Even more valuable for the Onyx team is that fleets of agents can securely leverage the full range of logic assets and the same scenarios framework. The Ontology enables agents to go beyond the data-centric limitations of retrieval-augmented generation, and instead interface with the interconnected data, logic, and action primitives in the Ontology through an extensible tools paradigm. As Onyx's analytics and data science teams create new machine learning models in their cloud workbenches, tune optimization algorithms within enterprise systems, and fine-tune LLMs using Palantir's open model building framework, the Ontology can securely surface these logic assets as AI-ready tools.

In this case, Onyx has created a tuned agent, "Disruption Bot", that can use a set of Ontology-driven tools to scan across the full range of enterprise data sources, the after-action reports on prior courses of action taken in similar situations, and the potentially applicable material reallocation models. Thanks to the rich, dense context provided through the Ontology, Disruption Bot is able to surface a novel reallocation plan, which uses a newer model that the supply chain analysts had not yet considered. With the consequences of the plan safely staged in a scenario, the agent's proposed decision is handed off to a human analyst for final review.



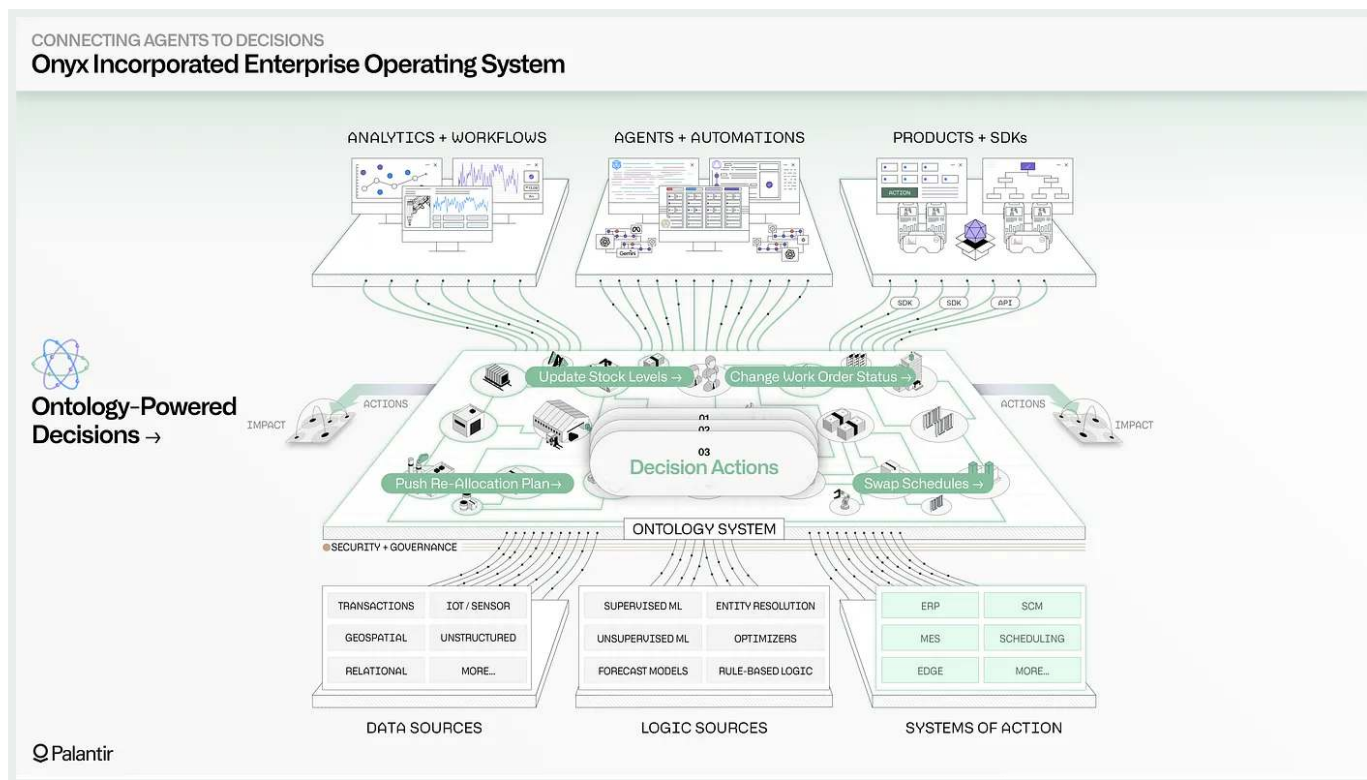
Executing decisions and taking action

With a viable plan to address the material shortage identified, Onyx needs to rapidly and safely push the decision to the operational systems that run the constituent processes. Given that the enterprise has grown through acquisition, and contains a diverse and delicate mix of critical operational systems, the Onyx IT team is vigilant about which processes can write back to these systems, and under which conditions. Fortunately, the Ontology applies the same rigorous control and validation to actions as it does to data and logic; enabling fine-grain control over who can invoke a given action, test-driven frameworks for publishing changes, the ability to stage and review changes in batch, and detailed logging for every event. In this case, the execution of the material reallocation plan automatically orchestrates a set of writeback routines, each tuned for the receiving system: the warehouse management system receives an API-driven update; the three ERP systems each receive updates via native Ontology-driven connectors, which abide by the safeguards in each system; and the production planning system receives a consolidated flat file, which it ingests asynchronously. As actions are executed, the Onyx IT team can monitor system responses and can always audit past activity.

The Ontology provides the guardrails needed for AI to safely take action within permitted boundaries. Alongside data and logic, actions can be automatically surfaced as tools for all types of agents. The scope of an action can be limited to simply reflecting a given change (such as an edit to an object or the creation of a new object) in the Ontology itself; or can write back to single or multiple systems. In this case, Onyx has granted Disruption Bot and a few other production AI agents access to a small set of actions. In the default case, these actions (like changing the status of a work order or pushing a reallocation plan) can only be staged by the AI, before being handed off to a human for final review. However, with the granular logging and operational instrumentation provided by the Ontology (and the wider Palantir platform), Onyx is able to carefully choose whether any trusted, well-tested AI processes can automatically close the action loop without human review. As conditions evolve, the latitude given to AI can be expanded or contracted, with any change instantly reflected across all Ontology-driven workflows.

[API Reference ↗](#)

[Send feedback](#)

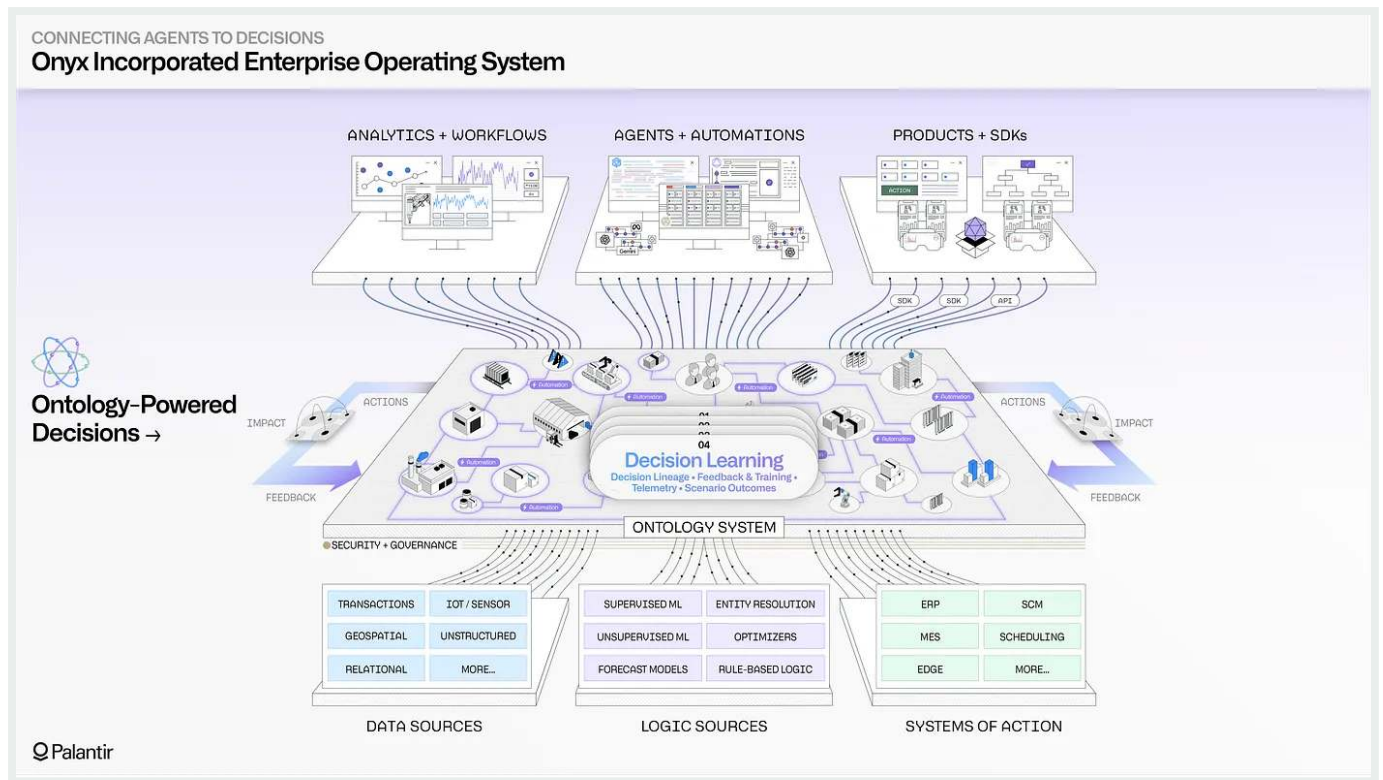


Learning from decisions

What comes after the immediate crisis is past? With data, logic, action, and security all connected into Onyx's ontology, the organization can conduct powerful decision-centric learning. The human-agent teaming that produced a specific solution to the material shortage also revealed generalizable workflows, which the organization will want to memorialize and surface in the future. Every data element, logic asset, and action assessed is captured in an end-to-end decision lineage, which serves as rich, contextual fuel for optimizing the performance of AI. The aggregate decisions made by thousands of users and agents throughout Ontology can be securely leveraged as training data when fine-tuning models, and can be distilled into targeted principles that are called upon during agent prompting. The tribal knowledge that has been traditionally trapped in the seams of workflows can be illuminated by AI to improve the operation of the entire enterprise.

[API Reference ↗](#)

[Send feedback](#)



Onward with the Ontology

The Ontology allows organizations to implement and scale human-agent operations, as well as precisely control how and when agent-driven recommendations, augmentations, and automations can be used in frontline contexts. This is possible because the Ontology is decision-centric, not simply data-centric, bringing together the constituent elements of decision-making — data, logic, action, and security — in a single software system.

With the Ontology, new data can be rapidly integrated into a full-fidelity semantic representation; new algorithms and business logic can be seamlessly surfaced for both human and AI users; and robust action integration can be achieved through real-time connections with the full range of operational systems. Each organization's ontology is a real-time representation of the changing conditions, goals, and decisions being made across teams, which ensures that AI usage remains anchored in the reality of the enterprise.

To learn more, explore our documentation on the Ontology's underlying decision-centric [architecture](#); the extensibility provided through the [Ontology SDK](#); and the [Global Branching framework that allows](#) for safe and zero-downtime evolution of the Ontology.

API Reference ↗

PREVIOUS
← Overview

Send feedback
Models in the Ontology →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Models in the Ontology

Models in the Ontology

Organizations are looking to leverage artificial intelligence (AI) and machine learning (ML) to accelerate and improve decision-making. But the reality of operationalizing AI/ML is complex, and the typical return on investment rarely lives up to expectations.

Foundry provides the key capabilities necessary to bridge this gap: a trustworthy data foundation, tools for evaluating and comparing models against organizational objectives, and functionality for deploying models into user-facing operational workflows. This page focuses on the last step: deploying an evaluated model into production.

End-to-end workflow

At a high level, these are the end-to-end steps required to operationalize AI/ML in Foundry for live inference with the Ontology:

1. [Create a model](#) in Foundry.
2. Configure a [direct model deployment](#).
3. [Publish a simple wrapper function for your model](#) and [optionally call it from another function](#) to orchestrate complex logic around your model.
4. Use that function for live inference in [Workshop](#), [Vertex](#) and other end-user facing applications.

Ontology Objects can also be backed with datasets that leverage a model for batch inference - [learn how to use a model in Code Repositories](#).

API Reference ↗

Send feedback

Just like mapping datasets to Ontology concepts provides [benefits](#) for workflow development and decision-making, mapping models to the Ontology provides a number of benefits:

- **Interpretability.** Because all modeling results are defined in terms of real-world concepts (properties of an object type), end users do not need to understand machine learning in order to use modeling results. Instead, users simply interact with simple concepts such as a *forecast*, *estimate*, or *classification*.
- **Economies of scale.** Instead of each modeling project being a bespoke effort created in service of a specific use case, modeling efforts can build on each other over time. For example, a forecast produced for one use case can immediately be used for subsequent use cases as well, reducing duplicated effort and providing end-user value more quickly over time.
- **Connectivity at scale.** By incorporating ML models, the Ontology becomes a single source of truth for the organization, not just in terms of data, but also in terms of *logic*. Models encode the organization's expectations for how things may change in the future. In this way, the Ontology becomes a "digital twin" for the entire enterprise, which unlocks the ability to simulate changes across the organization in ways that would never be possible with a wide array of disparate modeling efforts.

PREVIOUS
← [Why create an Ontology?](#)

NEXT
[Core concepts](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

—[Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Core concepts

Core concepts

This page describes major concepts related to the Ontology in Foundry.

Ontology

An Ontology is a categorization of the world. In Foundry, the Ontology is the digital twin of an organization, integrating the organization's digital assets (datasets and models) into a coherent whole. The Foundry Ontology creates a complete picture of an organization's world by mapping datasets and models to object types, properties, link types, and action types.

- An [object type](#) defines an entity or event in an organization.
- A [property](#) defines the object type's characteristics.
- A [link type](#) defines the relationship between two object types.
- An [action type](#) defines how an object type can be modified.

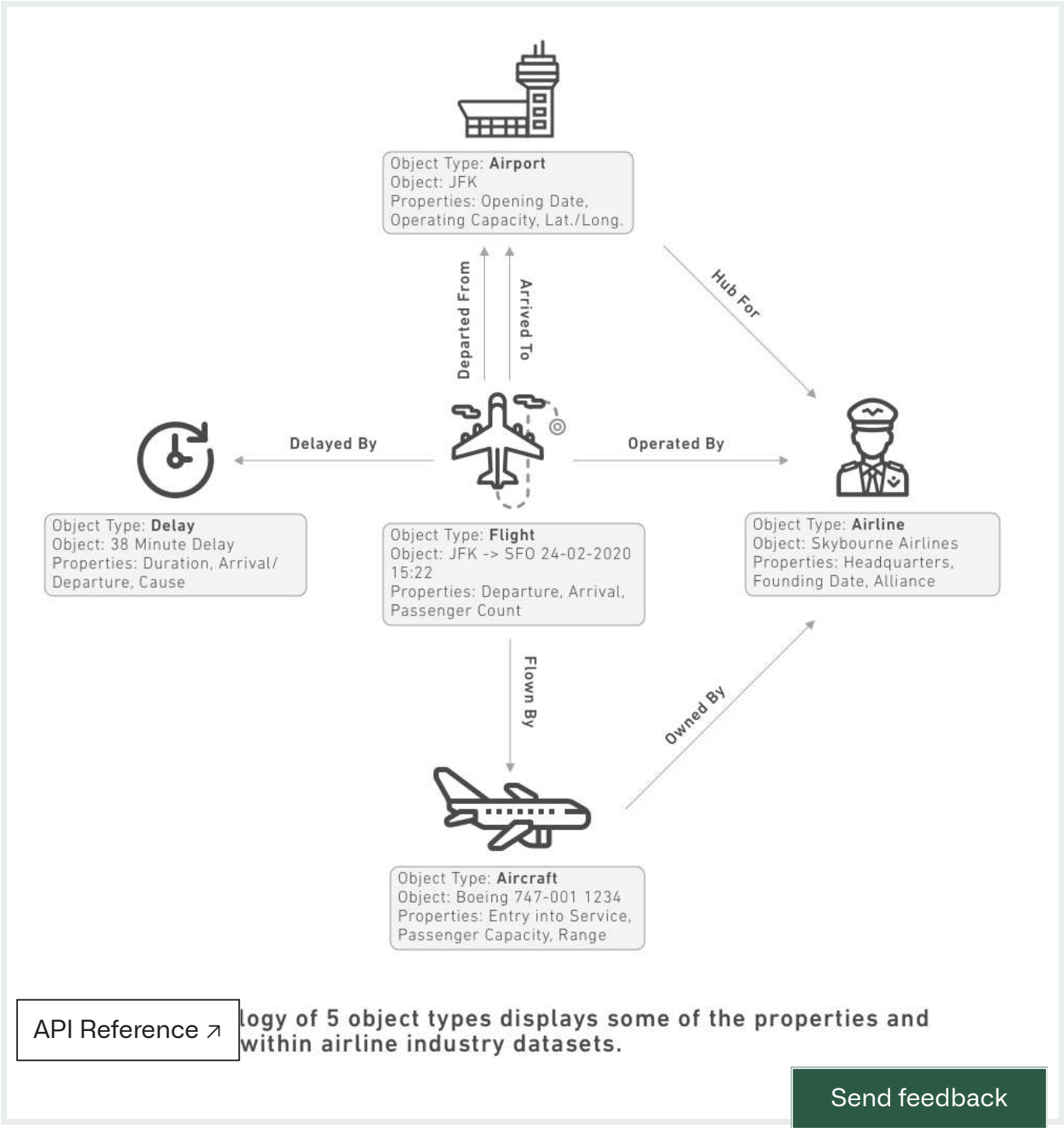
The concepts that comprise the Ontology have parallels in the structure of a dataset. You can think of each object type as analogous to a dataset; an object is an instance of an object type, just as a row is one entry in a dataset. The columns in a dataset are analogous to properties of an object, as they provide additional information for a given row. The value in a dataset field (like a cell in a spreadsheet) is akin to the property value of an object. And just as datasets can be joined together in various ways, objects can have links between them based on property values. The table below summarizes this comparison:

Datasets	Ontology
API Reference ↗	Object type
Row	Object
Property	Property

Send feedback

Datasets	Ontology
Field	Property value
Join	Link type

The diagram below demonstrates how these concepts can come together to create an Ontology. The content below continues to define the different components of the Ontology in more depth.



Object type

An **object type** is the schema definition of a real-world entity or event. An **object** refers to a single instance of an object type; an object corresponds to a single real-world entity or event. An **object set** refers to a collection of multiple object instances; that is, an object set represents a group of real-world entities or events.

[Learn more about object types.](#)

Property

A **property** of an object type is the schema definition of a characteristic of a real-world entity or event. A **property value** refers to the value of a property on an object, or a single instance of that real world entity or event.

[Learn more about properties.](#)

Shared property

A **shared property** is a property that can be used on multiple object types in your ontology. Shared properties allow for consistent data modeling across object types and centralized management of property metadata.

[Learn more about shared properties.](#)

Link type

A **link type** is the schema definition of a relationship between two object types. A **link** refers to a single instance of that relationship between two objects.

[Learn more about link types.](#)

API Reference ↗

Action type

Send feedback

An **action type** is the schema definition of a set of changes or edits to objects, property values, and links that a user can take at once. It also includes the side effect behaviors that occur with action submission. Once an action type is configured in the Ontology, end users can make changes to objects by applying actions.

[Learn more about action types.](#)

Roles

Roles are the central permissioning model in the Ontology. Similar to roles in the Foundry filesystem, Ontology roles grant access to ontological resources. Roles can be granted on the Ontology level or the individual resource level.

Learn more about [Ontology roles](#) and how they are used for object types, link types, and action types.

Functions

A **function** is a piece of code-based logic that takes in input parameters and returns an output. Functions are natively integrated with the Ontology: they can take objects and object sets as input, read property values of objects, and be used across action types and applications that build on the Ontology.

[Learn more about Functions in general](#), or [learn more about Ontology-based Functions](#).

Interfaces

An **interface** is an Ontology type that describes the shape of an object type and its capabilities. Interfaces provide object type polymorphism, allowing for consistent modeling of and interaction with object types that share a common shape.

Learn more about [Interfaces](#).
API Reference ↗

Object Views

Send feedback

Object Views are a central hub for all information and workflows related to a particular object. This includes key information about an object, any linked objects, and related metrics, as well as analyses, dashboards, and applications related to the object.

[Learn more about Object Views.](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

API Reference ↗

Send feedback

Ontology building / Ontology-aware applications

Ontology-aware applications

Foundry contains a number of applications developed to operate natively on top of the Ontology. Together, these object-aware applications deliver a powerful analytical and operational platform that supports a range of use cases and user profiles.

To learn more about why setting up an Ontology and using object-aware applications is valuable, see [this page](#).

This page provides a reference to the available applications, and explains when you should use each:

- [Application reference](#)
- [Application comparison](#)

Application reference

Object Views

Object Views are a central hub for all information and workflows related to a particular object. This includes key "biographical data" about an object, any linked objects, key related metrics, and links to (or embedding of) key analyses, dashboards, and applications related to the object.

For example, the `Airport` object type object view might provide the following information

for `API Reference` object:

Biographical data such as

country

,

city

,

longitude

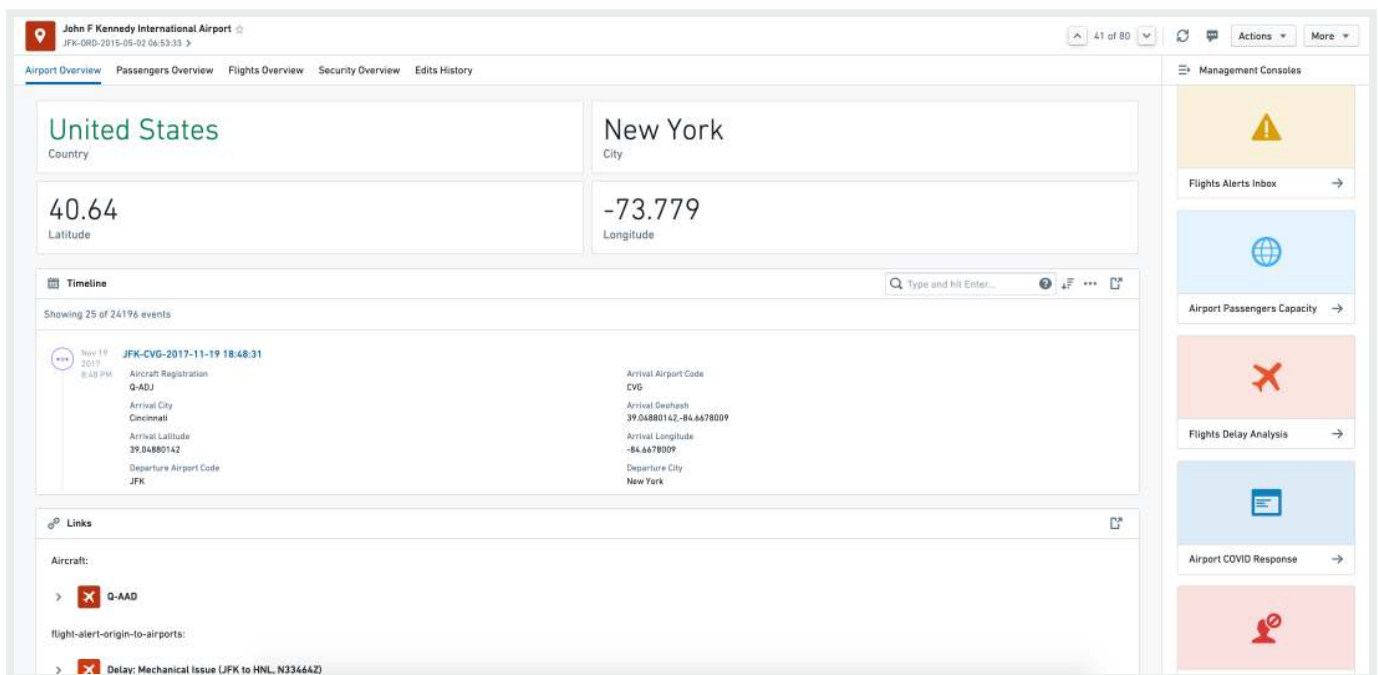
,

latitude

Send feedback

- 360-degree view of all `Aircraft` objects and `Flight` objects linked to the `Airport`

- Embedded Airport Covid Response workflow
- Link to a Root-Cause Analysis of a Flight delay event related to the Airport



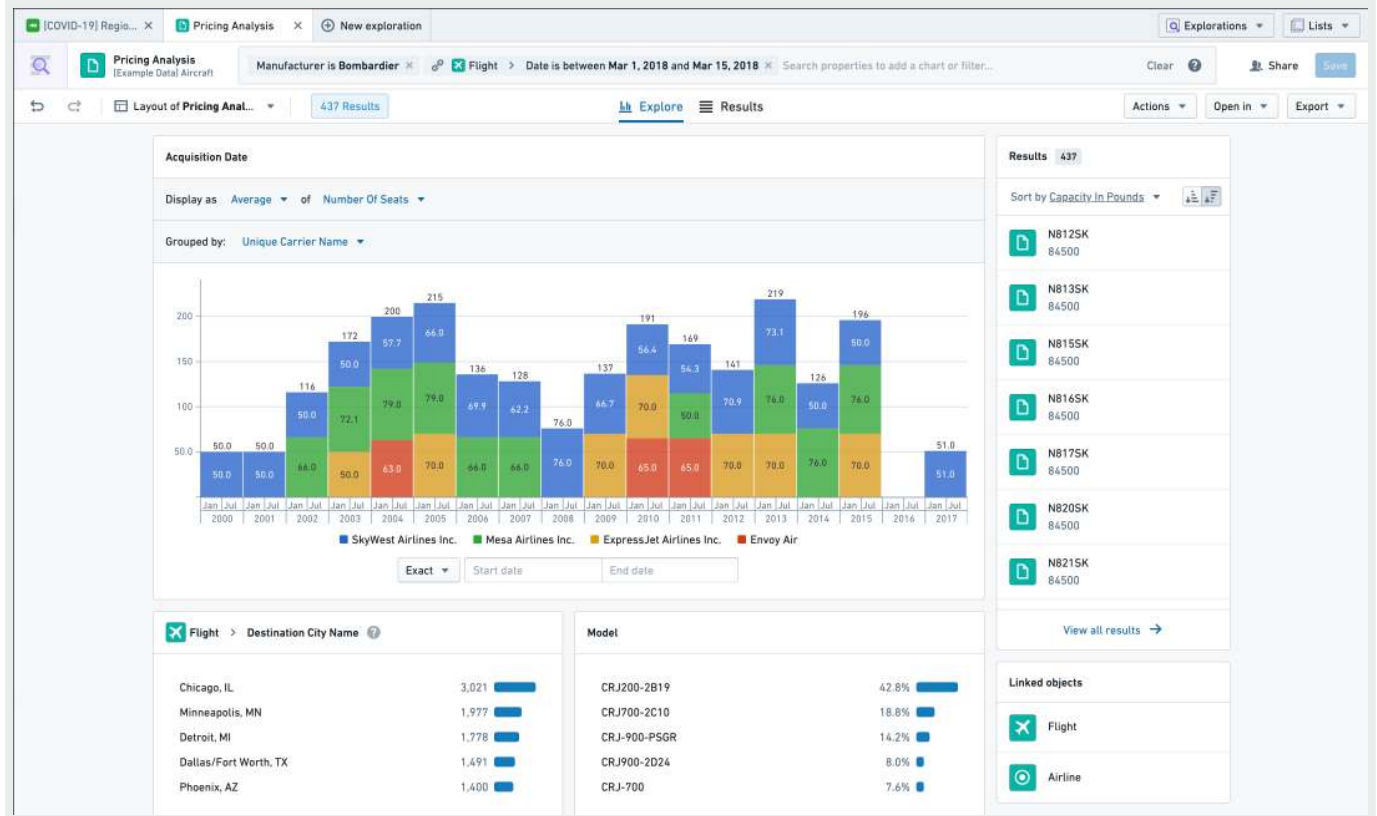
Object Explorer

Object Explorer is a search and analysis tool for answering questions about anything in the Ontology layer. Users can visually compose search queries ranging from simple filters to Search Arounds to find objects of interest. From there, they can explore the resulting object sets using the exploration view or view them as a table of results. Additionally, users can compare and contrast object sets and take bulk Actions (for example, writeback) on the object set. Then, users can export the object sets or open them in compatible applications, such as Workshop.

The exploration view is a set of preset and configurable visualizations (such as charts or maps) that the user can further leverage to drill-down into specific subsets of objects. Object Explorer requires no pre-configuration and is geared towards less technical users.

[API Reference ↗](#)

[Send feedback](#)

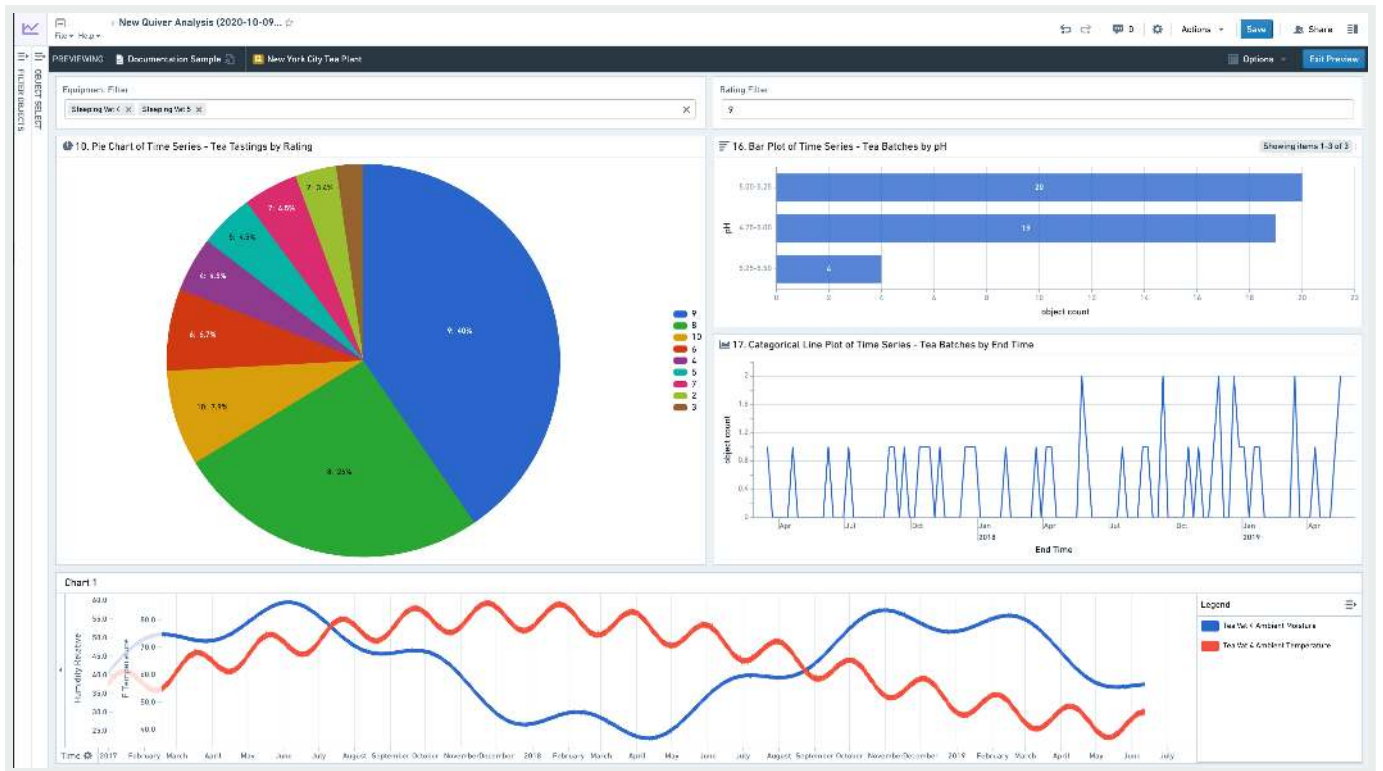


Quiver

Quiver enables advanced analytical workflows in the Ontology layer through a visual point-and-click interface and a powerful charting library. Quiver can be used to support anything from simple linear drill-down analyses to highly-branched and complex analyses with aggregations and statistical functions. Quiver also supports native time series analysis. Quiver analyses can be templated into read-only dashboards for broader consumption.

API Reference ↗

Send feedback



Workshop

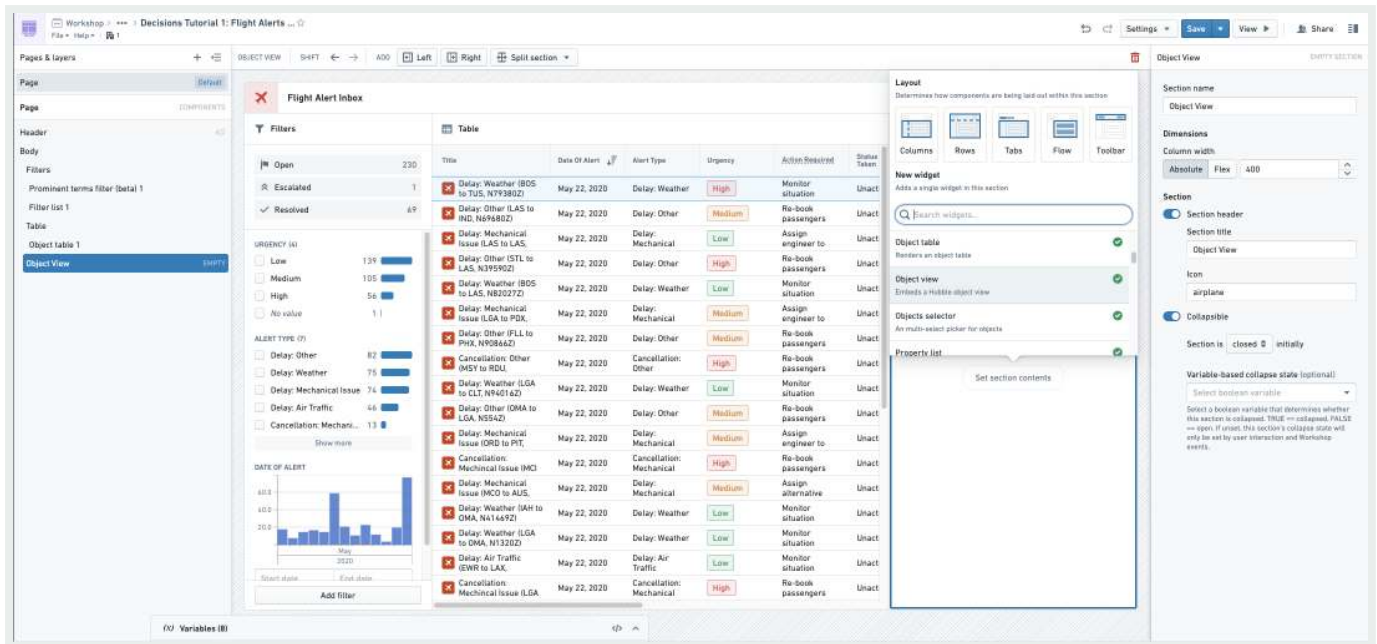
Workshop enables point-and-click code-less application-building natively on the Ontology layer. Applications built in Workshop are more dynamic and interactive than typical dashboards created in other point-and-click tools.

By leveraging high-quality **Layouts** and an easy-to-use but sophisticated **Events system**, Workshop applications aim to be as user-friendly and high-quality as custom React applications.

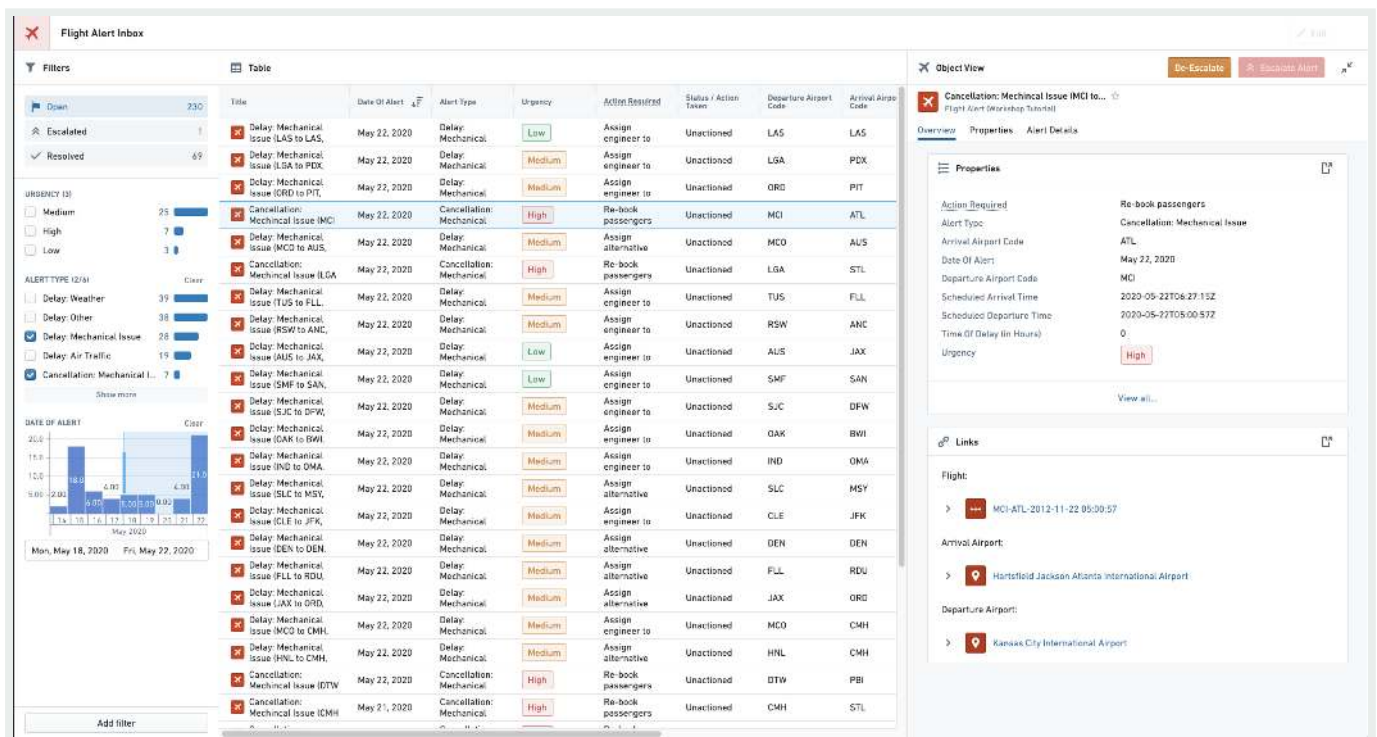
Workshop Editor View

API Reference ↗

Send feedback



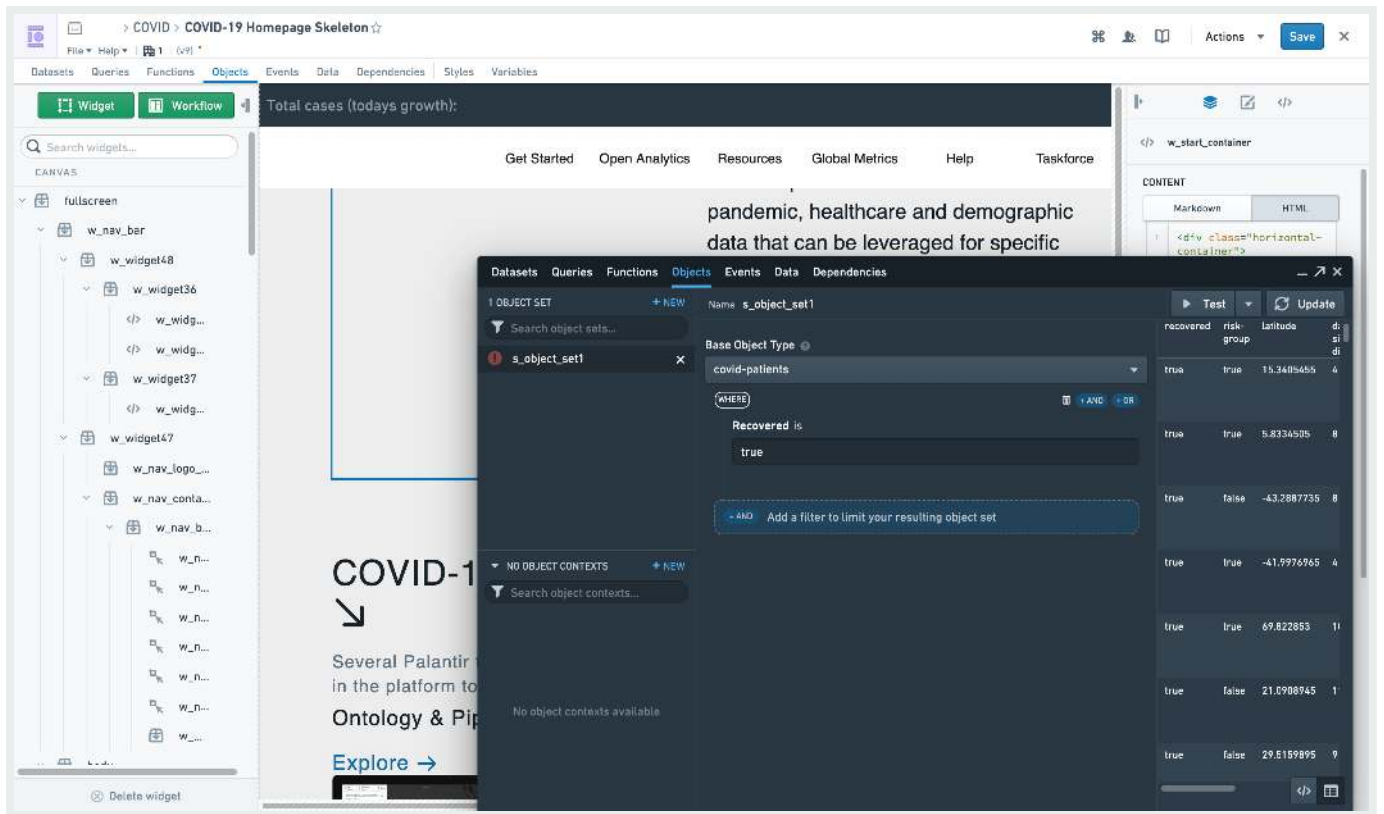
Final Workshop Module



Slate

Slate is a flexible application builder for Foundry that requires more technical configuration than Workshop. Slate applications interact with the Ontology layer, but can also interact with Foundry datasets. Slate enables significant visual customization based on web development paradigms and has a wide range of available features. Send feedback

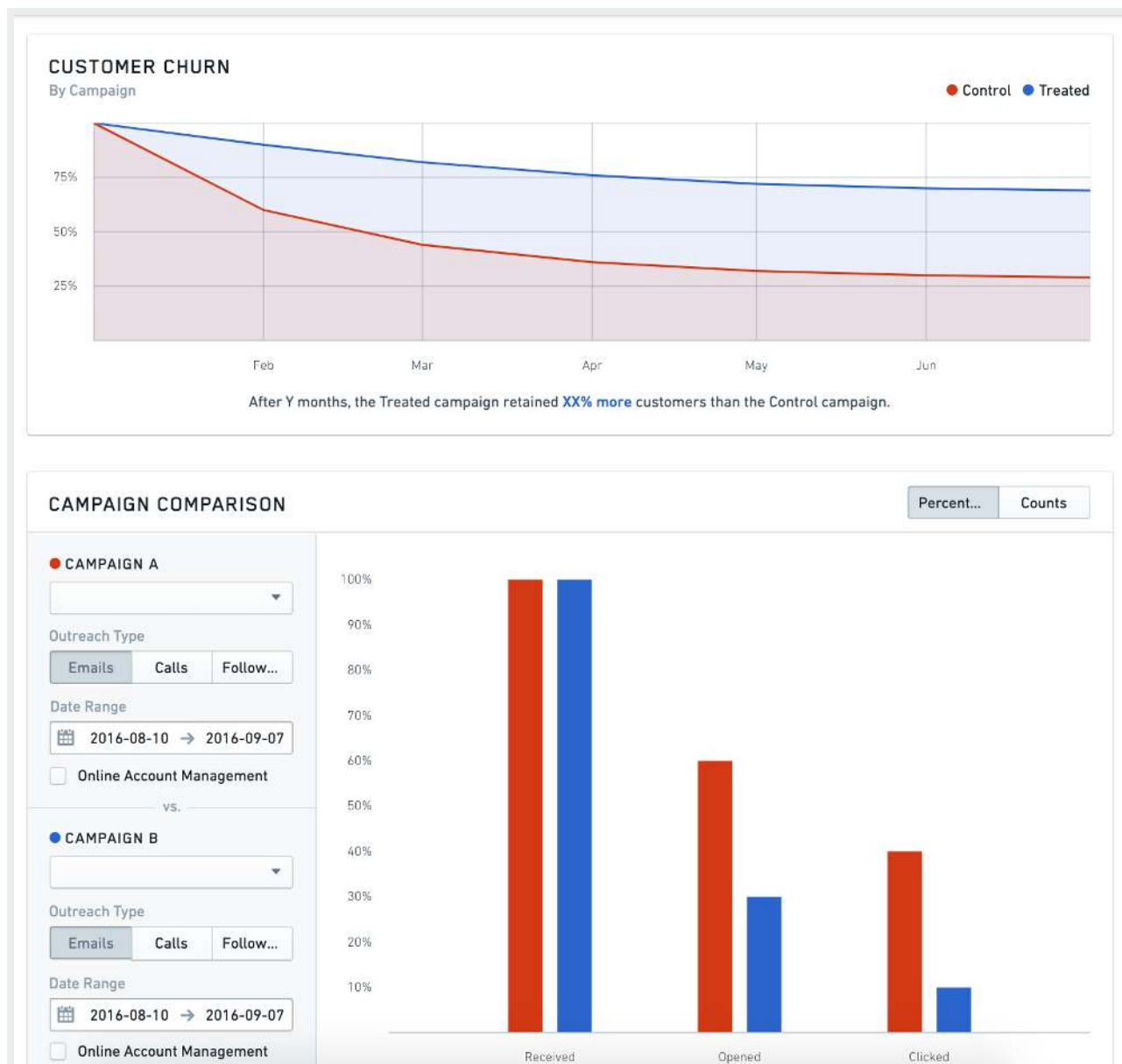
Slate Editor View



Slate Application View

[API Reference ↗](#)

[Send feedback](#)

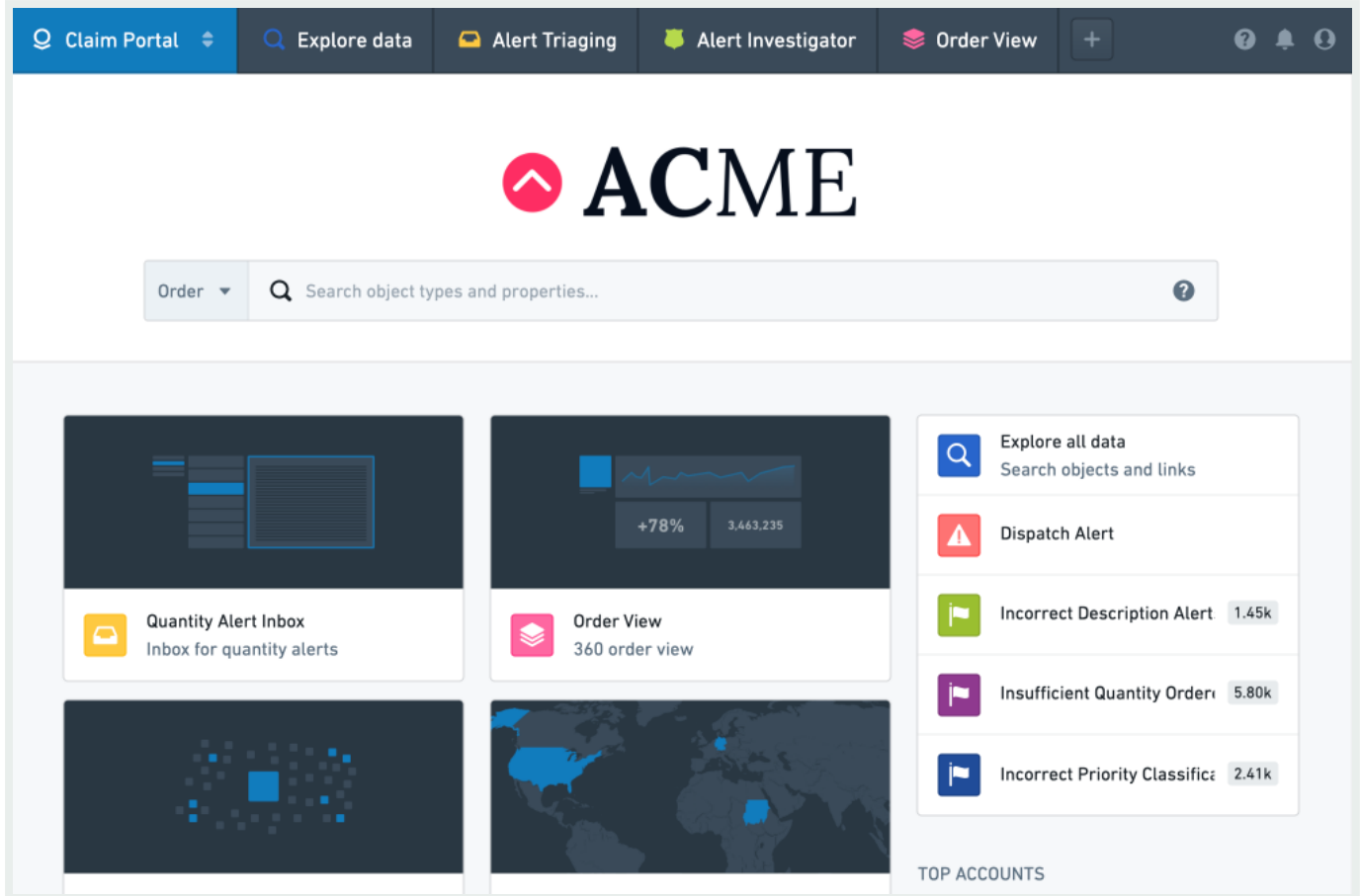


Carbon

Carbon enables combining multiple resources or applications in Foundry to create highly curated *workspaces* for operational users. By allowing you to combine analytical results such as dashboards, applications built in Workshop or Slate, and out-of-the-box capabilities such as Object Views and Object Explorer, Carbon enables workflow builders to perform the "last mile" of customization to create a highly tailored and usable experience for end users.

[API Reference ↗](#)

[Send feedback](#)

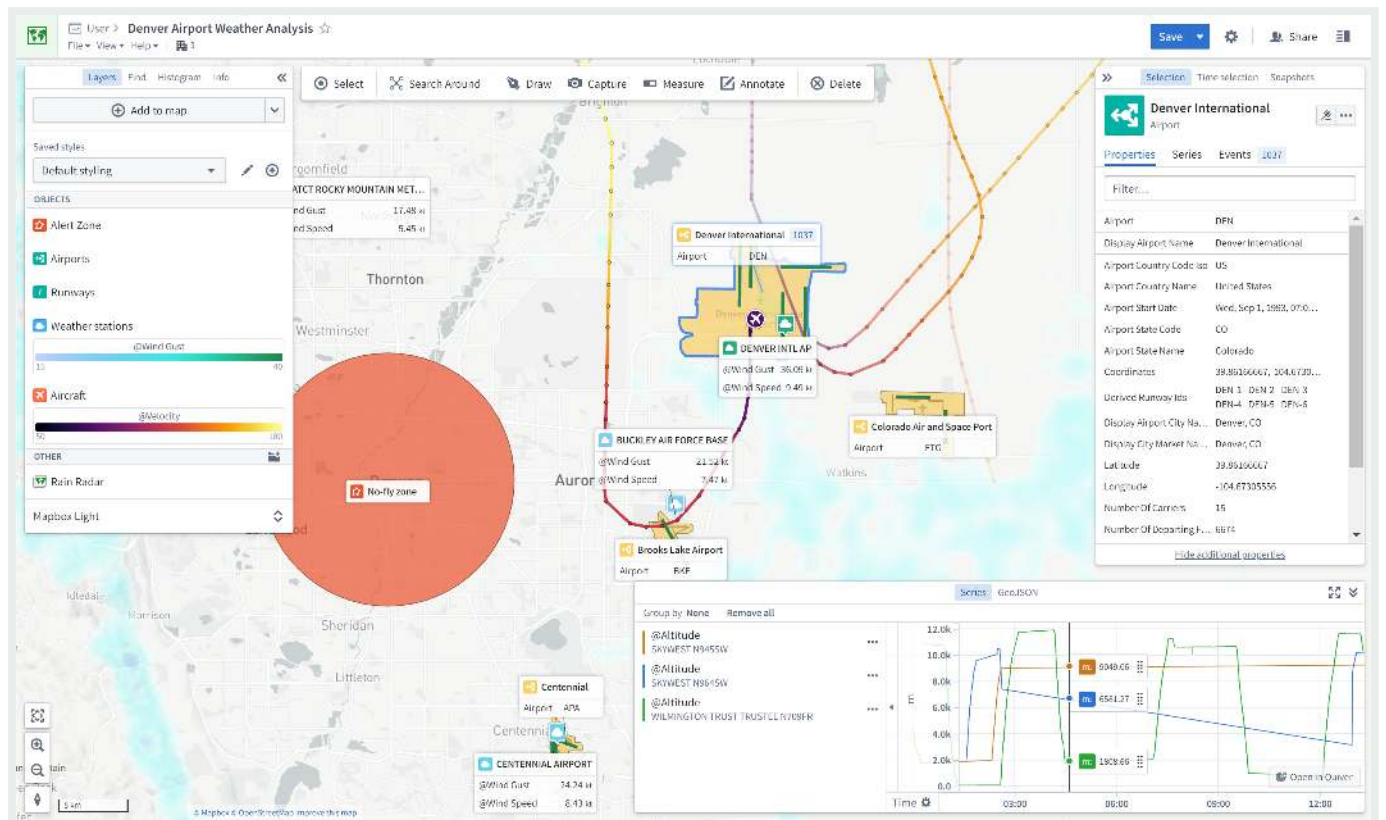


Map

The **Map** application allows you to bring together and analyze objects and other data in a geospatial context.

[API Reference ↗](#)

[Send feedback](#)



Application comparison

Each object-aware application varies on a few dimensions. Three particularly important dimensions are:

- the [primary use case\(s\)](#),
- the [workflow style](#), and
- the [configuration model](#) of the application.

Foundry Application	Primary use case	Workflow Style	Configuration Model	Objects or Datasets
Object Views	Discovery	Workflow-specific	Walk-up usable	Objects
Object Explorer	Discovery & Analysis	Exploratory	Walk-up usable	Objects
Quiver	Analysis & Dashboards	Exploratory (for Analytical mode);	Walk-up usable (for Analytical mode); customizable	Objects

[API Reference](#)

[Send feedback](#)

Foundry Application	<u>Primary use case</u>	<u>Workflow Style</u>	<u>Configuration Model</u>	Objects or Datasets
		workflow- specific (for Dashboard mode)	(for Dashboard mode)	
<u>Workshop</u>	Applications & Dashboards	Workflow- specific	Customizable	Objects
<u>Slate</u>	Applications & Dashboards (complex)	Workflow- specific	Customizable	Objects (recommended) and Datasets
<u>Map</u>	Geospatial	Exploratory or Workflow- specific	Walk-up usable	Objects

Use cases

The main use cases supported by object-aware applications are **Discovery**, **Analysis**, **Dashboards**, and **Applications**.

- **Discovery** enables users to find the right information or workflow. Discovery is primarily enabled through two core features: curated content hubs and search. Curated content hubs (sometimes called landing pages or "360 views") range from a comprehensive standard view for any user to targeted views for a specific set of users or use case. Search functionality supports discovery through free-text search of key words as well as more iterative search through link traversals or drill-downs.
- **Analysis** enables users to answer a broad range of questions. These range from the simple (*what is the average customer retention for a given product?*) to the very complex (*how do three different customer cohorts compare in terms of retention and overall revenue over time, across all products and for each individual product?*). The analytical path is exploratory, meaning that it is defined by the end user themselves and is often API Reference ↗; the initial questions are answered, new questions are developed and incorporated into the analysis path.

Send feedback

- **Dashboards** are sets of pre-configured visualizations consumed primarily in a read-only fashion by a broader set of consumer users. Dashboards are often used to turn meaningful Analyses into recurring reporting or operational monitoring. Dashboards are characterized by a large number of charts and other visualizations, but are not as customizable or interactive as *Applications* (see below). Dashboards are often parameterized such that users can filter down the visualizations to different subsets of data.
- **Applications** are interactive custom operational interfaces intended for a specific user group to solve a specific problem. Applications are often more complex than dashboards and are oriented at enabling users to follow a specific and on-rails workflow. While the application may contain some curated analytical content (e.g. statistics, charts, graphs, etc.) required to perform a decision, it also typically has several workflow elements and often captures user input (for example, writeback).

Workflow style

Object-aware applications are optimized for primary workflow styles.

- **Exploratory applications** do not need to be pre-configured by a "builder" user and are used out-of-the-box by end users as soon as data has been modeled into the Ontology. In exploratory applications, the end-user defines their analytical path, and can answer a wide range of questions that are not pre-determined. Exploratory applications typically contain a set of search, visualization, and transformation features to enable this. Object-aware applications that are primarily exploratory include Object Explorer and Quiver.
- **Workflow-specific applications** require pre-configuration by a "builder" user before an end user can actually use them. This is typical of Dashboards or Applications that have two primary user groups: (1) the builder group configuring the specific dashboard or application and (2) the downstream end users for whom the applications are built. Workshop and Slate modules both need to be pre-configured by a "Builder" user in edit mode.

Certain applications like Quiver accommodate both workflow styles because while their primary mode is exploratory, the outputs can be configured into a more broadly consumed workflow-specific artifact. While Quiver Analyses are highly exploratory, they can be

API Reference ↗ Dashboards that are pre-configured analytical views accessible to a broader audience.

Send feedback

Configuration model

The configuration model describes the extent to which the user interface must be configured before it can be leveraged by an end user.

- **Walk-up usable** applications can be employed effectively and immediately by users, with little to no configuration requirement or maintenance burden. For example, Object Explorer has minimal-to-no configuration requirement, making it immediately usable for end users once an Ontology is defined.
- **Customizable** applications require an upfront investment (often by a separate "builder" user) to implement an interface that solves a particular problem for an end user. It also implies a higher ongoing maintenance cost. However, the resulting application is typically a fit-for-purpose interface that exactly meets the need of the specific workflow. Workshop and Slate are examples of this type of customization.

PREVIOUS

← [Core concepts](#)

NEXT

[Define Ontologies / Ontologies / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology design: Best practices

A well-designed Ontology creates a unified, intuitive representation of your organization that enables seamless data integration, cross-functional collaboration, and powerful analytics. This section provides both a quick-reference set of design guidelines and a deeper treatment of the core design principles that underpin them.

Design guidelines

The following guidelines are a practical checklist for Ontology design. Where applicable, each one is grounded in a [core design principle](#), [anti-pattern](#), or [structural recommendation](#).

1. **Model reality, not systems:** Object types should represent real-world entities, not individual source system or department representations.
 - **Design principle:** [Domain-driven design](#)
2. **Curate intentionally:** Every property should have clear business or technical value.
 - **Structural recommendation:** [Normalization and derived properties](#)
3. **Collaborate across teams:** Ontology design should involve stakeholders from multiple departments or teams. Siloed teams are a leading cause of duplication.
 - **Design principle:** [Don't repeat yourself](#)
4. **Keep object types focused:** Each object type should represent one distinct entity.
 - **Design principle:** [Domain-driven design](#)
5. **Choose the right tool:** Use action types for human or agentic decisions and pipelines for automated transformations.
 - **Anti-pattern:** [The Golden Hammer](#)

API Reference ↗

Principle of abstraction: When entities share common characteristics, model the interfaces rather than creating wide, sparse object types.

☰

🔍

Design principle: [Composition over deep hierarchies](#)

Send feedback

7. **Document your decisions:** Document object types, properties, and links in Ontology Manager.

Core design principles

These four principles are derived from extensive field experience across government and commercial implementations. They are presented in priority order. In cases of conflict, higher-priority principles take precedence.

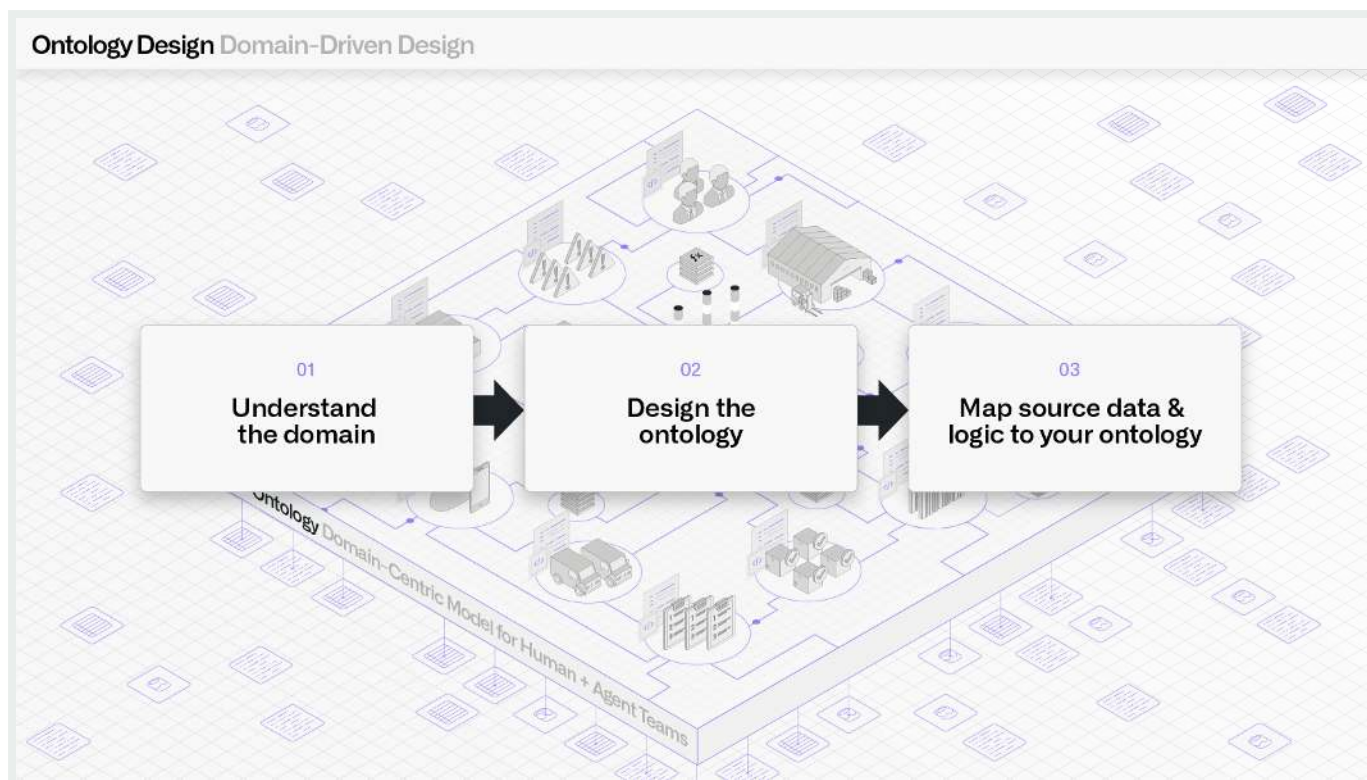
Priority	Principle	Core idea
1	Domain-driven design	Model the real world, not the source data.
2	Don't repeat yourself	If you built the same thing three times, refactor.
3	Open for extension, closed for modification	Protect core models. Enable builders to extend them.
4	Composition over deep hierarchies	Favor multiple inheritance via interfaces. Keep things pluggable.

Practical considerations should always factor into any decision. Review the section on [pragmatism and tradeoffs](#) for more information.

1. Domain-driven design

[API Reference ↗](#)

[Send feedback](#)



The Ontology models the real world, not the source data.

Objects should represent semantically meaningful real-world concepts (such as a `Patient`, a `WorkOrder`, or a `Vessel`) not database tables, API responses, or spreadsheet tabs. Links should represent real relationships ("this patient visited this facility"), not join keys or foreign key artifacts.

When asked to "ontologize a dataset", resist the urge to map columns 1:1 to properties and consider the work complete. The [Kitchen Sink](#) anti-pattern produces an Ontology that mirrors source system schema quirks rather than useful semantics. A well-designed Ontology should feel intuitive to use; a user, or an AI agent, should be able to navigate it without friction, because the structure matches how they already think about their domain.

Anti-patterns

- Object types mirror source system tables rather than domain entities
- Properties are mapped 1:1 from source columns without curation
- API Reference ↗ in source system conventions (`dtLastInspMod`) rather than business language (`lastInspectionDate`)

Send feedback

- The object model was designed by looking at the data rather than understanding the domain
- A single source row containing multiple entities is modeled as a single object type

Example

A CSV with columns `order_id`, `customer_name`, `customer_email`, `product_sku`, and `quantity` describes at least three real-world entities, not one:

x Avoid	✓ Prefer
OrderData - order_id - customer_name - customer_email - product_sku - quantity (One object type mirroring the CSV)	Order - order_id - quantity - Links to → Customer, Product Customer - name - email Product - sku (Three object types modeling the domain)

Impact of anti-patterns

Problem	Impact
Unintuitive model	Users and AI agents cannot navigate the Ontology naturally because the structure does not match how they think about the domain.
Fragile coupling API Reference ↗	Schema changes in source systems break Ontology consumers because the Ontology mirrors source structure rather than abstracting over it.
Missed relationships	Entities embedded as columns (like <code>customer_email</code>) cannot be linked, searched, or reasoned about independently.

Send feedback

Problem	Impact
Poor reuse	Object types shaped by one system's schema are difficult for other teams or use cases to adopt.

Best practices

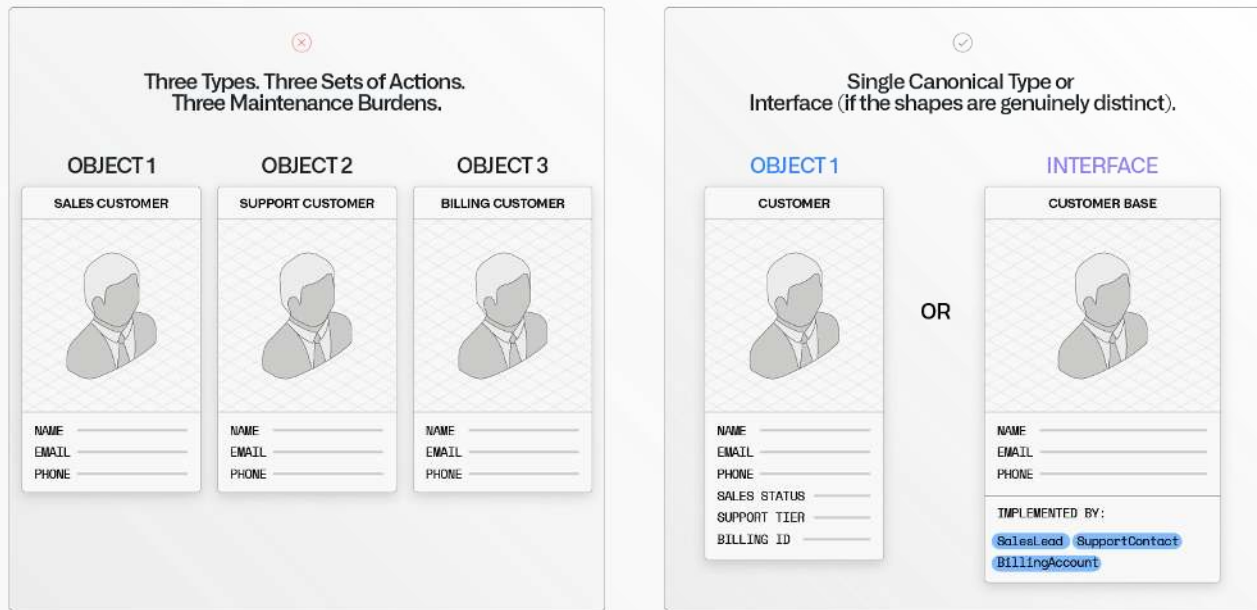
- 1. Identify real-world entities before looking at source schemas:** Work with domain stakeholders to define what concepts matter. A single dataset often describes multiple entities.
- 2. Separate identity from observation:** If a row represents a measurement or event about an entity, the entity and the observation are likely different object types.
- 3. Name things for humans:** API names should be intuitive and self-documenting. Prefer `person.children` over `person.linkedChildPersonObjects`. Prefer `equipment.lastInspectionDate` over `equipment.dtLastInspMod`.
- 4. Model the domain, then map the data:** Understand the domain, then design the object model, then map source data into that model. Do not attempt to look at the data and replicate its shape.
- 5. Mark non-semantic types as hidden:** When non-semantic types (types that serve a technical purpose rather than modeling real-world domain entities) are necessary for specific workflows, mark them as hidden to keep default views of the Ontology clean. They remain available for builders to leverage when building applications.

2. Don't repeat yourself (rule of three)

[API Reference ↗](#)

[Send feedback](#)

Ontology Design Don't Repeat Yourself



If you built the same thing three times, refactor.

Duplicated object types, redundant properties, and copy-pasted workflows are a maintenance burden and a context-management problem, for both humans and AI agents that need to reason about the Ontology. The goal is a single canonical representation for each concept, with a single canonical workflow for each operation on that concept. The rule of three is a practical trigger for applying this principle: one instance is a coincidence, two is a pattern, and three means it is time to refactor.

Anti-patterns

- Multiple object types share the same set of properties and similar links
- The same derived property logic or action logic appears across multiple types
- Different teams have created near-identical object types for slightly different purposes
- Copy-pasted workflows exist with minor variations across types

Example

API Reference ↗

Three teams have independently created customer-related object type schemas:

Send feedback

x Avoid		✓ Prefer
Sales Customer		Customer (single canonical type)
– name		– name
– email		– email
– phone		– phone
		– salesStatus
Support Customer	→	– supportTier
– name		– billingAccountId
– email		
– phone		– OR, if shapes are genuinely distinct
Billing Customer		Interface: CustomerBase
– name		– name
– email		– email
– phone		– phone
(Three types, three sets of actions, three maintenance burdens)		Implemented by: SalesLead, SupportCor BillingAccount

Impact of anti-patterns

Problem	Impact
Maintenance burden	Changes must be replicated across every duplicate. Missed updates cause drift between copies.
Ambiguous context	Users and AI agents cannot determine which of several near-identical types is canonical.
Inconsistent behavior	Duplicated action or derived property logic diverges over time, producing conflicting results.
Wasted development effort	Teams re-build the same thing in slightly different forms instead of collaborating on one shared model.

E API Reference ↗

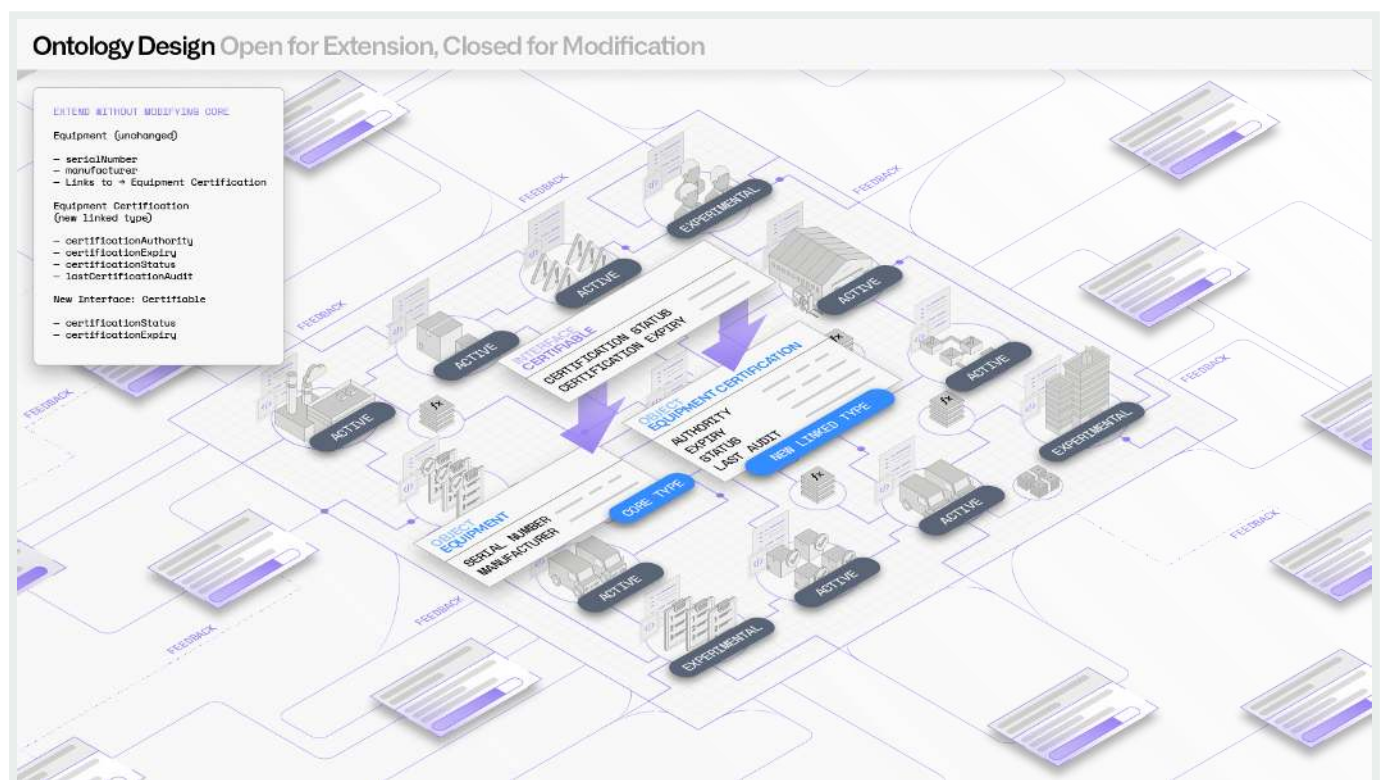
1. **Audit for duplicates:** If multiple object types share a common shape (e.g. similar links, similar actions), evaluate whether they should be a single type with a

Send feedback

distinguishing property or should implement a shared interface.

2. **Consolidate shared logic:** If the same derived property or action logic appears across multiple types, extract it into an interface or shared function.
3. **Unify team-specific copies:** When different groups have created near-identical object types, unify them into a single canonical representation with appropriate security or filtering.
4. **Apply the rule of three:** One duplicate can be acceptable. Two is a warning sign. Three means it is time to refactor.

3. Open for extension, closed for modification



Protect core models. Enable builders to extend them.

Once an object type, interface, or workflow is field-tested and in production, its core structure should be stable. Other developers and teams in the organization should be able to build on top of it, adding new object types that implement an interface or new workflows that API Reference ↗ g objects, without needing to modify the core model.

- Frequent breaking changes to established object types that cascade across dependent applications
- New use cases require modifying existing core types rather than extending them
- Teams need to edit shared interfaces or actions to accommodate team-specific needs
- Security changes for one team's extension inadvertently affect other consumers

Example

A core `Equipment` object type and `Inspectable` interface are in production. A new team needs to track certification data for some equipment:

x Avoid	✓ Prefer
Modify the core <code>Equipment</code> type:	Extend without modifying core:
Equipment – <code>serialNumber</code> – <code>manufacturer</code> – <code>certificationAuthority</code> (new) → – <code>certificationExpiry</code> (new) – <code>certificationStatus</code> (new) – <code>lastCertAudit</code> (new)	Equipment (unchanged) – <code>serialNumber</code> – <code>manufacturer</code> – <code>Links to → Equipment Certification</code>
(Four new properties, null for all non-certified equipment, existing consumers must handle the change)	Equipment Certification (new linked type) – <code>certificationAuthority</code> – <code>certificationExpiry</code> – <code>certificationStatus</code> – <code>lastCertificationAudit</code>
	New interface: <code>Certifiable</code> – <code>certificationStatus</code> – <code>certificationExpiry</code>
	(Core type untouched, new capability added via linked type and interface)

Problem	Impact
Breaking changes	Modifications to core types can break dependent applications, actions, and workflows across the organization.
Scope creep	Core types accumulate properties and logic for every new use case, trending toward a God Object .
Entangled ownership	Multiple teams modify the same core type, creating merge conflicts and unclear accountability.
Security leakage	Extending a core type without clean boundaries can inadvertently widen data access.

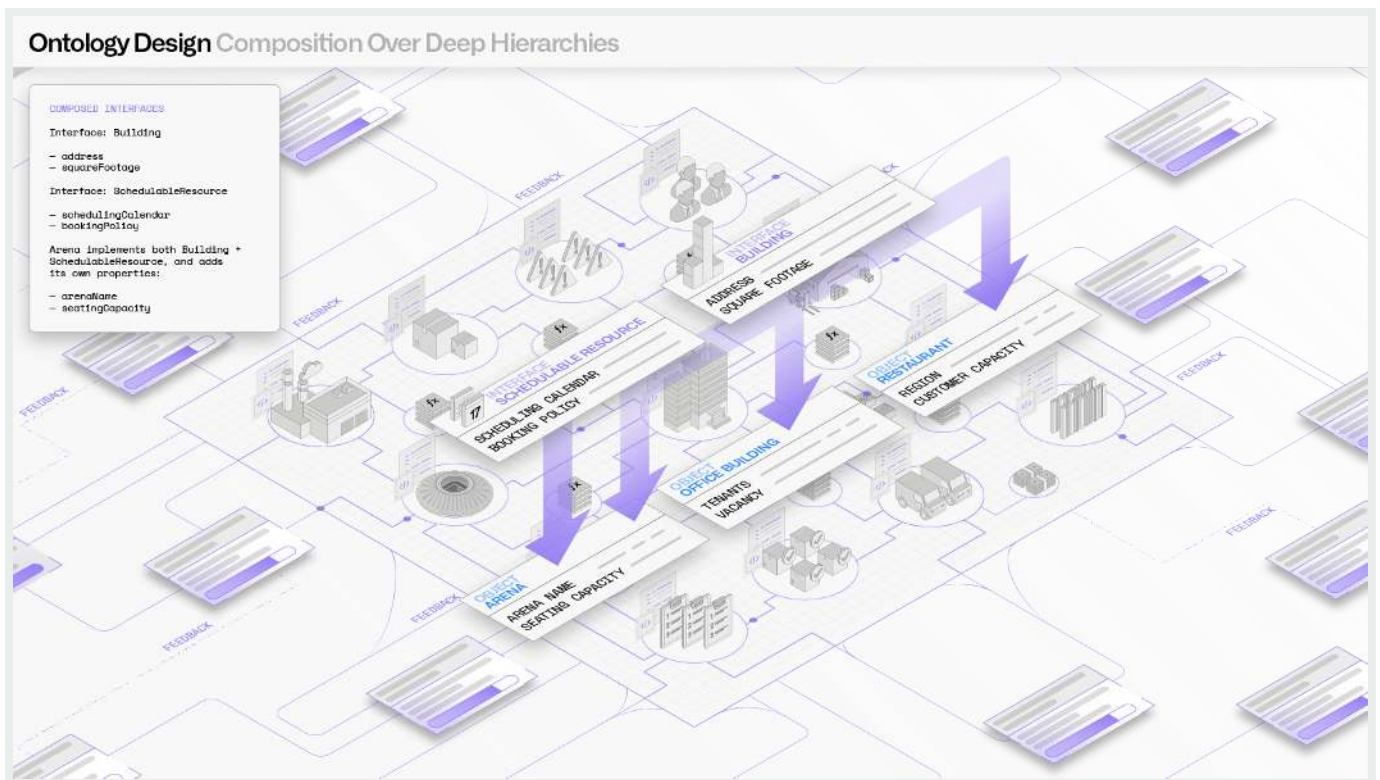
Best practices

1. **Identify what is essential:** Determine which properties and links are truly fundamental to the entity. Lock those down.
2. **Design for extension:** When creating core types and interfaces, anticipate that others will build on them. Leave room for linked extension types and new interface implementations.
3. **Extend rather than modify:** When adding to an existing model, consider if the addition belongs on the core type, or on an extension (a new linked object type, a new interface implementation, or a new property namespace).
4. **Enforce security boundaries:** Core data models should have well-defined security boundaries so that extending the Ontology does not inadvertently widen access.

4. Composition over deep hierarchies

API Reference ↗

Send feedback



Favor multiple inheritance via interfaces. Keep things pluggable.

Foundry's Ontology supports multiple inheritance through interfaces, so an entity can compose behavior from multiple focused abstractions instead of a single-inheritance chain.

Anti-patterns

- Deep single-inheritance chains where child types exist solely to combine parent capabilities
- "Combination" types like `SchedulableBuilding` or `InspectableVehicle` that merge two unrelated concepts into one type
- Workflows are tightly coupled to specific object types when they could operate on a shared interface
- Adding a new capability to an entity requires restructuring the inheritance chain

Example

[API Reference ↗](#)

An `Arena` needs to be both a building and a schedulable resource:

[Send feedback](#)

x Avoid	✓ Prefer
Deep single-inheritance: <pre> Asset ├── PhysicalAsset │ └── Building │ └── SchedulableBuilding │ └── Arena </pre> (Every new combination of capabilities requires a new intermediate type. A SchedulableWarehouse would need yet another branch.)	Composed interfaces: <pre> Interface: Building - address - squareFootage Interface: SchedulableResource - schedulingCalendar - bookingPolicy Arena implements both: Building + SchedulableResource - arenaName - seatingCapacity </pre> (Adding SchedulableWarehouse only requires implementing the same two interfaces – no new hierarchy needed)

Impact of anti-patterns

Problem	Impact
Combinatorial explosion	Every new combination of capabilities requires a new intermediate type in the hierarchy.
Brittle hierarchies	Changes to a parent type cascade unpredictably through all descendants.
Limited reuse	Workflows built on a specific type deep in the chain cannot be reused for other types that share the same capability.
Semantic distortion	Contrived parent types (like <code>SchedulableBuilding</code>) do not represent real-world concepts, violating the domain-driven design principle .
API Reference ↗	

Best practices

Send feedback

- 1. **Design interfaces around capabilities or roles:** Use focused interfaces like `Inspectable`, `Schedulable`, `Billable`, or `Depreciable` that capture a specific behavior or property set.
- 2. **Use taxonomic interfaces for aggregation:** Taxonomic identity interfaces (for example, `MilitaryAsset` implemented by `Aircraft`, `Vessel`, `GroundVehicle`) are particularly useful for drilldown investigations or similar aggregation workflows.
- 3. **Target interfaces in workflows:** When building actions, functions, and applications, target interfaces where possible. A workflow built on the `SchedulableResource` interface works for arenas, conference rooms, and vehicles without modification.
- 4. **Compose rather than inherit:** When an entity needs multiple capabilities, implement multiple interfaces rather than inserting it into a deep single-inheritance chain.

Pragmatism and tradeoffs

These principles are guides, not laws.

Real-world constraints, including deadlines, legacy systems, partial platform support, and user skill levels, mean that the ideal Ontology design is not always immediately achievable. Use the following guidelines to navigate tradeoffs:

Guideline	Detail
Steer toward good design without being a roadblock	If something needs to be working within a tight deadline, build something reasonable now with a clear path to improvement.
Name the tradeoffs explicitly	When recommending a shortcut, explain what is being traded away and when it might matter. For example, a denormalization could work fine at your current scale, but if you grow past 10k objects, you may want to revisit.
Prefer incremental improvement over big-bang refactors	A slightly imperfect Ontology that is in use and generating value is better than a theoretically perfect one that is still being designed.
Defend the critical API Reference ↗	Naming quality, semantic clarity, and security design are hard to fix later. Cut corners on implementation details, not on these.

Send feedback

The Ontology is the software that powers your organization. Treat it with the same care you would give to a production codebase, but prioritize business value over perfection.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

Ontology design: Structural guidance

The following sections provide guidance on how to structure properties, relationships, and access control within the Ontology.

Normalization and derived properties

Store each fact once. Use derived properties for convenience.

Denormalized data (copying values from linked objects onto a parent object) can be risky. When the data source changes, every copy must be updated. Normalization keeps data consistent, and derived properties give you the convenience of denormalized access without the upkeep.

Not all computed values are the same. The right approach depends on whether a value can be safely pre-computed from stable inputs or whether it needs to stay in sync with dynamic Ontology changes.

Pre-computed vs. dynamically derived values

Type	Characteristics	Recommended tool	Example
API Reference ↗ Computed	Computed from properties on the same object; inputs rarely change or	Pipeline transform	<pre>fullName = firstName + " " + lastName</pre> Send feedback Inputs are stable and

Type	Characteristics	Recommended tool	Example
	only change due to pipeline ingestion.		updated in the same pipeline, so pre-computing is safe and adds zero runtime overhead.
Dynamically derived	Depends on linked objects or values that change via actions, automations, or other Ontology-level operations.	Derived property	directReportCount Employees are reassigned, onboarded, and offboarded through actions. A derived property that counts linked Employee objects stays correct automatically.

 When a value depends on changes made through actions, every action that could affect the value must also update the value. If any action fails to do so, the value will remain incorrect until the discrepancy is identified.

Anti-patterns

- The same value is stored as a property on multiple object types
- Properties go stale because they are copies of values maintained elsewhere
- Updating a single real-world fact requires writes to multiple objects
- Integer or count properties are manually maintained rather than computed from links

Example

A [API Reference ↗](#) page needs to display a count of direct reports:

Send feedback

x Avoid	✓ Prefer
Manager – directReportCount: 5 (manually maintained integer; must be updated every time an employee joins or leaves) Employee – managerName: "Alice" (copied from the linked Manager; breaks if the manager's name changes)	Manager – directReportCount (derived): counts linked Employee objects at query time Employee – manager (link to Manager)

Performance considerations

Derived properties are evaluated at runtime. The performance characteristics vary by scale:

Scale	Recommendation
Low to moderate (<~10k objects per query)	Use derived properties freely. Runtime evaluation is sufficiently performant for most workflows.
High (>~10k objects per query)	Derived properties may introduce latency due to higher-overhead query paths. Denormalization may be an appropriate tradeoff, but it should be a conscious, documented decision and not the default.

Best practices

1. **Store each fact in one place**, on the object where it semantically belongs.
2. **Use derived properties** to compute or aggregate values from linked objects at query
3. **Monitor performance** as scale grows. If derived properties introduce latency at high scale, consider selective denormalization.

API Reference ↗

Send feedback

4. **Document any denormalization** with the rationale, the source of truth, and the update strategy for keeping copies in sync.

Structs

Group semantically related fields into structs.

When a property is naturally multi-field (for example, an address with street, city, state, and postal code), use a struct rather than flattening into separate properties. Structs preserve semantic grouping and enable richer metadata capture.

When to use structs

Scenario	Example
Multi-field values	Address (street, city, state, postal code), coordinates (geopoint, altitude)
Values with metadata	AI-generated outputs with confidence scores, source references, and reasoning
Multi-valued properties with selection logic	Multiple phone numbers where a reducer surfaces the primary one

Example

Modeling an address on a `Facility` object type:

x

Avoid

Facility

- addressStreet

- addressCity

API Reference ↗

- addressPostalCode

- addressCountry

- addressGeopoint

→

✓

Prefer

Facility

- address (struct array)

- street (Main field)

- city (Main field)

- state (Main field)

- postalCode

- country (Main field)

Send feedback

- addressLastOccupied - addressDatasource - addressLlmConfidence - addressLlmReasoning (Ten unrelated properties with a naming convention as the only link between them)	- geopoint - lastOccupied (used for reducing) - datasource - llmConfidence - llmReasoning (One semantic concept with main fields and structured sub-fields)
---	--

Key benefits

Benefit	Details
Semantic grouping	An address is one concept, not ten unrelated properties. The Ontology reflects this.
Metadata capture	Structs can carry source, confidence, and timestamp information alongside the primary value.
Reducer support	In multi-valued scenarios, reducers can surface the most relevant value (for example, the address with the most recent lastOccupied field).
Main field behavior	A struct can designate one or more main fields so it behaves like a simple property or as a struct with a subset of the fields in interfaces and queries.

Structs are especially valuable in AI-first workflows where large language model (LLM) outputs have both a primary result and associated metadata (reasoning, source references, confidence scores). Capture these together rather than scattering them across unrelated properties.

Best practices

- API Reference ↗

Id properties where the fields are semantically related and always used together.
2. **Define the struct** with clear field names and types.

3. **Designate a main field** so the struct behaves like a simple property in most contexts.
- Send feedback

- 4. **Use reducers** for multi-valued struct properties to surface the most relevant value.
- 5. **Capture metadata** (source, confidence, timestamps) in the struct alongside the primary value, especially for AI-generated outputs.

Interfaces

Use interfaces to build reusable, future-proof abstractions.

Interfaces are the primary tool for achieving the ["Don't repeat yourself" design principle](#) and [open/closed extensibility](#). They define a shared shape (properties, links, actions) that multiple object types can implement, enabling workflows to target the interface rather than individual types.

When to use interfaces

Scenario	Example
Common properties across types	Inspectable interface with <code>lastInspectionDate</code> and <code>inspectionStatus</code> , implemented by <code>Vehicle</code> , <code>Equipment</code> , <code>Facility</code>
Shared workflows	A scheduling workflow targeting <code>SchedulableResource</code> works for arenas, conference rooms, and vehicles without modification
Taxonomic grouping	A <code>MilitaryAsset</code> interface implemented by <code>Aircraft</code> , <code>Vessel</code> , <code>GroundVehicle</code> for drilldown aggregation workflows
Multi-level abstraction	<code>SchedulableResource</code> extends <code>Trackable</code> , adding scheduling-specific properties to a broader tracking abstraction

Example

Multiple object types need inspection tracking:

API Reference ↗

x Avoid

✓ Prefer

Send feedback

Vehicle - lastInspectionDate - inspectionStatus - (duplicate action: Schedule Vehicle Inspection)		Interface: Inspectable - lastInspectionDate - inspectionStatus - (shared action: Schedule Inspection)
	→	Vehicle implements Inspectable - make, model, mileage, ...
Equipment - lastInspectionDate - inspectionStatus - (duplicate action: Schedule Equipment Inspection)		Equipment implements Inspectable - serialNumber, warrantyExpiry, ...
Facility - lastInspectionDate - inspectionStatus - (duplicate action: Schedule Facility Inspection)		Facility implements Inspectable - address, capacity, ...
(Three copies of the same properties and logic, maintained independently)		

Platform considerations

Even where current platform tooling does not fully support interface-backed workflows, designing with interfaces establishes a foundation that pays off as support expands.

Situation	Guidance
The interface is fully supported in your workflow	Target the interface directly. A single workflow covers all implementing types.
The interface is not yet supported in a API Reference ↗	Define the interface now and duplicate the workflow per type as a temporary measure. This approach is no less efficient than working without an interface, and it establishes a clear path to consolidation once support is available.

Review our [interface documentation](#) for current support details.

Send feedback

Best practices

1. **Identify common shapes:** If multiple object types share properties, links, or actions, define an interface that captures the shared shape.
2. **Design interfaces around capabilities or taxonomy:** Capability interfaces may include `Inspectable`, `Schedulable`, or `Billable`. Taxonomic interfaces may include `MilitaryAsset` or `MedicalDevice`.
3. **Target interfaces in workflows:** Build actions, functions, and applications against interfaces where possible.
4. **Extend interfaces for multi-level abstraction:** Interfaces can extend other interfaces to build layered abstractions.
5. **Scaffold now, consolidate later:** Define interfaces even if some workflows must temporarily be duplicated per-type due to current platform support gaps.

Links and object-backed link types

Links should represent semantically meaningful relationships.

Every link type should answer a clear domain question, such as:

- Which facility did this patient visit?
- Which team does this employee belong to?
- Which equipment was used in this work order?

When to use link types

Link type	Use when	Example
Direct link	The relationship is meaningful but carries no metadata of its own.	Employee → Department
Object-backed link	The relationship carries its own metadata (dates, roles, status, allocation).	Employee → VentureStaffing → Venture (with role, startDate, allocation)

API Reference ↗

Send feedback

Not every linking object needs to be visible in every context. Some workflows care about the join metadata, others just want the direct connection. Object-backed links let you expose either view depending on the workflow.

Example

Modeling the relationship between employees and ventures, where each assignment has a role and start date:

x Avoid	✓ Prefer
Employee → Venture (direct link) (no way to capture role, start date, or allocation per assignment) →	Employee → Venture Staffing → Venture
– OR –	Venture Staffing – role – startDate – allocationPercentage – status
Employee – ventureRole – ventureStartDate (ambiguous if employee has multiple venture assignments)	Workflows can expose either: – Direct: Employee → Venture – Detailed: Employee → Staffing → \

Impact of incorrect link design

Problem	Impact
Lost metadata	Direct links cannot capture when, why, or in what capacity a relationship exists.
Ambiguous multi-links	Properties like <code>ventureRole</code> on the source object become ambiguous when an entity participates in multiple relationships.
API Reference ↗ links	Links that exist only because two datasets share a foreign key add noise to the Ontology and confuse navigation

Send feedback

Best practices

- 1. **Validate semantic meaning:** Avoid links that exist only because two datasets share a foreign key. Ask if the relationship is meaningful in the domain.
- 2. **Evaluate whether the relationship carries metadata:** If it does (dates, roles, status), use an object-backed link type to capture that metadata.
- 3. **Expose the right level of detail:** Design workflows to use either the direct relationship or the detailed relationship through the linking object, depending on the context.
- 4. **Name links for clarity:** Link names should describe the relationship from each direction. Review the section on [naming conventions](#) for more information.

Naming conventions

Optimize for human readability and agent navigability.

Consistent, descriptive naming is one of the most impactful investments you can make in Ontology quality. Clear names make the Ontology easier for both humans and AI agents to navigate, and they are far harder to correct once the Ontology is in use.

Naming rules

Element	Convention	Good examples	Bad examples
Object types	Singular, concrete nouns a domain expert would recognize	Patient , WorkOrder , FlightSegment	Data , Item , Record
Properties	Concise, self-evident; no encoded type info or implementation details	age , status , lastInspectionDate	dtLastInspMod , nVAL01 , fieldX
LINKS	API Reference ↗ had naturally from each direction	department (Employee → Dept), employees (Dept → Employee)	relatedItems , Send feedback

Element	Convention	Good examples	Bad examples
Dates	Follow a single convention consistently across the Ontology	createdDate , updatedDate , effectiveDate	Mixing createdDate and dateOfCreation
Ambiguous terms	Qualify with specific meaning	monetaryValue , quantityOnHand , riskScore	value , quantity , score

Example

x Avoid		✓ Prefer
Object type: Item	→	Object type: Product
Property: dtLastInspMod	→	Property: lastInspectionDate
Property: value	→	Property: monetaryValue Property: quantityOnHand
Link: Item → Related Item	→	Link: Product → Supplier Link: Employee → Supervisor

Best practices

- 1. Establish naming conventions before building:** Agree on patterns for dates, statuses, identifiers, and links up front.
- 2. Follow the Ontology's established conventions:** If the Ontology already uses `createdDate`, do not introduce `dateOfCreation`.
- 3. Qualify ambiguous properties:** Use `monetaryValue`, `quantityOnHand`, and `riskScore`. Do not use `value`, `quantity`, and `score`.
- 4. API Reference ↗ Relationship:** A link from `Employee` to `Department` should be `department` (from the employee's perspective) and `employees` (from the department's perspective).

Send feedback

5. Review names with end users: Names that seem clear to the builder may be ambiguous to consumers. Validate with the people who will use the Ontology every day.

Security design

Design security semantically, following the principle of least privilege.

Security in the Ontology should be expressed in terms that make sense in the domain, not in terms of data infrastructure. Users should be able to look at a security configuration and understand what is protected and why.

Security model

Combine row-level and column-level security for fine-grained cell-level access control:

Security layer	Controls	Example
Row-level	The objects a user can view	VIP patients are restricted to senior staff
Column-level	The properties a user can view on visible objects	Clinical notes are restricted to the care team
Cell-level (combined)	The intersection of row and column restrictions	VIP patients' clinical notes are visible only to the senior care team

Example

Controlling access to sensitive patient data:

x Avoid	✓ Prefer
PublicPatient (object type) API Reference ↗ - dob - diagnosis	Patient (single object type) - name - dob - diagnosis (do not use for care team only)

RestrictedPatient (object type)	→	- clinicalNotes (column-restricted: care team only)
- name		
- dob		- mentalHealthRecords (column-restricted: psychiatry team only)
- diagnosis		
- clinicalNotes		
- mentalHealthRecords		
(Duplicated schemas; security achieved by splitting types. Properties added to one type are easily forgotten on the other.)		Row-level security: - VIP patients: senior staff only
		Column-level security: - clinicalNotes: care team only - mentalHealthRecords: psychiatry c
		(One type; security achieved by policy. Domain boundaries drive access rules)

Impact of incorrect security design

Problem	Impact
Duplicated types for security	Schemas drift out of sync; properties added to one type are easily forgotten on the other. Violates the "Don't repeat yourself" design principle .
Over-permissive defaults	Starting with broad access and restricting later risks exposing sensitive data before lockdown is complete.
Ad-hoc filtering instead of policy	Security logic scattered through application code rather than enforced at the Ontology layer is fragile and difficult to audit.
Misaligned boundaries	Security boundaries that do not follow domain boundaries are harder to reason about and more likely to have gaps.

Best practices

- 1. Start restrictive, open up deliberately:** Default to minimal access and widen as needed, [API Reference ↗](#) ng open and restricting later.
- 2. Use row-level and column-level security together** for fine-grained, cell-level access control.

Send feedback

3. **Align security with domain boundaries:** If your domain has natural access boundaries (a regional manager sees their region's data; a care team sees their patients), model those boundaries using Ontology relationships and security policies rather than ad-hoc data filtering.
4. **Avoid duplicating object types for security:** A single type with well-designed security policies is better than multiple types with duplicated schemas.
5. **Review new ontology paths for access-control consistency:** Ensure added links, types, or properties preserve the intended protections around restricted data.

Use the guidance on this page to ensure security boundaries align with domain boundaries, then refer to our [security and governance documentation](#) for configuration details.

PREVIOUS

← [Ontology design: Best practices](#)

NEXT

[Ontology design: Anti-patterns](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology design: Anti-patterns

Even experienced Ontology designers can fall into common design traps that seem reasonable initially but create significant problems as the Ontology grows. This section identifies recurring anti-patterns, explains why they occur, and provides concrete guidance for avoiding or resolving them.

Avoiding these anti-patterns will help you build an Ontology that accurately represents your business domain, reduces maintenance overhead, and enables powerful cross-functional workflows.

Anti-pattern	Description	Solution
System Silos	Creating separate object types for each source system.	Merge data in pipelines; create unified object types.
The Kitchen Sink	Including unnecessary technical columns as properties.	Curate properties intentionally; exclude ETL metadata.
Department Silos	Each department creates their own version of shared entities.	Create shared object types; use properties and links for department-specific data.
The God Object	One object type represents multiple distinct entities.	Create distinct object types; use interfaces for shared characteristics.
The Golden Rule	Relying too heavily on a single tool (action types, pipelines, or functions, for example) for every problem instead of choosing the right capability.	Match the tool to the job: batch or streaming pipelines for data processing, actions for human decisions, automations for event-driven reactions, complex real-time

Anti-pattern	Description	Solution
Action Sprawl	Creating many single-property actions instead of cohesive business operations.	Design actions around business operations that bundle related changes into meaningful workflows.
The Time Machine	Modeling historical versions as separate objects or object types.	Use a single object per entity with linked history/amendment objects and time series properties.
The Misnomer	Using vague, generic, or misleading names for Ontology elements.	Use specific, descriptive names; qualify ambiguous properties; name links by relationship.

Anti-pattern: System Silos

System Silos occur when you create separate object types for the same real-world entity based on the source system the data originates from, rather than modeling the entity itself.

Common causes

- Different teams own different source systems and build independently
- Uncertainty about how to merge data from multiple sources
- Desire to preserve system-specific fields without deciding what's essential

Example

Your organization has employee data in three systems: an HR system, a badge access system, and a project management tool. Instead of creating a single `Employee` object type, you create:

- `HR System Employee`
- `API Reference Employee`
- `Project Management Employee`

Send feedback

Problems

Problem	Impact
Fragmented view of reality	End users cannot see a unified view of an employee; they must navigate multiple object types to understand the full picture.
Duplicated effort	Action types, link types, and applications must be built multiple times for what is conceptually the same entity.
Inconsistent data	The same employee may have conflicting information across object types with no clear source of truth.
Complex maintenance	Changes to business logic must be replicated across all system-specific object types.

Solution

Create a single object type representing the real-world entity and use data pipelines to merge information from multiple source systems into a unified backing dataset.

x Avoid HR System Employee Badge System Employee Project Management Employee	✓ Prefer Employee → (backed by merged dataset from all three systems)
---	--

To implement this:

1. Identify the primary key that uniquely identifies the entity across systems (for example, employee ID).
2. Build a transform that joins data from all source systems.
3. Define clear precedence rules for conflicting values (for example, HR system is authoritative for job title).
4. Create a single object type backed by the merged dataset.

API Reference ↗

Send feedback

Anti-pattern: The Kitchen Sink

This anti-pattern (also known as "everything but the kitchen sink") occurs when object types include unnecessary columns from external systems that have no business relevance in the Ontology context, cluttering the data model with technical artifacts.

Common causes

- "Just in case" mentality (keeping fields that might be useful later)
- Lack of clarity on what fields are meaningful
- Direct mapping from source systems without curation
- Fear of losing data by excluding columns

Example

When creating a **Customer** object type from a CRM system integration, you include all available columns:

- customer_id ✓
- customer_name ✓
- email ✓
- _crm_extracted_at ✗
- _crm_received_at ✗
- _crm_batched_at ✗
- _crm_sequence ✗
- _crm_table_version ✗
- _crm_internal_record_id ✗
- last_etl_update_timestamp ✗

Problems

API Reference ↗	Impact	Send feedback
Confusion	End users see irrelevant technical fields and data.	

Problem	Impact
Performance degradation	Unnecessary properties increase data scale, compute, index size, and slow down searches.
Obscured insights	Important business properties are buried among system metadata.

Solution

Curate properties intentionally. Only include columns that have clear business meaning and will be useful for workflows.

Use these guidelines when deciding which properties to include:

Include	Exclude
Business identifiers (customer ID, order number)	Pipeline metadata
Human-readable attributes (name, description)	Internal system IDs with no business meaning
Dates relevant to business processes	Timestamps only relevant to data engineering
Status fields needed for filtering or actions	Audit columns for pipeline debugging

To implement this:

1. Review each column and ask: "Would someone ever need to see, search, or filter by this?"
2. Keep technical metadata in the backing dataset for debugging, but do not expose it as properties.
3. Use property visibility settings to hide any borderline properties that must exist but are rarely needed.
4. Document why each property exists and who uses it.

API Reference ↗

Send feedback

Department Silos occur when different departments create their own versions of the same object type, leading to a fragmented Ontology that mirrors organizational structure rather than business reality.

Common causes

- Departments work in isolation without cross-functional coordination
- Each team believes their view of the customer is unique
- Lack of governance or central Ontology design authority
- Teams want autonomy and control over "their" data

Example

Multiple departments need to work with customer data, and each creates their own object type:

- Sales team creates Sales Customer
- Support team creates Support Customer
- Finance team creates Billing Customer
- Marketing team creates Marketing Contact

All four object types represent the same real-world entity: a customer.

Problems

Problem	Impact
No single source of truth	Different departments have conflicting information about the same customer.
Impossible cross-functional workflows	Cannot easily answer questions like "Show me all interactions with this customer across sales, support, and billing".
API Reference ↗ development	Each department builds redundant actions, links, and applications.

Send feedback

Problem	Impact
Governance nightmare	Data quality issues multiply; fixes in one object type do not propagate to others.

Solution

Create shared object types that serve multiple departments, using properties and links to capture department-specific information where needed.

x Avoid		✓ Prefer
Sales Customer		Customer
Support Customer	→	└─ salesStatus (property)
Billing Customer		└─ supportTier (property)
Marketing Contact		└─ billingAccountId (property)
		└─ Links to:
		└─ Sales Opportunities
		└─ Support Tickets
		└─ Invoices

To implement this:

1. Identify entities that exist across departmental boundaries.
2. Establish a cross-functional working group to define shared object types.
3. Use properties to capture department-specific attributes on shared objects.
4. Use link types to connect shared objects to department-specific objects (such as `Customer` → `Support Ticket`).
5. Leverage object views or curated Workshop and OSDK applications if departments need different "views" of the same underlying entity.
6. Use restricted views if specific properties can only be accessible by a specific team.

The God Object anti-pattern occurs when a single object type is overloaded to represent multiple distinct real-world entities, resulting in a bloated, confusing, and unmaintainable object type.

Common causes

- Over-abstraction driven by superficial similarities ("they are all assets")
- Desire to minimize the number of object types
- Lack of clear entity definitions before building
- Scope creep as more use cases are added to an existing object type

Indicators

- An object type has many properties that are frequently null
- Property meanings change based on another property's value (such as type or category)
- You find yourself asking "What kind of [Object] is this?" when viewing an object
- Business rules and validations require extensive conditional logic based on object "type"

Example

You create an `Asset` object type intended to represent "anything valuable," which ends up including:

- Physical equipment (trucks, machinery)
- Software licenses
- Real estate properties
- Financial instruments
- Employees (as "human assets")

The object type has 150+ properties, most of which are null for any given object, and the meaning of properties like value, location, and status varies completely depending on what kind of object represents.

[API Reference ↗](#)

Problems

[Send feedback](#)

Problem	Impact
Semantic confusion	End users cannot understand what an <code>Asset</code> actually represents.
Sparse data	Most properties are null for most objects, making the data hard to interpret.
Impossible validation	Cannot enforce business rules because rules differ by entity type.
Poor search experience	Searching for <code>Assets</code> returns a mix of unrelated things.
Action type complexity	Actions must handle wildly different entity types with complex conditional logic.

Solution

Create distinct object types for distinct real-world entities. Use interfaces to model shared characteristics when entities genuinely share common properties or behaviors.

<code>✗ Avoid</code>	<code>✓ Prefer</code>
<code>Asset</code> <code>- assetType</code> <code>- assetSubtype</code> <code>- value</code> <code>- location</code> <code>- status</code> <code>- 145 more properties...</code>	<code>Equipment</code> <code>Vehicle</code> <code>Software License</code> <code>Property (Real Estate)</code> <code>Financial Instrument</code> <code>Interface: Depreciable Asset</code> <code>- purchaseDate</code> <code>- purchaseValue</code> <code>- depreciationSchedule</code>

To implement this:

API Reference ↗

1. List the distinct real-world entities currently represented by the object type.
2. Create separate object types for each distinct entity.
3. Identify genuinely shared properties and behaviors.

Send feedback

4. Use interfaces to model shared characteristics across object types.
 5. Migrate existing objects to appropriate new object types.
-

Anti-pattern: The Golden Hammer

The Golden Hammer anti-pattern occurs when you rely too heavily on a single tool to solve every problem, even when other approaches are more appropriate. The name comes from the saying: "If all you have is a hammer, everything looks like a nail" ↗.

This anti-pattern manifests in the overuse of action types in work better suited for pipelines, building pipelines for logic that should be event-driven automations, or writing functions for calculations that are better pre-computed in a transform.

Common causes

- Overreliance on a tool due to familiarity and visibility within the team
- Desire to give end users "control" over when computations happen
- Lack of familiarity with the full platform (including pipelines, automations, functions, and scheduled builds)
- Thinking exclusively in one layer (Ontology-first, pipeline-first, or code-first) without considering the full toolkit

Examples

Overreliance on action types:

You need to calculate aggregate metrics for a dashboard showing total sales by region. Instead of using a data pipeline to pre-compute these metrics, you create an action type called `Calculate Regional Sales Totals` that end users must manually trigger. Results are written back to objects via the action.

API Reference ↗

elines:

Send feedback

An alert object is created by a pipeline when sensor readings exceed a threshold. You want to automatically assign that alert to the on-call engineer and send a notification. Instead of using an automation that reacts to the new object, you build additional pipeline logic that tries to resolve the assignee and write the assignment into the backing dataset, mixing operational workflow logic into data integration.

Overreliance on functions:

You implement a simple property derivation like `fullName = firstName + lastName` as a function-backed column, adding runtime overhead and a code repository to maintain, when a single pipeline `concat` expression would suffice.

Problems

Problem	Impact
Scalability limits	Each tool has different execution limits; using the wrong one hits ceilings early.
Unnecessary complexity	Maintaining logic in the wrong layer increases the number of moving parts.
User burden	End users must perform steps that the platform could handle automatically.
Performance issues	Real-time calculations via actions or functions are slower than pre-computed pipeline results. Conversely, scheduled pipelines are too slow for event-driven reactions.
Difficult debugging	When logic lives in the wrong layer, failures are harder to diagnose and resolve.

Solution

Choose the right tool for the job based on your use case:

API Reference ↗	Best for	Not ideal for
Action types	Human decisions, user-initiated edits to one or a few objects, input-	Batch calculations, updates, Send feedback

Tool	Best for	Not ideal for
	driven changes that should apply immediately.	reactions with no human involvement.
Pipelines (batch)	Batch data processing, aggregations, cleansing, enrichment, pre-computing derived values on a schedule or on data arrival.	Real-time reactions to individual object changes, logic that requires human input.
Pipelines (streaming)	Continuous, low-latency data processing where results must stay current as source data arrives (real-time dashboards, live status tracking, continuous enrichment).	Infrequent updates where batch is sufficient, logic that requires human input, reacting to Ontology-level events (use automations).
Automations	Event-driven reactions to Ontology changes (object created, property updated, schedule triggered), orchestrating actions or notifications without user involvement.	Heavy data processing, complex multi-dataset joins, logic that requires human judgment.
Functions	Complex real-time computations across multiple objects, validation logic, derived values that depend on live Ontology state and cannot be pre-computed.	Simple derivations computable in a pipeline, batch processing of large datasets.
Schedules	Recurring pipeline builds, time-based or event-based orchestration of data refresh.	Reacting to individual object-level changes in real time.

Examples of applying this guidance:

x Avoid	✓ Prefer
Action: "Get Total Regional Sales" API Reference ↗	Pipeline that aggregates sales into a "Regional Sales Summary"
Action: "Standardize Address Format"	Pipeline that... Send feedback

Action: "Update Inventory Status" (based on quantity thresholds)	→	Pipeline that sets status based on quantity thresholds during each sync.
Action: "Assign Risk Score" (using a formula)	→	Pipeline or model that calculates risk score and writes to the backing database.
Pipeline that assigns alerts to on-call engineers by writing to the backing dataset	→	Automation that triggers an "Assign Risk Score" action when a new "Alert" object is created.
Pipeline that sends a notification when an object meets a condition	→	Automation that monitors for changes and sends a notification or triggers an action.
Batch pipeline polling every minute for new IoT sensor readings	→	Streaming pipeline that continuously processes sensor data as it arrives.
Function-backed column for fullName = firstName + " " + lastName	→	Pipeline that computes fullName = firstName + " " + lastName in the backing database.
Scheduled pipeline running every minute to check for objects needing follow-up	→	Automation that reacts to the state change and triggers the follow-up action.

To implement this:

- Before creating an action type**, ask: "Does this require human judgment or user input?" If not, it likely belongs in a pipeline or automation.
- Before adding logic to a pipeline**, ask: "Is this a data transformation, or is it an operational workflow?" Data cleansing, aggregation, and enrichment belong in pipelines. Assigning work, sending notifications, and reacting to individual changes belong in automations.
- Before writing a function**, ask: "Can this be pre-computed in the backing pipeline?" If the result only depends on source data columns and does not need live Ontology traversal, compute it upstream.

4. **Before building a polling pipeline** (running every N minutes to detect changes), ask:

API Reference ↗

Can we react to this event directly?" Automations respond to Ontology changes in real-time without the overhead of scheduled builds. If the need is for continuous data processing from a source system, consider a streaming pipeline instead.

Send feedback

5. **Before defaulting to a batch pipeline**, ask: "Does this data need to be continuously current?" If consumers depend on low-latency freshness, a streaming pipeline avoids the compromise of a batch schedule.
 6. **Use automations** to bridge the gap between "something changed" and "something should happen", without requiring a user to click a button or poll a pipeline.
-

Anti-pattern: Action Sprawl

Action Sprawl occurs when you create many narrowly-scoped action types that each modify a single property, rather than designing cohesive actions that represent meaningful business operations.

Common causes

- Thinking of actions as database column updates rather than business operations
- Building actions incrementally without considering the overall user experience
- Lack of understanding of how actions can bundle multiple property changes
- Mimicking CRUD operations from traditional application development

Indicators

- More than 10 action types for a single object type
- Multiple actions that are always performed in sequence
- Action names that read like `Set [Property]` or `Update [Property]`
- End users complaining about too many steps to complete a task

Example

For an `Employee` object type, instead of creating meaningful business actions, you create:

- API Reference ↗

 - `Update Employee First Name`
 - `Update Employee Last Name`
 - `Update Employee Email`

Send feedback

- Update Employee Phone
- Update Employee Department
- Update Employee Manager
- ...and 20 more single-property actions

Problems

Problem	Impact
Overwhelming experience	End users face a long, cluttered list of actions and struggle to find the right one.
Fragmented workflows	Simple updates require multiple action submissions to complete a single business task.
No cohesive business representation	Actions do not map to real-world processes, making the Ontology unintuitive.
Fragmented audit trails	History of changes is scattered across many small actions, making it difficult to understand what happened and why.

Solution

Design action types around business operations, not database updates. Create actions that bundle related changes into meaningful workflows.

x Avoid	✓ Prefer
Update Employee First Name Update Employee Last Name Update Employee Email Update Employee Phone	Update Employee Contact Information – firstName – lastName – email – phone
Update Employee Department Update Employee Manager Update Employee Location	Transfer Employee to New Department – newDepartment – newManager – newLocation – effectiveDate

API Reference ↗

Send feedback

Create Employee Record
Set Employee Start Date
Assign Employee Badge
Assign Employee Equipment

→

Onboard New Employee
– All required fields for a new hire
– Triggers downstream workflows
(badge assignment, equipment request)

To implement this:

1. Map out the real business processes that involve changing object data.
2. Group related property changes into single actions that represent those processes.
3. Use action parameters to allow optional fields within a cohesive action.
4. Name actions after the business operation: `Transfer Employee`, `Approve Purchase Order`, `Escalate Support Ticket`.
5. Use action rules and validation logic to enforce business constraints within the action.

Anti-pattern: The Time Machine

The Time Machine anti-pattern occurs when you model historical versions of an entity as separate objects or object types rather than using time series data, snapshots, or proper versioning strategies.

Common causes

- Desire to preserve a complete history of every change
- Misunderstanding of how to model temporal data in the Ontology
- Applying file-versioning mental models (v1, v2, v3) to object design
- Lack of awareness of time series properties or linked history patterns

Indicators

API Reference ↗

ins multiple objects representing the same real-world entity at different points in time

- Properties like `version`, `revision`, or `isCurrent` exist to distinguish

Send feedback

- Object counts grow proportionally with the number of changes rather than the number of entities
- End users are confused about which object to reference or link to

Example

To track changes to a `Contract`, you create:

- `Contract v1`, `Contract v2`, `Contract v3` as separate objects within the same object type
- Or worse: `Contract 2023`, `Contract 2024`, `Contract 2025` as separate object types for each year

Each "version" is a full copy of the contract with slightly different property values, and links to other objects (such as `Vendor` or `Department`) are duplicated across all versions.

Problems

Problem	Impact
Object count explosion	Every change creates a new object, rapidly inflating the Ontology with redundant data.
Ambiguous current state	It is difficult to identify which version is the "current" or authoritative version.
Ambiguous links	Links to contracts become unclear; which version should a <code>Vendor</code> or <code>Department</code> link to?
Complex reporting	Reporting across time periods requires filtering and deduplication logic that is error-prone.

Solution

Use `API Reference` per entity with properties for current state. Store historical changes in a separate object type, enable edits history, or leverage time series properties.

Send feedback

x Avoid		✓ Prefer
Contract v1 (object)		Contract (single object per contract)
Contract v2 (object)	→	– currentValue
Contract v3 (object)		– currentStatus
		– effectiveDate
– OR –		– Links to:
		└─ Contract Amendments
Contract 2023 (object type)		– amendmentDate
Contract 2024 (object type)		– previousValue
Contract 2025 (object type)		– newValue
		– changeReason

To implement this:

1. Use a single object per real-world entity with properties reflecting the current state.
2. Create a separate linked object type (such as `Contract Amendment` or `Contract History`) to capture historical changes.
3. Leverage time series properties for values that change frequently and need temporal tracking.
4. Use the backing dataset or edits history to maintain full historical records for audit trails if needed.

Anti-pattern: The Misnomer

The Misnomer anti-pattern occurs when you use vague, generic, or misleading names for object types, properties, and link types that do not clearly communicate their meaning, leading to confusion and misinterpretation across the Ontology.

Common causes

API Reference ↗

names that make sense to you but not to others

- Names are carried over directly from source system column names
- Desire for brevity over clarity

Send feedback

- Lack of naming conventions or governance standards
- Assumption that context will make meaning obvious

Indicators

- End users frequently ask "What does this property mean?" or "What kind of [Object] is this?"
- The same name could reasonably refer to multiple different concepts
- Property names are single generic words like `value`, `type`, `status`, `date`, or `name` without qualification
- Link types use generic labels like "related to" without specifying the nature of the relationship

Example

You create the following Ontology elements with ambiguous names:

- Object type: `Item` (What kind of item? Product? Line item? Inventory item?)
- Property: `value` (Monetary value? Quantity? Score? Rating?)
- Property: `type` (Type of what? What are valid values?)
- Property: `date` (Created date? Modified date? Due date? Effective date?)
- Link type: `Item` → `Related Item` (How are they related? Parent-child? Substitute? Accessory?)

End users encountering these names must guess at their meaning or dig into documentation to understand what the data actually represents.

Problems

Problem	Impact
API Reference ↗	End users cannot understand the Ontology without additional context, leading to incorrect analysis and decisions.

Send feedback

Problem	Impact
Steep learning curve	New team members must spend significant time learning what vague names actually mean.
Documentation dependency	Documentation becomes essential rather than supplementary, and falls out of date quickly.
Cross-team confusion	Different teams interpret the same vague names differently, leading to inconsistent usage.

Solution

Use specific, descriptive names for all Ontology elements. Names should be self-documenting so that anyone can understand meaning without additional context.

x Avoid		✓ Prefer
Object type: Item	→	Object type: Product Object type: Sales Order Line Item Object type: Warehouse Inventory Recc
Property: value	→	Property: monetaryValue Property: quantityOnHand Property: riskScore
Property: type	→	Property: productCategory Property: serviceTier
Property: date	→	Property: orderPlacedDate Property: contractEffectiveDate
Link: Item → Related Item	→	Link: Product → Purchasing Customer Link: Employee → Supervisor Link: Equipment → Manufacturing Faci1

API Reference ↗

To implement this:

Send feedback

1. Establish naming conventions before building and enforce them through governance reviews.
2. Use specific, descriptive names: `Product`, `Sales Order Line Item`, `Warehouse Inventory Record`.
3. Qualify ambiguous properties: `monetaryValue`, `quantityOnHand`, `riskScore`.
4. Name links explaining the relationship: `Purchasing Customers`, `Manufacturing Facility`, `Supervisor`.
5. Add descriptions to all Ontology elements explaining their meaning and valid values.
6. Review names with end users to ensure they are intuitive and unambiguous.

Building a successful Ontology

The anti-patterns described in this guide are common but avoidable. By focusing on the fundamental best practices (modeling reality rather than systems, curating properties intentionally, collaborating across teams, and choosing the right tools for each task), you can build an Ontology that scales with your organization's needs.

Remember that effective Ontology design is iterative. Start with clear entity definitions, involve stakeholders early, and refine your model as you learn what works. When you encounter challenges, revisit the principles in this guide to identify whether an anti-pattern may be emerging and course-correct before it becomes difficult to change.

PREVIOUS

← [Ontology design: Structural guidance](#)

NEXT

[Ontology search / Semantic search / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

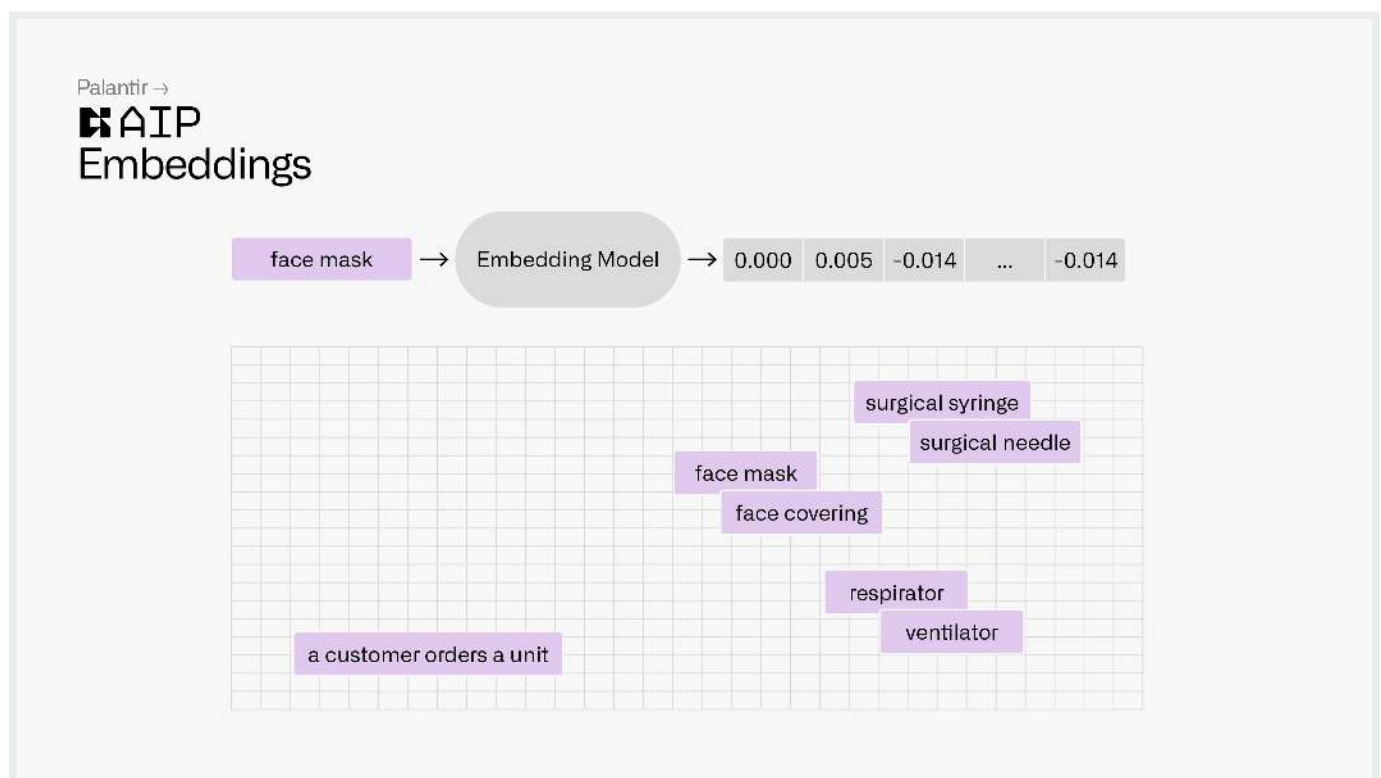
[API Reference](#) ↗

[Send feedback](#)

Semantic search

Semantic search is a way to search for text based on the inherent meaning or context, rather than relying solely on keywords or other traditional search methods.

Semantic search is accomplished using AI models to transform the text into vectors, which are arrays of numbers, and are called "embeddings". If the model is effective, the vectors, each of size N, that are close to each other in N-dimensional space are the ones that have similar underlying or semantic meaning. For example, the embedding vector of "face mask" will be closer to the embedding vector of "face covering" than it is to "respirator."



If [API Reference ↗](#) is then associated with a particular object in the [Ontology](#), then your traditional workflows become much more useful. Finding related entities or entities related to a particular search query is simply finding the nearest neighbors in the embedding space.

Send feedback

Review the following documentation pages for topics related to semantic search:

- [Learn how to create a semantic search workflow using a Palantir-provided model](#)
- [Learn how to create a semantic search workflow using a custom model](#)
- [Learn how to incorporate chunking into your semantic search workflow](#)
- [Learn how to use PDFs in your semantic search workflow](#)
- For additional learning materials, see our [YouTube video on "Building with Palantir AIP: Semantic Search"](#) ↗ and [blog on "Building with Palantir AIP: Semantic Search"](#) ↗.

PREVIOUS

← [Define Ontologies / Ontology design: Anti-patterns](#)

NEXT

[Document processing](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Semantic search / Document processing

Document processing

This page will discuss a few useful considerations for extracting data from a PDF document or image.

Data extraction

This page offers a basic guide for using [Pipeline Builder](#) to parse PDFs for semantic search and includes a recommendation for presenting the information in a [Workshop](#) application when you just have text content.

Semantic search is a powerful tool to use with PDFs, particularly if the content is broken down into smaller "chunks" that are embedded separately, helping users and workflows find important information that might otherwise be hard to access. This is especially useful considering the vast amount of unstructured knowledge in PDFs that often goes unnoticed.

To use, simply upload your PDFs to Foundry, extract the text, chunk the same text, search for those chunks, and surface the results of that search with the corresponding PDF rendered on the side for source-of-truth cross-validation for the users.

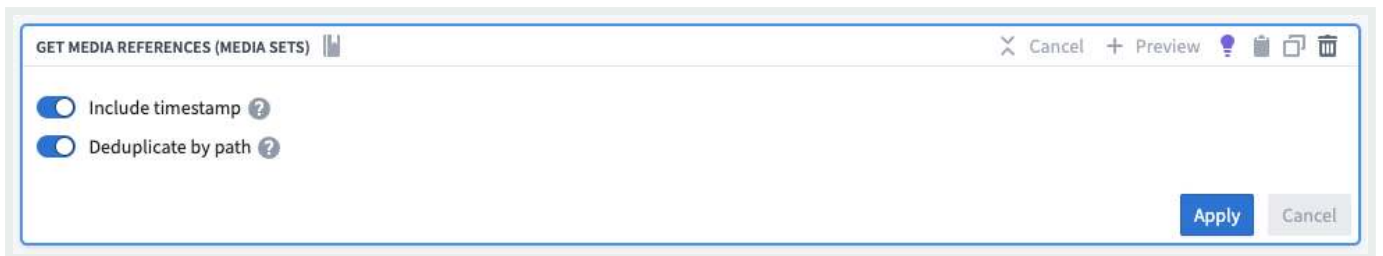
Follow the steps outlined below to import PDFs and extract text from PDFs:

1. [Import the PDFs as a media set.](#)
2. [Add the media set to Pipeline Builder.](#)
3. Use the **Get Media References** board.

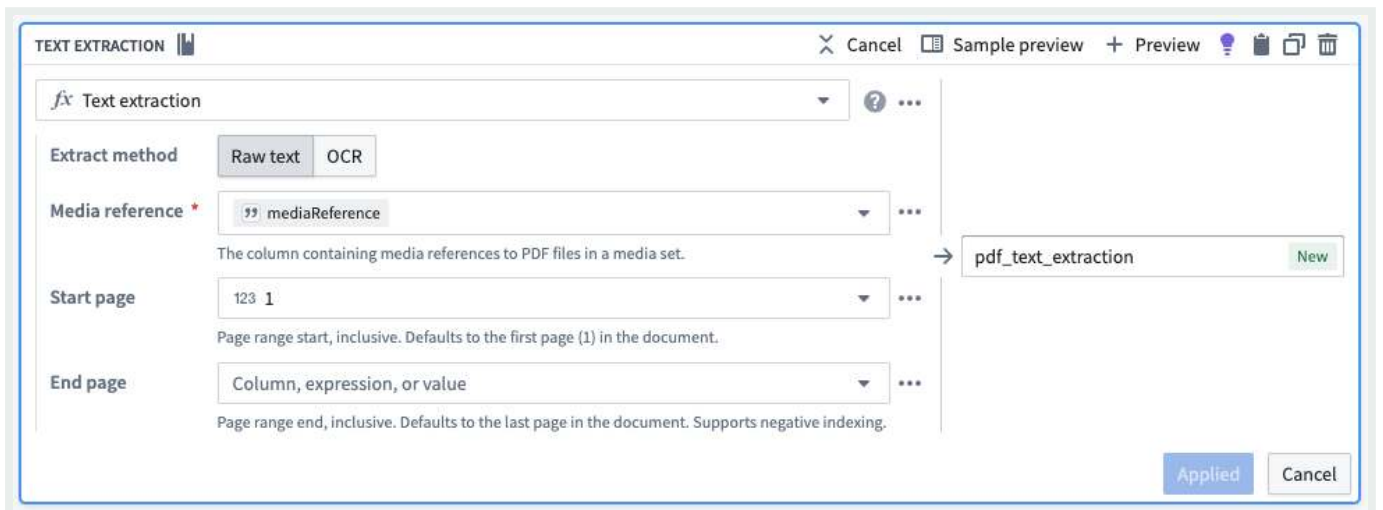
API Reference ↗



Send feedback



4. Use the **Text Extraction** board.



Chunking

This page outlines how to incorporate a basic chunking strategy into your [semantic search](#) workflows. Chunking, in this context, means breaking up larger pieces of text into smaller pieces of text. This is advantageous because embedding models possess a maximum input length for text, and crucially, smaller pieces of text will be more semantically distinct during searches. Chunking is often used when parsing large documents like [PDFs](#).

Primarily, the objective is to split long text into smaller "chunks", each with an associated Ontology object [linked](#) back to the original object.

i Where does chunking happen?

Chunking is performed by the **Chunk string** board in [Pipeline Builder](#). Add the board from the transform menu, then configure the following arguments:

API Reference ↗

- **Expression:** The text column you want to split into chunks.
- **Chunk size:** The approximate number of characters in each

Send feedback

- **Chunk overlap:** The number of characters each chunk shares with the previous chunk, which helps preserve context across chunk boundaries.
- **Separators:** An ordered, prioritized list of separators the board uses to split the text. The board splits on the highest priority separator first and only moves to the next separator when a chunk is still larger than the chunk size.

For the full argument reference, see the [Chunk string](#) function documentation.

Chunking example

As a starting point, we will show how a basic chunking strategy can be accomplished without using code in [Pipeline Builder](#). For more advanced strategies, we recommend using a [code repository](#) as part of your pipeline.

For illustrative purposes, we will use a simple two row dataset with two columns, `object_id` and `object_text`. For ease of understanding, the `object_text` examples below are purposefully short.

<code>object_id</code>	<code>object_text</code>
abc	gold ring lost
xyz	fast cars zoom

We initiate the process by employing the **Chunk String** board, which introduces an extra column containing an array of `object_text` segmented into smaller pieces. The board accommodates various chunking approaches, such as overlap and separators, to ensure that each semantic concept remains coherent and unique.

The below screenshot of a **Chunk String** board shows a simple strategy which you may alter for use toward your own use case. The below configuration would attempt to return chunks that are roughly 256 characters in size. Effectively, the board splits text on the highest priority separator until each chunk is equal to or smaller than the chunk size. If there are no more highest priority separators to split on and some chunks are still too large, it moves to the next priority separator until either all the chunks are equal or smaller than the chunk size or all separators are used. Finally, the board will ensure that for each chunk identified, the chunk following has an overlap that covers the last 20 characters of the previous chunk.

[API Reference ↗](#)

[Send feedback](#)

CHUNK STRING

fx

Chunk string

?

...

Expression *

'' object_text

...

The column containing the document text to chunk.

Chunk size

256

^

v

Create chunks of size around this number. Must be greater than 0.

Chunk overlap

20

^

v

Allow chunks to overlap in content by around this number. Must be greater than or equal to 0 and less than the selected chunk size.

Separators

\n\n

...

x

New line character (\n)

...

x

1 space

...

x

Empty string

...

x

+ Add value

Chunk string on these provided separators. The default separators has the effect of trying to keep all paragraphs, and then sentences, and then words together for as long as possible.

Keep separator ?

True

False

No value

chunks

New

Applied

Cancel

object_id	object_text	chunks
abc	gold ring lost	[gold,ring,lost]
xyz	fast cars zoom	[fast,cars,zoom]

Next we want each element in the array to have its own row. We will use the **Explode Array with Position** board to transform our dataset to one with six rows. The new column in each of the rows (as seen below) is a struct (map) with two key-value pairs, the position in the array and the element in the array.

EXPLODE ARRAY WITH POSITION

fx

Explode array with position

?

...

Array *

{-} chunks

...

Array of values to explode.

Keep empty / null arrays ?

True

False

No value

chunks_with_position

New

Applied

Cancel

API Reference ↗

object_id	object_text	chunks	chunks_with_position
abc	gold ring lost	[gold,ring,lost]	{position:0, element:gold}
abc	gold ring lost	[gold,ring,lost]	{position:1, element:ring}

Send feedback

object_id	object_text	chunks	chunks_with_position
abc	gold ring lost	[gold,ring,lost]	{position:2, element}
xyz	fast cars zoom	[fast,cars,zoom]	{position:0, element}
xyz	fast cars zoom	[fast,cars,zoom]	{position:1, element}
xyz	fast cars zoom	[fast,cars,zoom]	{position:2, element}

From there, we will pull out the position and the element into their own columns.

GET STRUCT FIELD

fx Get struct field

Struct *

chunks_with_position

Locator *

position

position

AppliedCancel

GET STRUCT FIELD

fx Get struct field

Struct *

chunks_with_position

Locator *

element

chunk

AppliedCancel

object_id	object_text	chunks	chunks_with_position	position	ch
abc	gold ring lost	[gold,ring,lost]	{position:0, element}	0	gol
abc	gold ring lost	[gold,ring,lost]	{position:1, element}	1	rin
abc	gold ring lost	[gold,ring,lost]	{position:2, element}	2	los
xyz	fast cars zoom	[fast,cars,zoom]	{position:0, element}	0	fas
fast cars		[fast,cars,zoom]	{position:1, element}	1	ca
xyz	fast cars zoom	[fast,cars,zoom]	{position:2, eleme		oc

API Reference ↗

Send feedback

To create a unique identifier for each chunk, we will convert the chunk position in its array to a string and then concatenate it to the original object ID. We will also drop the unnecessary columns.

CAST

fx Cast to String

?

...

Expression *

position

...

Type *

String

position

Replace

Applied

Cancel

CONCATENATE STRINGS

fx Concatenate strings

?

...

Separator

_

...

Separator to be added between the strings.

Expressions *

?

object_id

...

...

position

...

...

Add item

chunk_id

New

Applied

Cancel

DROP COLUMNS

Columns to drop *

?

position

chunks_with_position

chunks

object_text

Search for columns...

Applied

Cancel

object_id	chunk	chunk_id
abc	gold	abc_0
abc	ring	abc_1
abc	lost	abc_2
xyz	fast	xyz_0
xyz	cars	xyz_1
API Reference ↗		xyz_2

Now, we have six rows representing six different chunks, each with the original object ID (for linking), the new `chunk_id` to be a new primary key, and the `chunk` to be embedded as

Send feedback

described in [semantic search workflow](#). This results in the table as follows:

object_id	chunk	chunk_id	embedding
abc	gold	abc_0	[-0.7,...,0.4]
abc	ring	abc_1	[0.6,...,-0.2]
abc	lost	abc_2	[-0.8,...,0.9]
xyz	fast	xyz_0	[0.3,...,-0.5]
xyz	cars	xyz_1	[-0.1,...,0.8]
xyz	zoom	xyz_2	[0.2,...,-0.3]

Ontology building / Semantic search / Process multimodal and embedding models

Process multimodal and embedding models

This page discusses some methods you can use to process multimodal and embedding models.

Multimodal models

If you want to answer questions based on diagrams, LLMs with the text-in-text-out architecture will be of no help. While GPT-4o and GPT-4o mini are able to take image inputs, there are other open-source options available for your consideration.

- **Pix2Struct** [↗](#): Performed quite well during initial tests on quality assurance for a table in German. You can try it on [huggingface](#) [↗](#).
- **Microsoft UDOP (Universal document processing)** [↗](#): Open source, but not available on huggingface.

In this setup, you can use the initial text extraction just to have something to (semantic-)search for, but then later run the multimodal model on top of the raw source page (image).

Embedding models

API Reference [↗](#)

If you are working with English, you can try MSMARCO models from the [sentence-](#)

[t5](#)   [s docs](#) [↗](#).

Send feedback

[MS MARCO ↗](#) is a collection of large scale information retrieval datasets that were created based on real user search queries using the Bing search engine. The provided models can be used for semantic search, in that given keywords, a search phrase, or a question, the model will find passages that are relevant for the search query.

This means these models were **specifically trained to put queries and relevant passages close together in embedding space**.

By this definition, embedding models may be a better fit for semantic search workflows that start from a user query than general-purpose OpenAI Ada. When you use Ada to embed a query directly and compare that to chunk embeddings, you are not comparing the same concept and may instead use asymmetric embedding models to bridge that gap.

Alternatively, you can attempt using an [LLM to get generate a hypothetical chunk](#) first.

Ada, in turn would make more sense when your starting point is a chunk, and you are searching for similar chunks. Note that most non-ada embedding models only support 512 tokens, so you need to adapt your chunking strategy accordingly.

If you are working in German, for example, GPT is currently the only LLM that performs decently for the language. With a German document corpus, try ada.

PREVIOUS

← [Document processing](#)

NEXT

[Ontology augmented generation](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Semantic search / Ontology augmented generation

Ontology-augmented generation

LLMs are immensely powerful when applied to business-specific context. When presented with a certain task, the first step is almost always to find the relevant context that should be given to the LLM. Finding the relevant context is often the most challenging part of designing a retrieval augmented generation system. This section outlines some common approaches for context retrieval. Note there is no singular "best approach", as the best solution will be highly dependent on the specifics of the data. However, the themes outlined here are a good starting point and can be modified and combined as appropriate.

With new model generations' increased context lengths, you may not need to use semantic search at all and can instead pass the full context in the prompt. For example, GPT-4o's 128k context window corresponds to 300+ pages of text. If your application's full context is within this limit, we recommend you start without search.

Basic semantic search

To create a basic semantic search, do the following:

1. Follow a [chunking](#) strategy.
2. Create the chunk objects with a [media reference](#) property.
3. Search for the chunk as part of a [semantic search workflow](#).
4. Use the [PDF Viewer widget](#) in Workshop, noting the configuration options.

Embeddings

API Reference ↗

For more information on embeddings, [review the documentation on how to use the Embedding model to create a semantic search workflow.](#)

Send feedback

Retrieval

For more information on retrieval, [review the documentation on using a Palantir-provided model to create a semantic search workflow.](#)

Advanced retrieval

If you are finding that your AIP-associated tool is failing to answer questions that should be found in the document corpus, you should first investigate whether the relevant context was retrieved and passed into the prompt. Often it is the retrieval step that fails to surface the most relevant context leading the LLM to respond appropriately in the subsequent step.

Many approaches exist to improve the retrieval depending on the content of the data and queries, some of which are outlined below:

- **HyDE (Hypothetical Document Embeddings)**
- **Ranked keyword-based search**
- **Query augmentation**
- **Hybrid search**

Technique consideration

This will ultimately depend on your specific use case requirements and how much time you are willing to invest. Depending on your use case, it could be that the original simple setup works well enough for you. Otherwise, you may just need to add HyDE and semantic chunking and leave the rest as it is. Our recommendation would be to start with the basic implementation, and then add features as it becomes necessary.

HyDE (Hypothetical document embeddings)

One approach to improve retrieval performance is HyDE - otherwise known as Hypothetical document embeddings. The principle idea is that instead of embedding the query directly, you first ask an LLM to produce a hypothetical chunk that answers the question, which you then embed. Intuitively, this helps balance out the asymmetry between

API Reference ↗

Send feedback

answer. You can also review the related academic journal titled "[Precise Zero-Shot Dense Retrieval without Relevance Labels](#)" ↗

This can be particularly helpful in specific cases where chunks are formatted in a particular way to encode the [origin document and chapter](#).

As an example, consider the following chunk as an appropriate answer to the question: “How do we deal with animal collisions?”:

```
Claim Management – Motor: Animal Collision:  
Animal collision claims are generally covered in type A, B, D policies. However,  
exclusions apply...
```

We would first prompt an LLM to generate a hypothetical chunk like so:

```
You are an insurance specialist assistant tasked to assist your colleagues with  
finding relevant documents for their queries.  
  
Given the following user query:  
{query}  
  
Produce a hypothetical paragraph answering it. Give your response in the following  
format:  
{Document Name}: {Chapter name}: {Section} > ...  
{Content}  
  
where {Document Name} is the name of the document that contains the passage,  
{Chapter name} is the name of the chapter, {Section} is the name of the section  
and {Content} is the content of the section.
```

This prompt would return us a response such as the following:

```
API Reference ↗ Claim Management: General Terms:  
is commonly insured in fully comprehensive packages...
```

Send feedback

As the LLMs response is already “closer” to the real answer (structurally), it makes its embedding closer to the chunk that contains this real answer. Our semantic search in a function would then look like the following:

```
1 async searchChunksByEmbedding(query: string, k: Integer): Promise<Chunk[]> {
2   // create the full prompt for the hypothetical
3   const prompt = `...`
4
5   // generate hypothetical chunk
6   const hypothetical = await GPT_4o.createChatCompletion({messages: [{"r
7   // embed the result
8   const embedding = await TextEmbeddingAda_002.createEmbeddings({inputs:
9   // use the embedding in the nearest neighbor search
10  const docs = Objects.search()
11    .chunks()
12    .nearestNeighbors(chunk => chunk.vectorProperty.near(
13    .orderByRelevance()
14    .takeAsync(k)
15  return docs
```

Ranked keyword-based search

Generic embedding models such as OpenAI Ada are trained on a large corpus of diverse data. If your use case requires search on a domain-specific corpus (for example, manufacturing), you may find that the retrieval does not work as well as expected. This is due to the generic embedding model only using a small part of the embedding space for a specific domain.

Fine-tuning a custom model is one approach to improve retrieval in these cases, however, a much simpler out-of-the-box solution is to use a ranked keyword search, with potentially some LLM preprocessing.

This is because the index that Object Storage v2 runs on already comes with a notion of “relevance” when given a query. This relevance is *relative to other chunks*, meaning it automatically considers the domain-specific context of a chunk.

[API Reference ↗](#)

Functions on Objects support ordering the results of Object queries by said relevance so you can write a function like the following:

[Send feedback](#)

```

1  async searchChunksByKeywords(query: string, k: Integer): Promise<Chunk[]> {
2      const chunks = Objects.search()
3          .chunks()
4          .filter(chunk => chunk.text.matchAnyToken(query))
5          .orderByRelevance()
6          .takeAsync(k)
7      return chunks
8  }

```

However, this method abstracts away the semantic element of a semantic search. For example, if a user asks “How do we deal with **deer** collisions?” and we just input that directly into the function, we would not find chunks that talk about **animal** collision in general. LLMs can bring the semantic element back in through [query augmentation](#), described below.

Query augmentation

Query pre-processing is an important step to maximize relevancy of returned results. In essence, you want to distill the user query into its core components dependent on the type of search. You can consider [query enriching](#) and the other is [query extraction](#).

Query enriching

Injecting an LLM step between the user query and what is passed to the keyword search allows the possibility of distilling the query to make it more relevant: the LLM can be prompted to remove stopwords and irrelevant filler phrases (“Help me find ...”), and add other related words or synonyms.

You can set up a prompt like the following:

You are an insurance AI assistant tasked to help users find relevant documents. To do so, you can use keyword-search in the company's internal database.

Given the following user query: {query}

API Reference ↗

Give a list of search terms that would find relevant results. Be concise.

Send feedback

and add synonyms and related terms to the most important terms.
Give your answer as a list of comma-separated values.

For our example question of "How do we deal with animal collisions?", the LLM's response would be:

deer, animal, collision, claims, wildlife, accidents, vehicle, damage, car, insu

This would allow users to find documents that never mention "deer collisions", but also those that talk about "animal", "wildlife", and "accidents" in general.

In code:

```
1 async searchChunksByAugmentedKeywords(query: string, k: Integer): Promise<SearchResults> {
2   // create the full prompt for the query augmentation
3   const prompt = `...`
4
5   const augmentedQuery = await GPT_4o.createChatCompletion({messages: [
6
7   const chunks = Objects.search()
8     .chunks()
9     .filter(chunk => chunk.text.matchAnyToken(augmentedQuery))
10    .orderByRelevance()
11    .takeAsync(k)
12   return chunks
13 }
```

Query extraction

Query augmentation works well for relevance-ordered keyword search. For semantic search, however, you need to extract the core ask of the user query, and by doing so, remove extra terms that provide no semantic meaning such as stop words, and potentially [lemmatizing or stemming](#) query terms.

To [API Reference](#) query extraction to convert a question to the key ask of the user.

An example prompt could be:

Send feedback

You are preparing a user-given query in order to perform a semantic search. Extract the key user actions from the given query, removing unnecessary stop words in the process.

Given the following user query: {query}

Return concatenated actions delimited by full stops.

For our example question of “How do we deal with animal collisions?”, the LLM would return:

Deal with animal collisions.

This response maximizes the semantic content of our query and increasing likelihood of stronger matching downstream once we run a semantic search. The above example could also be solved by removing stopwords only.

```
1 async searchChunksByExtractedQuery(query: string, k: Integer): Promise<Ch
2     // create the full prompt for the query augmentation
3     const prompt = `...`
4
5     const augmentedQuery = await GPT_4o.createChatCompletion({messages: [
6     const embedding = await TextEmbeddingAda_002.createEmbeddings({inputs
7
8     const chunks = Objects.search()
9         .chunks()
10        .nearestNeighbors(obj =>obj.embeddingProperty
11        .near(embedding, { kValue: k }))
12        .allAsync()
13    return chunks
14 }
```

Hybrid search: Combining semantic and keyword search with reciprocal

[API Reference ↗](#)

Reciprocal rank fusion (RRF) is a simple algorithm to combine results of different search types into a single list. In essence, it gives a document a higher score the higher it is ranked

[Send feedback](#)

in a given list. The total score is the sum of scores across lists.

k acts as a regularizer - the higher k , the less it matters *where* a document appears in a list, but merely *that* it appears in the list at all.

```
`RRFscore(d ∈ D) = Σ [1 / (k + r(d))]
```

```
`# k is a constant that helps to balance between high and low ranking.`
```

```
`# r(d) is the rank/position of the document`
```

```
1 public combineResultsWithRRF(vectorSearchResults: Chunk[], keywordSearchResults: Chunk[]) {
2
3     // define the RRF scoring function
4     const RRF = (r: number, k: number) => 1 / (r + k);
5
6     // initialize a map to keep track of the scores of each chunk
7     // note we assume later that each Chunk has a string primary key property
8     const resultMap: Map<string, {chunk: Chunk, score: number}> = new Map();
9     const combinedResults: Chunk[] = [];
10
11     const searchResultsList = [vectorSearchResults, keywordSearchResults];
12
13     searchResultsList.forEach((searchResults) => {
14         searchResults.forEach((chunk, rank) => {
15             // calculate the score for each Chunk in the list
16             // and add it to the Chunk's total in the map
17             const rrfScore = RRF(rank, k);
18             const chunkData = resultMap.get(chunk.id) || {chunk: chunk, score: 0};
19
20             chunkData.score += rrfScore;
21             resultMap.set(chunk.id, chunkData);
22         });
23     });
24
25     // get all Chunks into a list
26     resultMap.forEach((chunkData) => {
27         combinedResults.push(chunkData.chunk);
28     });
29 }
```

API Reference ↗

```
combinedResults.sort((a, b) => resultMap.get(b.id).score - resultMap.get(a.id).score);
```

Send feedback

```
33     return combinedResults;
34 }
```

A full hybrid search implementation would then look like the following:

```
1  async hybridSearch(query: string, k: Integer, n1: Integer, n2: Integer): Promise<SearchResults> {
2
3      // Start the keyword and vector searches in parallel
4      const keywordSearchResultsPromise = searchChunksByKeywords(query, n1);
5      const vectorSearchResultsPromise = searchChunksByEmbedding(query, n2);
6
7      const [keywordSearchResults, vectorSearchResults] = await Promise.all([
8          keywordSearchResultsPromise,
9          vectorSearchResultsPromise
10     ]);
11     const rerankedResults = combineResultsWithRRF(vectorSearchResults, keywordSearchResults);
12     return rerankedResults.slice(0, k);
13 }
```

PREVIOUS

← [Process multimodal and embedding models](#)

NEXT

[Use Palantir-provided models to create a semantic search workflow](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Semantic search / Use Palantir-provided models to create a semantic search workflow

Use Palantir-provided models to create a semantic search workflow

i To use Palantir-provided language models, [AIP must first be enabled on your enrollment](#). You also must have permissions to use [AIP developer capabilities](#). Using a custom model? Review [Using custom models to create a semantic search workflow](#) instead.

This page illustrates the process of building a notional end-to-end semantic search workflow using a [Palantir-provided embedding model](#).

Instructions

To begin, you need to generate embeddings and store them in an object type with a [vector type](#). Then, you can set up a semantic search workflow in [Workshop](#), build an [AIP Chatbot Workshop widget](#) solution, or create a custom semantic search function for use in [Workshop](#) and [AIP Logic](#).

Prerequisite:

- [Generate embeddings and create object type](#)

API Reference ↗



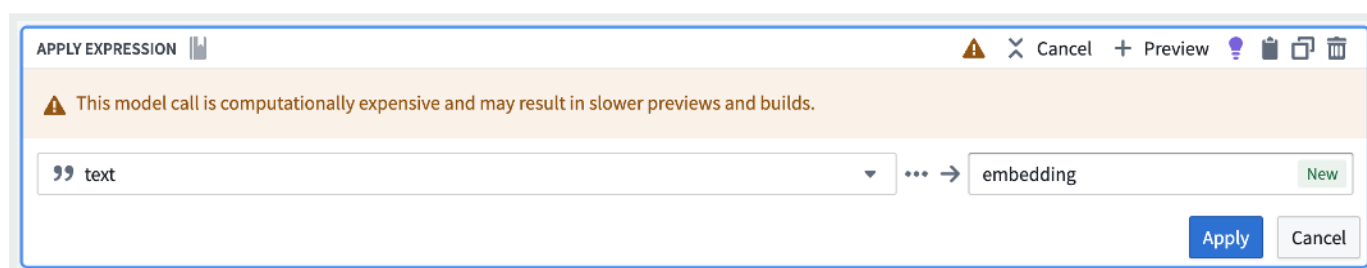
[simple semantic search workflow within Workshop using code](#)

Send feedback

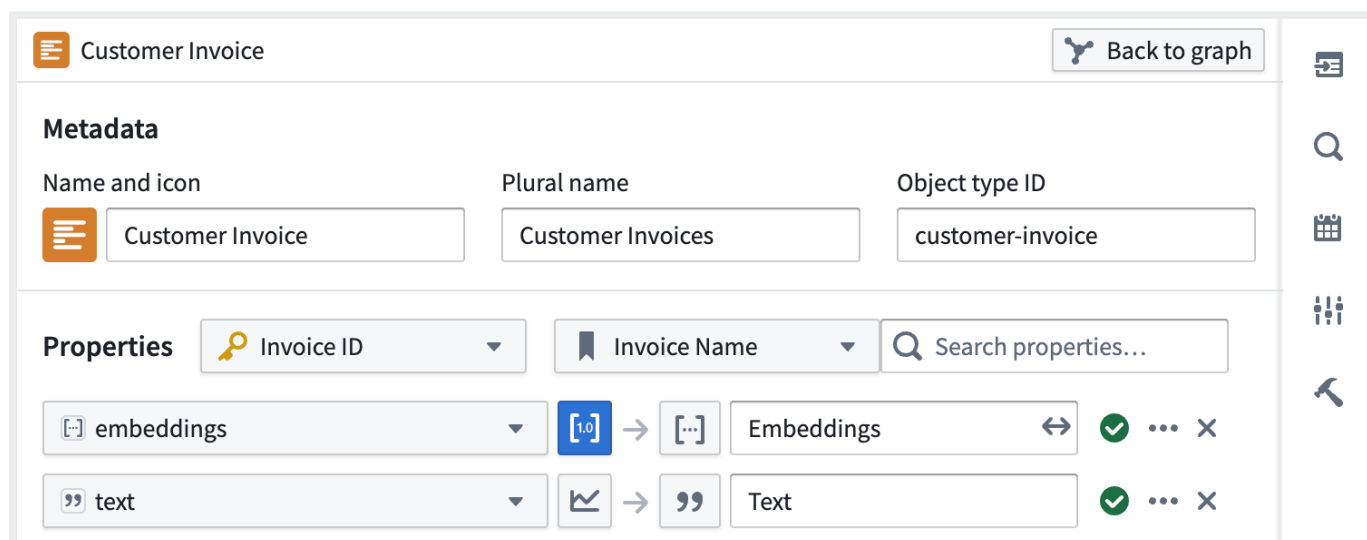
- [Enable an AIP Chatbot to semantic search for objects \(no-code\)](#)
- [Create a function to semantically search across objects for use in Workshop and/or AIP Logic](#)

Generate embeddings and create object type

We will use [Pipeline Builder](#) to embed text in the dataset as vectors with the [Text to Embeddings expression](#). The expression takes a string and converts it to a vector using one of the Palantir-provided models; in our case, this is the `text-embedding-ada-002` embedding model.



These embeddings can then be added to the Ontology as a vector property.



If you would like more control around the generation of embeddings using Palantir-provided models, see [Language models within Python transforms](#).

[API Reference ↗](#) [Example semantic search workflow within Workshop using a KNN object set \(no-code\)](#)

[Send feedback](#)

i The KNN object set cannot be sorted by relevancy. If you need ordered results, use the [function approach](#).

Configuring a [KNN object set](#) within [Workshop](#) is an easy no-code way to build a semantic search workflow.

1. Create an object set [variable](#) and select the object type that contains an embedding property.
2. Select the filter **+ On a property** option, then from the list of properties in the menu, select your embedding property.
3. Once selected, the K-nearest-neighbors configuration should appear. If this configuration does not appear, verify that the property you selected is an embedding property.

The screenshot shows the configuration interface for a KNN object set. At the top, there's a header with 'var1' and an 'Object' button. Below this are two tabs: 'Definition' and 'Settings'. The 'Settings' tab is active. The main configuration area includes:

- Starting object set:** A dropdown menu showing 'public docs'.
- Embedding:** A dropdown menu showing 'K-nearest neighbors' with a help icon.
- Query:** A text input field with 'Query' and a quote icon.
- K value (1-100):** A numeric input field with '3' and up/down arrows.
- Filter...** A section with three options: '+ On a property', '(x) Using a variable', and 'On a link'. The 'Using a variable' option is selected.
- API Reference** A button with a right arrow.
- Traverse to:** A section with a 'Get linked objects' option.

Send feedback

Within this panel, you can configure:

- **K-value:** A number between 1-100 for how many objects to return in the semantic search.
 - **Query:** The string variable to use as a query when performing the semantic search.
4. Next, create a [text input](#) widget and add its output variable to the KNN query option seen above.
5. Lastly, add an [object table](#) widget and configure its input variable to be the newly created [KNN object set](#).

SEMANTIC SEARCH

android

Documentation

Foundry Docs - Workshop - Widgets Mobile Navbar.md # Widget: Mobile Navigation Bar

Foundry Docs - Workshop - Mobile App Launcher.md # Mobile application launcher

Foundry Docs - Announcements - 2023 01.md # Announcements ## Workshop: On-the-go with Mobile Apps ##### Physical data enrichment

For more customized semantic search logic, see the [section on functions](#).

[API Reference ↗](#)

Use AIP Chatbot (no-code)

Send feedback

AIP Chatbots (formerly AIP Agents) created in [AIP Chatbot Studio](#) are good for beginning semantic searches across your objects because they do not require any code. Learn more about [incorporating semantic search with more control over the functionality](#).

Follow the instructions on the [getting started](#) guide to create an AIP Chatbot and either add [Ontology context](#) or an **Ontology semantic search tool**. This initial setup will enable you to ask the AIP Chatbot to semantically search the objects.

Create a function to semantically search across objects for use in Workshop and/or AIP Logic

We can [create a typescript repository](#) and create a function to query our object type. The overall goal is to be able to take some user input, generate a vector using the same Palantir-provided model [used earlier](#), and then do a [KNN search](#) over our object type. For more information on how to import Palantir-provided models, review [Language models within Functions](#).

Substitutions

In the code snippet below, replace every instance of `ObjectApiName` for your unique ObjectType. Note that the identifier may sometimes appear as `objectApiName` with the first letter in lowercase.

Enabling vector properties for functions

Before proceeding, ensure that the entry `"enableVectorProperties": true` is present in the `functions.json` file in your Functions code repository. If this entry is not present, add it to `functions.json` and commit the change to proceed. Contact your Palantir representative if you need further assistance.

for `typescript/src/index.ts`

API Reference ↗

Send feedback

```

1 import { Function, Integer } from "@foundry/functions-api";
2 import { Objects, ObjectApiName } from "@foundry/ontology-api";
3 import { TextEmbeddingAda_002 } from "@foundry/models-api/language-models";
4
5 export class MyFunctions {
6     @Function()
7     public async findRelevantObjects(
8         query: string,
9         kValue: Integer,
10    ): Promise<ObjectApiName[]> {
11        if (query.length < 1) {
12            return []
13        }
14        const embedding = await TextEmbeddingAda_002.createEmbeddings({input: query});
15
16        return Objects.search()
17            .objectApiName()
18            .nearestNeighbors(obj => obj.embeddings.near(embedding))
19            .orderByRelevance()
20            .take(kValue);
21    }
22 }

```

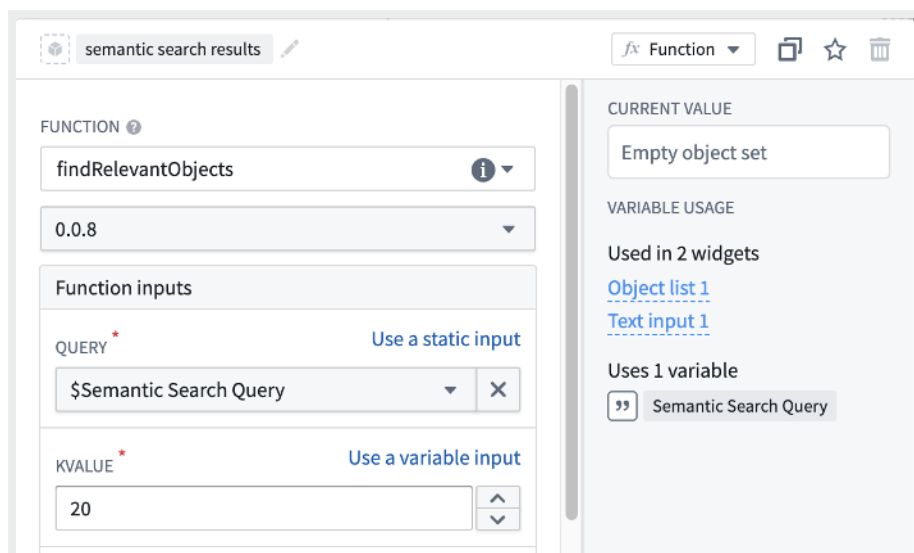
At this point, we have a function that can run semantic search to query objects with natural language. Remember to [publish the function](#) so the function can be used anywhere within Foundry.

Use semantic search functions in Workshop

1. Start by [creating a Workshop application](#).
2. Add a [text input widget](#), which will be used as an input to the published KNN document fetch function.
3. Add an [object list widget](#) with an input [object set generated from the function](#) and the selected inputs as shown below:

API Reference ↗

Send feedback



4. Set the `kValue` to however many results you want returned, subject to the [specified limits](#).

Use semantic search functions in AIP Logic

Add the published function as a [tool](#) within AIP Logic. Instruct the language model to use the tool with a prompt similar to this:

Use the fetchRelevantObjects tool with a kValue of 5 to find the most related objects. Remember to add quotes around query when using the tool.

← PREVIOUS
Ontology augmented generation

Use custom models to create a semantic search workflow → NEXT

API Reference ↗

Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

Send feedback

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Semantic search / Use custom models to create a semantic search workflow

Using custom models to create a semantic search workflow

⚠ This tutorial is for those using an embedding model not supplied by Palantir, which is no longer a recommended workflow. See the list of [Palantir-provided models](#) and the [Palantir-provided model semantic search tutorial](#).

This page illustrates the process of building a notional end-to-end documentation search service that is capable of retrieving relevant docs when given a prompt. The service will use a [Foundry modeling objective](#) to embed documents and extract their features into a vector. These documents and embeddings will be stored in an object type with the vector property.

For this example, we begin by setting up a model in Foundry and creating a pipeline to generate embeddings. Then, we will create a new object type and a function to query it through natural language.

We begin with a dataset that currently has our parsed documents and metadata, such as `Document_Content` and `Link`. Next, we will generate embeddings from the `Document_Content` to enable us to query them via semantic search.

	UID ▾ Dec...	Category ▾ String	Document_Content ▾ String	Link ▾ String
1	3	Data Integration	Similar to a dataset, a st	/data-integration
		API Reference ↗ ology ology	Ontology proposals allow y An object type is the sche	/ontologies/onto /object-link-type
		4 Data Integration	As you author data trans	
	5	Security	You can check someone's	per/security/checki

Send feedback

To understand the details of the KNN feature, review [KNN Functions on Objects](#) section in the Foundry documentation.

Value substitution

Throughout this workflow you can substitute a value of your choosing, as long as it is consistent for each instance. For example, every instance of `ObjectApiName` is always substituted with `Document`.

The values you must substitute are:

- `ObjectApiName` : identifier for a unique ObjectType, in our case `Document`. *NOTE:* The identifier may sometimes appear as `objectApiName` with the first letter lowercased.
- `ModelApiName` : identifier for a function wrapping a Model
- `OutputDatasetRid` : identifier for the output dataset from the [embedding transform](#).
- `InputDatasetRid` : identifier for the input dataset for the [embedding transform](#).
- `ModelRid` : identifier for the model used for the [embedding transform](#) and in the [creation of the Live Modeling Deployment](#)

1. Create embeddings using models in Foundry

There are a few options for creating embeddings from a model in Foundry. In this example, we will create a transform to interact with an [imported open-source model](#). We will use the `all-MiniLM-L6-v2` model, a general purpose text-embedding model that will create vectors of dimension (size) 384. This model can be swapped out with any other existing model that outputs vectors compatible with the [Foundry Ontology vector type](#). To import a new open-source model, review our [Hugging Face model documentation](#).

The code below expects the model to expose an API with a tabular input containing a `text` string column and a column for tabular outputs containing an `embedding` list of floats. For more details on defining model APIs, refer to the [model adapter API documentation](#).

API Reference ↗

```
1 import palantir_models as pm
2
3
```

Send feedback

```

4 class EmbeddingModelAdapter(pm.ModelAdapter):
5     ...
6
7     @classmethod
8     def api(cls):
9         inputs = {
10             "inference_data": pm.Pandas(columns=[("text", str)])
11         }
12         outputs = {
13             "output_data": pm.Pandas(columns=[("text", str), ("embedding",
14             )])
15         }
16         return inputs, outputs

```

The transform below runs the data through the model to return an `embedding`, then casts the `embedding` value (double arrays) to floats in order to match the type necessary for vector embeddings.

A couple of points to consider:

- Each `StructField` in the `schema` variable relates to a columns that are present in the processed input dataset (`InputDatasetRid`) plus the `embedding` column added by the model.
- When working with data at larger scales, the transform might fail if using a Pandas dataframe that is excessively large. In these cases, the transform will have to be performed in Spark.
- Graphics Processing Units (GPUs) can be leveraged to increase the speed at which embeddings are produced by a transform. GPUs can be used by adding the `@configure` decorator to your transform. Contact your Palantir representative if you are interested in enabling this in your environment.

An example transform is shown below:

```

1 from transforms.api import configure, transform, Input, Output
2 from palantir_models.transforms import ModelInput
3 from pyspark.sql.functions import pandas_udf, PandasUDFType
4 from pyspark.sql.types import StructType, StructField, IntegerType, StringType
5
6 as np

```

API Reference ↗

```

7
8 @configure(profile=["DRIVER_GPU_ENABLED"]) # Remove the profile if you are not using GPUs
9 @transform(

```

Send feedback

```

10     dataset_out=Output("OutputDatasetRid"),
11     dataset_in=Input("InputDatasetRid"),
12     embedding_model=ModelInput("ModelRid")
13 )
14 def compute(ctx, dataset_out, dataset_in, embedding_model):
15     # Match input column of model
16     spark_df = dataset_in.dataframe().withColumnRenamed("Document_Content", "text")
17
18     def embed_df(df):
19         # Create embeddings
20         output_df = embedding_model.transform(df).output_data
21         # Cast to float array
22         output_df["embedding"] = output_df["embedding"].apply(lambda x: np.array(x).astype(float))
23         # drop unnecessary column
24         return output_df.drop('inference_device', axis=1)
25
26     # Updated schema
27     schema = StructType([
28         StructField("UID", IntegerType(), True),
29         StructField("Category", StringType(), True),
30         StructField("text", StringType(), True),
31         StructField("Link", StringType(), True),
32         StructField("embedding", ArrayType(FloatType()), True)
33     ])
34
35     udf = pandas_udf(embed_df, returnType=schema, functionType=PandasUDFType.SCALAR)
36     output_df = spark_df.groupBy('UID').apply(udf)
37
38     # Write the output DataFrame
39     dataset_out.write_dataframe(output_df)

```

Next, we will need a Live Modeling Deployment to create embeddings off of a user query to be used to search against our existing vectors. The model used in this part should be the same as the one used to generate the initial embeddings in this current step.

2. Create object type

By now, we should have a new dataset with a column containing float vector embeddings generated from a batch modeling deployment [from our first and previous step](#). Next, we will create an object type.

API Reference ↗

Send feedback

We will name the object type `Document` , and set the `embedding` property to be of property type `Vector` . This requires configuring two values:

1. **Dimension:** this is the length of the array produced in the column `embedding` .
2. **Similarity Function:** the method by which distance between two `embedding` values from different objects will be calculated.

OVERVIEW

Name

Embedding

Description

Type

[1.0] Vector ▼

DIMENSIONS ?

384

^ ▼

SIMILARITY FUNCTION ?

Cosine Similarity ▼

Once this object type is created, we will have a property (`embedding`) that can be used to semantically search through the `Documentation` objects.

The `ApiName` will be available after the object type is saved, and can be found on the [API Reference](#) ⤵ page for the object type created. More information can be found about this on the [Create an object type](#) section of the documentation

Send feedback

3. Create a Live Modeling Deployment

Now that our objects have embeddings as a property, we need to generate embeddings for user queries with low-latency. These embeddings will be used to find objects with similar embedding values. To do this, create a live model deployment for fast, low-latency access with Functions.

Review the [instructions for configuring a live deployment in Modeling Objectives](#) or [directly from a model](#). A [Function then needs to be published](#) for that model.

4. Create an embedding with Functions on Models

⚠ Enabling Vector properties for functions

Before proceeding, ensure that the entries `"enableVectorProperties": true`, `"enableResourceGeneration": true`, and `"useDeploymentApiNames": true` are all present in the `functions.json` file in your Functions code repository. If these entries are not present, add them to `functions.json` and commit the change to proceed. Contact your Palantir representative if you need further assistance.

The final step is to create a function to query this object type. For the search phase, the overall goal is to be able to take some user input, generate a vector using the live modeling deployment [created earlier](#), and then do a [KNN search](#) over our object type. A sample function for this use case is shown below, including the file structure they should reside within.

Edits to vector properties can be applied by Actions and Functions.

Further information on how to use a model in a Function can be found in the [Functions on models documentation](#).

```
|-- functions-typescript
|   |-- src
|   |   |-- tests
|   |   |   |-- index.ts
|   |   |-- index.ts
|   |   |-- semanticSearch.ts
|   |   |-- service.ts
|   |   |-- tsconfig.json
|   |   |-- types.ts
|   |-- functions.json
|   |-- jest.config.js
|   |-- package-lock.json
|   |-- package.json
|-- version.properties
```

functions-typescript/src/types.ts

```
1  import { Double } from "@foundry/functions-api";
2
3  export interface IEmbeddingModel {
4      embed: (content: string) => Promise<IEmbeddingResponse>;
5  }
6
7  export interface IEmbeddingResponse {
8      text: string
9      embedding: Double[]
10     inference_device?: string
11 }
12
13 export interface IEmbeddingRequest {
14     text: string
15 }
```

f [API Reference ↗](#) escript/src/service.ts

Send feedback

```

1 // View the Model in the repository's Resource imports sidebar to know what
2 import { ModelApiName } from "@{YOUR_NAMESPACE_HERE}/models";
3 import { IEmbeddingRequest, IEmbeddingResponse } from "./types";
4
5 // service to hit model
6 export class EmbeddingService {
7     public async embed(content: string): Promise<IEmbeddingResponse> {
8         const request: IEmbeddingRequest = {
9             "text": content,
10         };
11         return await ModelApiName([request])
12             .then((output: any) => output[0]) as IEmbeddingResponse;
13     }
14 }

```

functions-typescript/src/semanticSearch.ts

```

1 import { Function, Integer, Double } from "@foundry/functions-api";
2 import { Objects, ObjectApiName } from "@foundry/ontology-api";
3
4 import { EmbeddingService } from "./service";
5 import { IEmbeddingResponse, IEmbeddingModel } from './types';
6
7 export class SuggestedDocs {
8     embeddingService: IEmbeddingModel = new EmbeddingService;
9
10    @Function()
11    public async fetchSuggestedDocuments(userQuery: string, kValue: Integer) {
12        const embedding: IEmbeddingResponse = await this.embeddingService.embed(userQuery);
13        const vector: Double[] = embedding.embedding;
14
15        return Objects.search()
16            .objectApiName()
17            .filter(obj => obj.category.exactMatch(category))
18            .nearestNeighbors(obj => obj.embedding.near(vector), kValue)
19            .orderByRelevance()
20            .take(kValue);

```

API Reference ↗

```

23 /**
24  * The following is an alternative to fetchSuggestedDocuments
25  * Otherwise, kValue number of documents are always returned, no matter

```

Send feedback

```

26     * The computation of the distance function depends on the distance f
27     * property. Here we assume it's cosine similarity, which can be comp
28     * product if the embedding model produces normalized vectors.
29     */
30     @Function()
31     public async fetchSuggestedDocumentsWithThreshold(userQuery: string,
32         const embedding: IEmbeddingResponse = await this.embeddingService
33         const vector: Double[] = embedding.embedding;
34
35         return Objects.search()
36             .objectApiName()
37             .filter(obj => obj.category.exactMatch(category))
38             .nearestNeighbors(obj => obj.embedding.near(vector,
39             .orderByRelevance()
40             .take(kValue)
41             .filter(obj => SuggestedDocs.dotProduct(vector, obj
42     }
43
44     private static dotProduct<K extends number>(arr1: K[], arr2: K[]): nu
45     if (arr1.length !== arr2.length) {
46         throw EvalError("Two vectors must be of the same dimensions")
47     }
48     return arr1.map((_, i) => arr1[i] * arr2[i]).reduce((m, n) => m +
49 }
50 }

```

functions-typescript/src/index.ts

```

1 export { SuggestedDocs } from "./semanticSearch";

```

5. Publish the function and use in an example

At this point, we have a function that can run semantic search to query objects with natural language. The final step is to [publish the function](#) and use it in a workflow. To continue

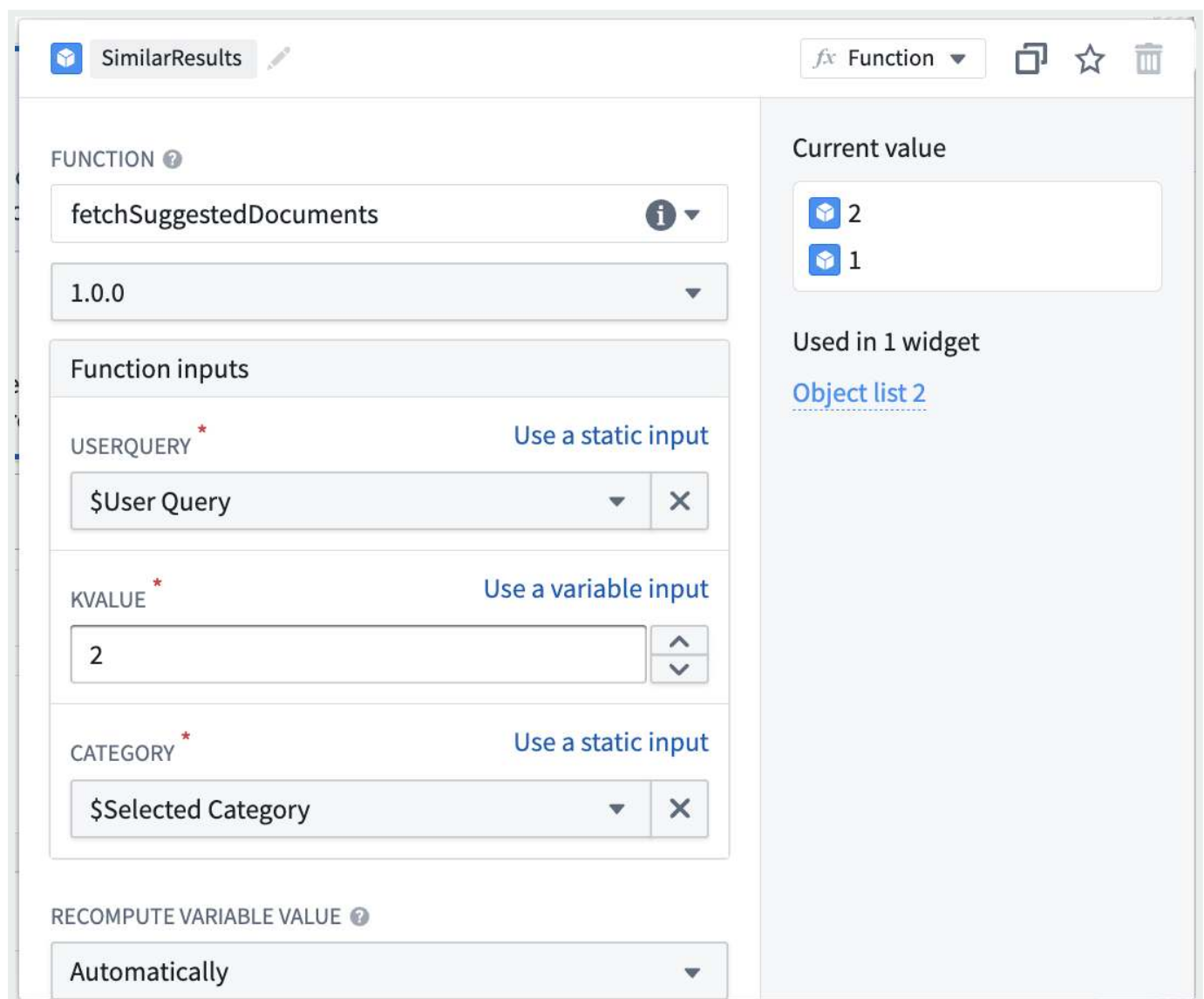
building the semantic search example, we will create a Workshop application to interact with a text input to return the top two matching documentation articles to a user.

API Reference ↗

Send feedback

The process to creating a semantic search for the documentation service in the example is as follows:

1. Start by [creating a Workshop application](#).
2. Add a [text input](#) and a [string selector](#). The string selector will be used to choose a documentation category with which to filter. Both the text input and string selector will serve as inputs into the published KNN document fetch function.
3. Finally, add an [object list widget](#) with an input object set generated from the function and the selected inputs as shown below:



From this point, the inputs will be used to semantically search through documents in the [object type](#) and return the two most relevant. This is just one simple use case of vector p [API Reference ↗](#) ntic search. See an example of the resulting Workshop application in the screenshot below:

Send feedback

SEARCH

How can I debug my failing streaming pipeline

CATEGORY

Select an
option...



Text •

As you author data transformation code in Foundry, you will likely run into cases where a job fails, either from the beginning or after some time. This page documents a suggested workflow for debugging f



Text •

Similar to a dataset, a stream is a representation of data from when it lands in Foundry through when it is processed by a downstream system. A stream is a wrapper around a collection of rows which are stored by both a persistent “hot buffer” and “cold storage” backed by a file

PREVIOUS

← [Use Palantir-provided models to create a semantic search workflow](#)

NEXT

[Search syntax](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Search syntax

This page describes syntax for different kinds of searches in the Ontology. You can search in the Ontology from Workshop using the [filter list](#) or Object Explorer from the [search bar](#).

Regular expression

Regular expression (regex) search in the Ontology uses a syntax that is similar to typical regular expressions but with some differences:

- String properties must be indexed for regex search. To use regular expression search on a string property, the property must be indexed for regex search in Ontology Manager. To configure this, navigate to the **Properties** tab of your object type and select the **Interaction** tab, then choose the **Enable regex queries** option.
- Patterns match the full value, not substrings. Because the index stores the complete string as a single unanalyzed value, a pattern is always matched against the entire field value from start to end. This means searching for `cat` will only match the exact value `cat`, not values that contain it such as `concatenate`. To perform a substring match, add `.*` before and after your pattern, for example, `.*cat.*` would match any value containing `cat`.
- `^` and `$` anchors are not supported. Because every match already implicitly starts at the beginning and ends at the end of the value, these anchors are redundant.

Supported operators

API Reference ↗

Single character.

☰

🔍

Searching for `c.t` would match `cat`, `cot`, `cut`, and so on.
`.` the previous character optional (matches zero or one time)

Send feedback

- Searching for `colou?r` would match both `color` and `colour`.
- `+` repeats the previous character one or more times.
 - Searching for `go+d` would match `god`, `good`, `goood`, and so on, but not `gd`.
- `*` repeats the previous character zero or more times.
 - Searching for `go*d` would match `gd`, `god`, `good`, `goood`, and so on.
- `{}` defines the minimum and maximum number of times the preceding character can repeat. `{2}` means the previous character must repeat exactly twice, `{2,}` means the previous character must repeat at least twice, and `{2,4}` means the previous character must repeat between 2 and 4 times (inclusive).
 - Searching for `go{2}d` would match only `good`. Searching for `go{2,4}d` would match `good`, `goood`, and `gooodd`.
- `|` is the OR operator, allowing you to match one pattern or another.
 - Searching for `cat|dog` would match either `cat` or `dog`.
- `()` forms a group within an expression so that operators can apply to the entire group rather than just the previous character.
 - Searching for `(un)?happy` would match both `happy` and `unhappy`, because the `?` makes the entire `un` group optional.
- `[]` matches any single character listed inside the brackets.
 - Searching for `gr[ae]y` would match both `gray` and `grey`. You can use `-` to define a range, so `[a-z]` matches any lowercase letter, `[0-9]` matches any digit, and `[A-Za-z]` matches any letter regardless of case. If the sequence begins with `^`, the set is negated, so `[^0-9]` matches any character that is not a digit. If `-` is the first character or escaped with `\`, it is treated as a literal dash.
- `"` creates groups of string literals, allowing you to match an exact phrase rather than interpreting each character as a regex operator.
 - Searching for `"v2.0"` would match the literal text `v2.0`. Without the quotes, the `.` would be treated as a wildcard, potentially matching `v2X0` or `v200`.
- `\` is used as an escape character, allowing you to search for characters that would otherwise be treated as operators. It also provides shorthand character classes:
 - `\d` matches any digit (`0-9`). `\D` matches any character that is not a digit (letters, punctuation, spaces, and so on).
 - `\s` matches any whitespace character, such as spaces, tabs, and newlines. `\S` matches any character that is not whitespace.
 - `\w` matches any word character (letters, digits, and underscores: `a-z`, `A-Z`, `0-9`, `_`). `\W` matches any character that is not a word character, such as punctuation, spaces, and so on.
 - For example, searching for `\d{3}-\d{4}` would match pattern `5678`. Searching for `\w+\s\w+` would match any two words separated by a space,

API Reference ↗

Send feedback

such as `hello world` or `John Smith`. To search for a literal dot, use `\.`. For example, `example\.``com` would match `example.com` without matching `exampleXcom`.

PREVIOUS

← [Semantic search / Use custom models to create a semantic search workflow](#)

NEXT

[Derived properties](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Derived properties [Beta]

Beta

Derived properties are in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development.

Derived properties are properties that are calculated at runtime based on the values of other properties or links on objects. This includes aggregating on or selecting properties of linked objects. Derived properties are then available for further operations, such as filtering, sorting, or aggregating within the same request. Derived properties use the security of all objects involved in the calculation, so they do not expose information a user would otherwise be unable to see.

Availability

Derived properties are available in the following workflows:

- **Ontology SDK:** Derived properties can be used in the TypeScript OSDK with the `withProperties` operation, to be returned or used in additional filters, aggregations, or sorts. The TypeScript OSDK must be running the `2.2.0-beta.x` version of the `@osdk/client` package or later. Review the API documentation in [Developer Console](#) for more details.

API Reference ↗ations

  At a time when the feature is under active development, derived properties currently have some capability limitations. Many of these capabilities will become available over time as

[Send feedback](#)

additional functionality is built. Current limitations are listed below:

- **OSv1 support:** Queries with derived properties may not contain any object types indexed using [OSv1](#).
- **Text search:** Derived properties cannot be used in text search or keyword filters.
- **Structs in OSDK:** Specifically in current versions of the Typescript OSDK, queries with derived properties may not contain any [struct](#) property types. You can use a `$select` operation in Ontology SDK to exclude struct properties.
- **Marketplace:** Functions using derived properties are not currently supported in Marketplace.

PREVIOUS

← [Search syntax](#)

NEXT

[Ontology scenarios / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Scenarios [Beta]

Beta

Ontology scenarios are in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development. Contact Palantir Support to request access to scenarios.

With Ontology scenarios, you can create and compare "what-if" analyses using objects and the ontological concept of a model. A scenario is essentially a sandbox to apply edits on top of the data in your Ontology, generated by applying one or more actions.

Common use cases

We recommend using Ontology scenarios for the following workflows:

- **What-if analysis:** Simulate changes to your data or system by creating an isolated fork of your Ontology, without affecting live data.
- **Business logic simulation:** Apply actions and functions to simulate business processes or evaluate the impact of operational changes before committing them.
- **Agent evaluation and forecasting:** Test agents against different input parameters to compare outcomes and refine predictions.
- **Comparative analysis:** Create multiple scenarios side by side to identify the optimal decision or understand the trade-offs between different strategies.

API Reference ↗



Scenarios are not data versioning tools. They cannot provide a snapshot of your Ontology at a historical point in time and should not be used for

Send feedback

Time to live

To manage storage costs, a scenario has a default lifespan of 30 days. After 30 days, the scenario is automatically deleted along with all edits applied to that scenario and the backing persisted scenario object.

Auto-rebasing

A scenario is auto-rebased every 10 minutes on top of their main or global branch. The rebase feature allows you to incorporate the latest changes from the base Ontology into your active scenario, keeping it in sync with the latest state of your data.

Global Branching vs. Ontology scenarios

[Global branching](#) is a tool used by builders to develop and test end-to-end workflows in the Palantir platform. These workflows would otherwise be too disruptive to run against a live production environment. An Ontology scenario provides a sandbox for users (humans and agents) of these workflows to apply edits to the Ontology without disrupting the main Ontology data. Users can create and compare multiple scenarios and merge edits to main Ontology data.

Functions on a scenario

All functions, except AIP Logic functions, can be run on a scenario as is. If the function reads from the Ontology, it will read the state of the Ontology on the scenario. If the function makes Ontology edits, it will make those edits on the scenario. Contact Palantir Support if you want to use AIP Logic on a scenario.

Limitations

[API Reference ↗](#)ing limitations when building out your workflow using scenarios:

[Send feedback](#)

- Because merging a scenario uses an action, [apply action limits](#) apply to merging a scenario.
- Scenarios are auto-rebased every 10 minutes on top of their base.
- Scenarios cannot be rebased at will or be configured with rebase intervals.

PREVIOUS

← [Ontology search / Derived properties](#)

NEXT

[Temporary scenarios](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Temporary scenarios

Temporary scenarios

Temporary scenarios are created and used only within the current session. Their metadata is not persisted in the platform and will be deleted if the user ends the current session (on Workshop, for example). These scenarios are useful for quick, ad-hoc "what-if" analyses where you do not need to collaborate or share the results. They allow you to experiment with different actions without affecting the state of your Ontology or cluttering your workspace with unnecessary saved scenarios.

Workshop tutorial

In this tutorial, you will walk through building a basic [Workshop](#) application supporting temporary scenarios.

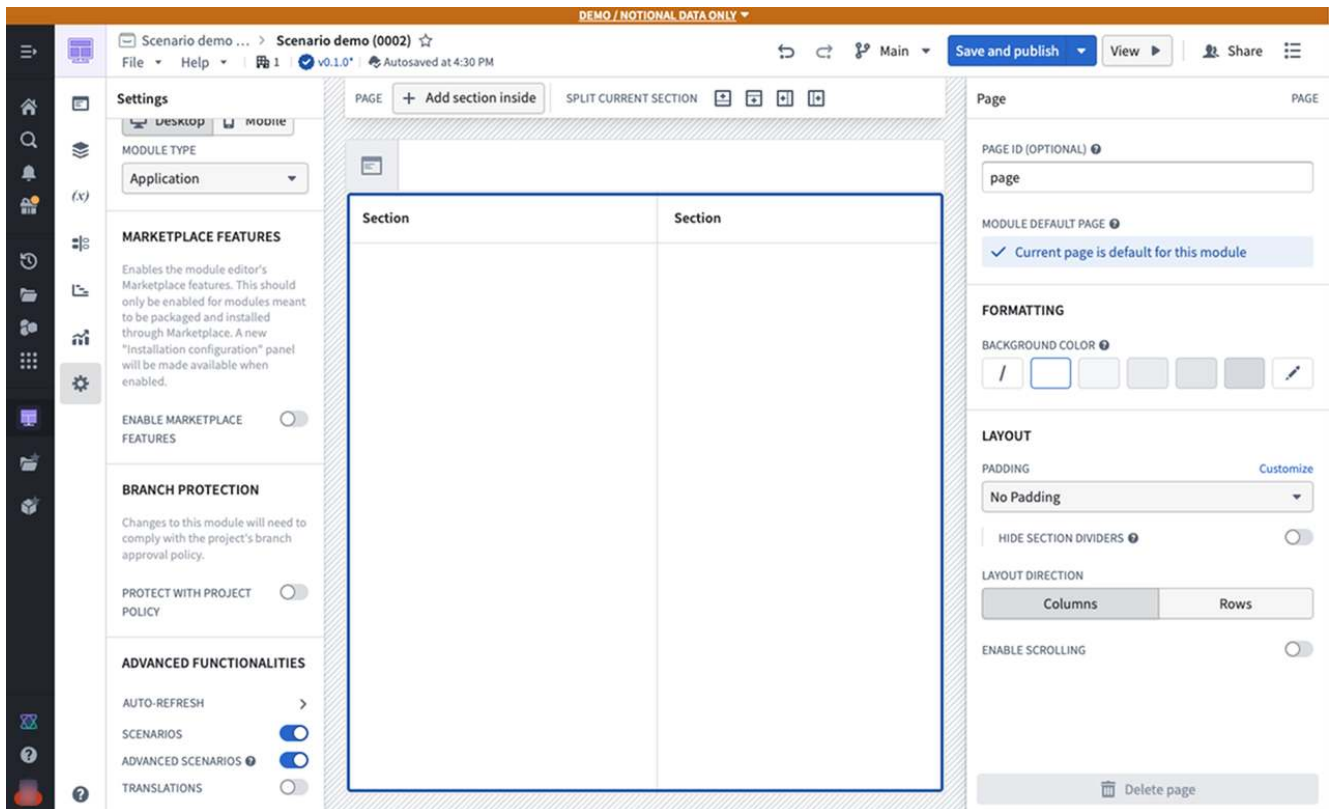
Part I: Add the Scenario Manager widget

1. From a new, empty Workshop module, navigate to the **Settings** panel on the left side navigation and ensure the **Scenarios** setting is enabled in the **Advanced Functionalities** section.

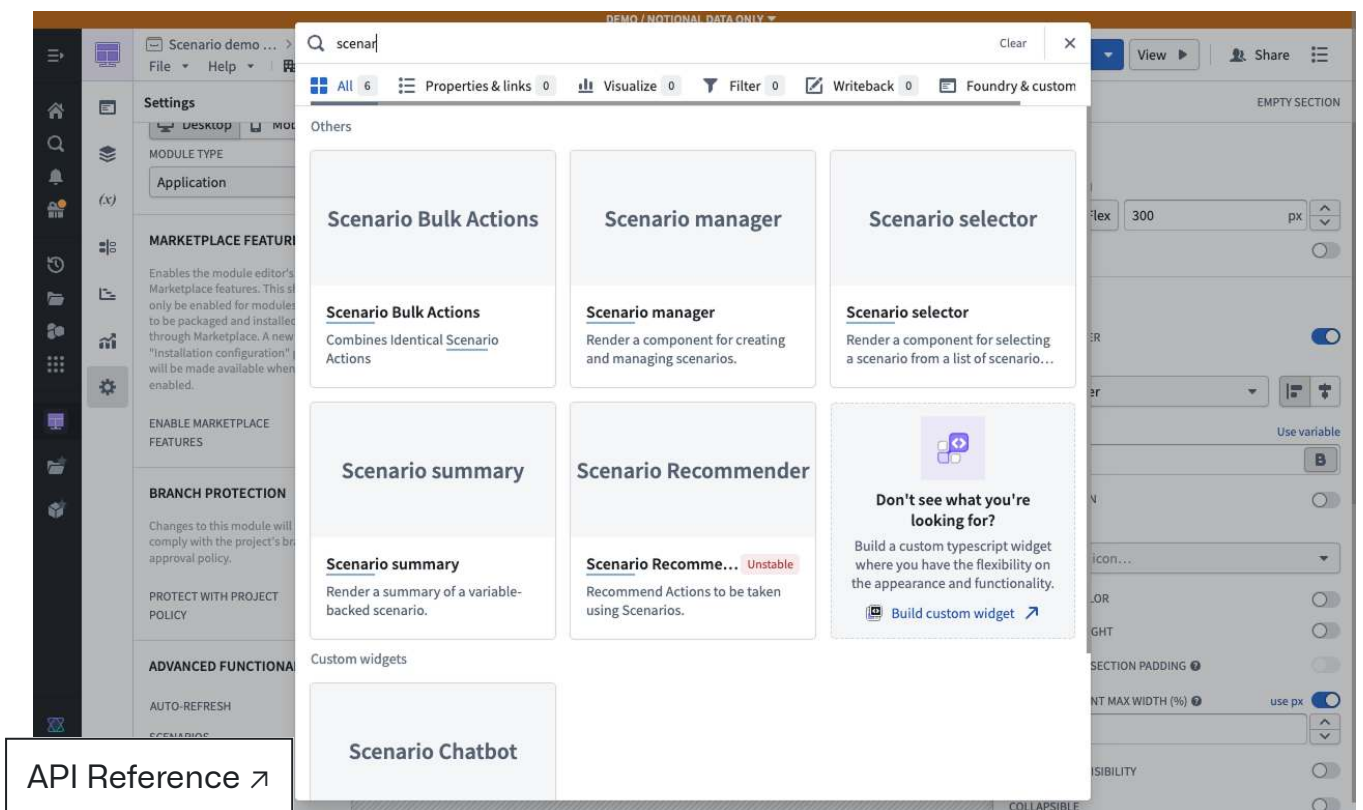
API Reference ↗



Send feedback

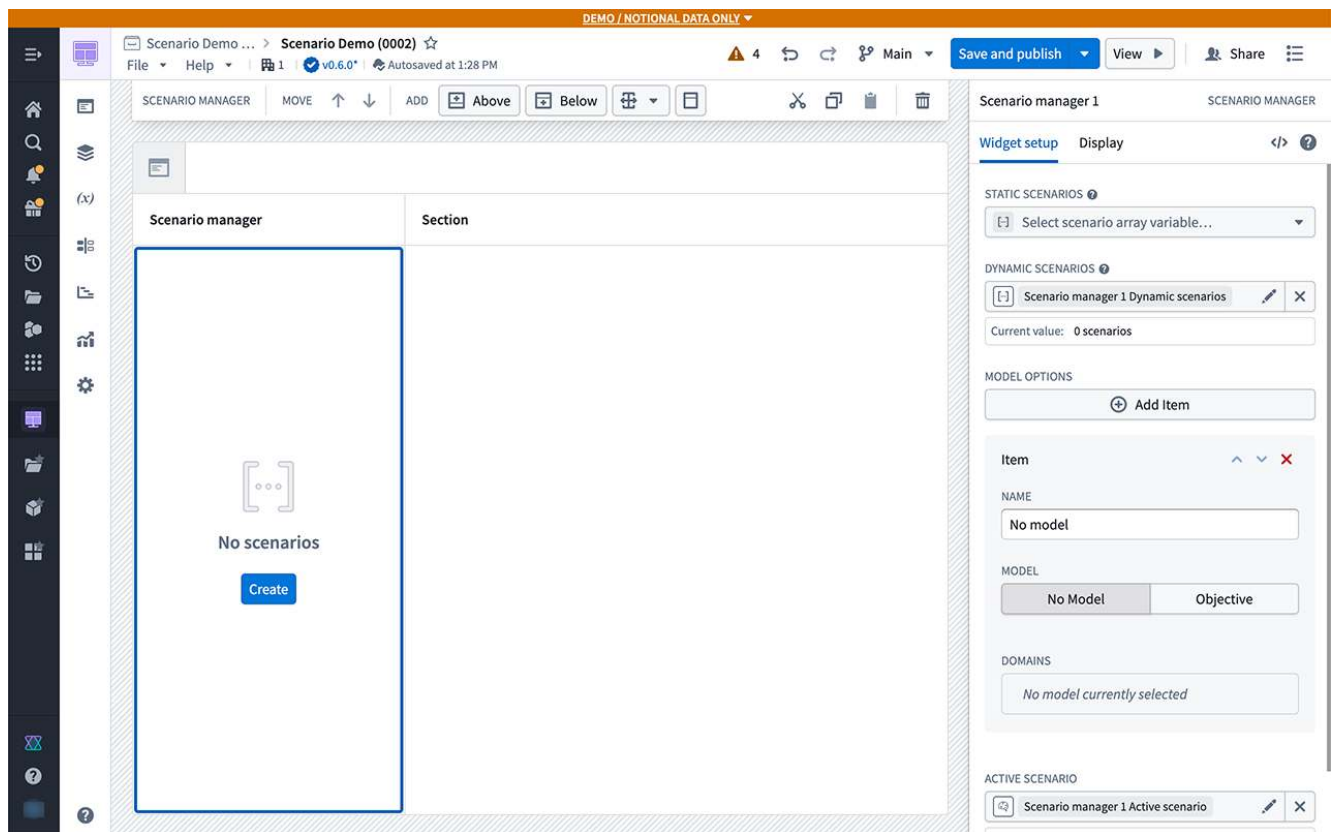


2. In the sidebar section, add a **Scenario Manager** widget. This is one of the scenario-specific widgets in Workshop and is used to create and manage temporary scenarios that will be used throughout the module.



Send feedback

3. From the widget setup panel on the right, configure the **Dynamic scenarios** field to use a new dynamic scenario. The **Create** button will appear in the center of the widget.

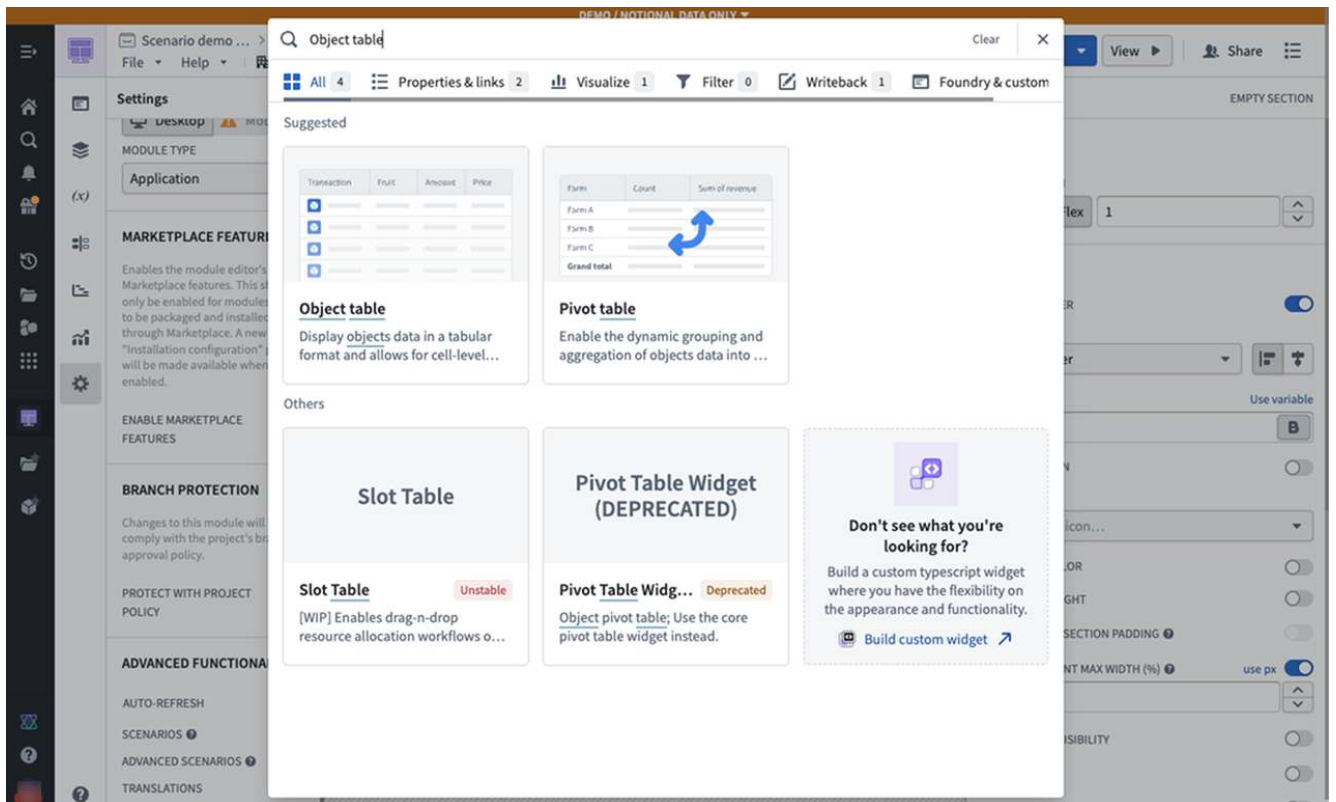


Part II: Add an Object Table widget

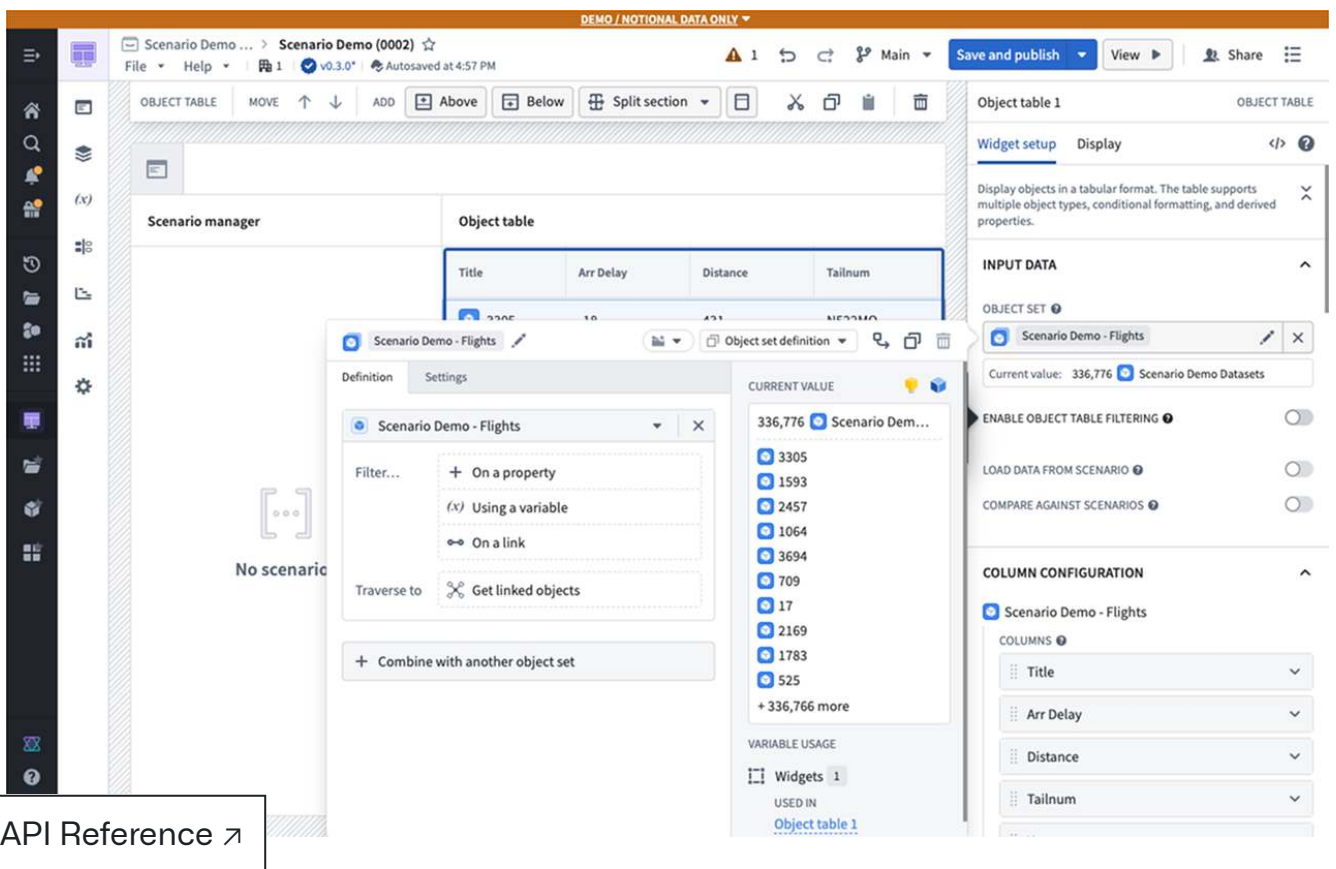
1. From the body of the module, add an **Object Table** widget.

[API Reference ↗](#)

[Send feedback](#)



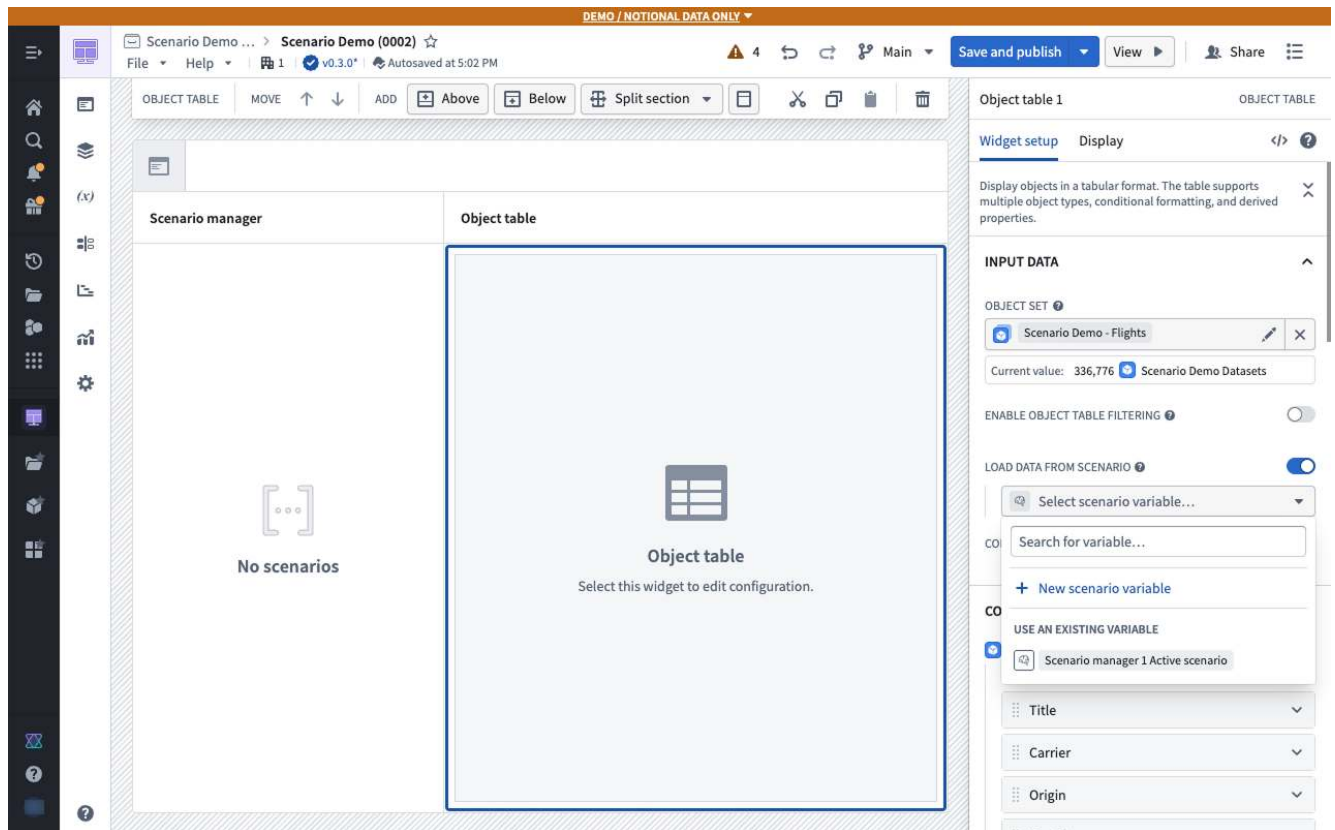
2. You can use any object set to populate the table, but we recommend starting with an object type that already has at least one associated [action](#) configured.



3. Now, add some properties to display in the table.

[Send feedback](#)

- Below the properties, enable the option to **Load Data from Scenario** into this widget.
This option effectively makes the Object Table widget scenario-aware.
- Select the scenario array variable produced by the Scenario Manager widget. This will cause the data in the Object Table widget to reflect any modifications to scenarios in the Scenario Manager widget rather than the raw Ontology.

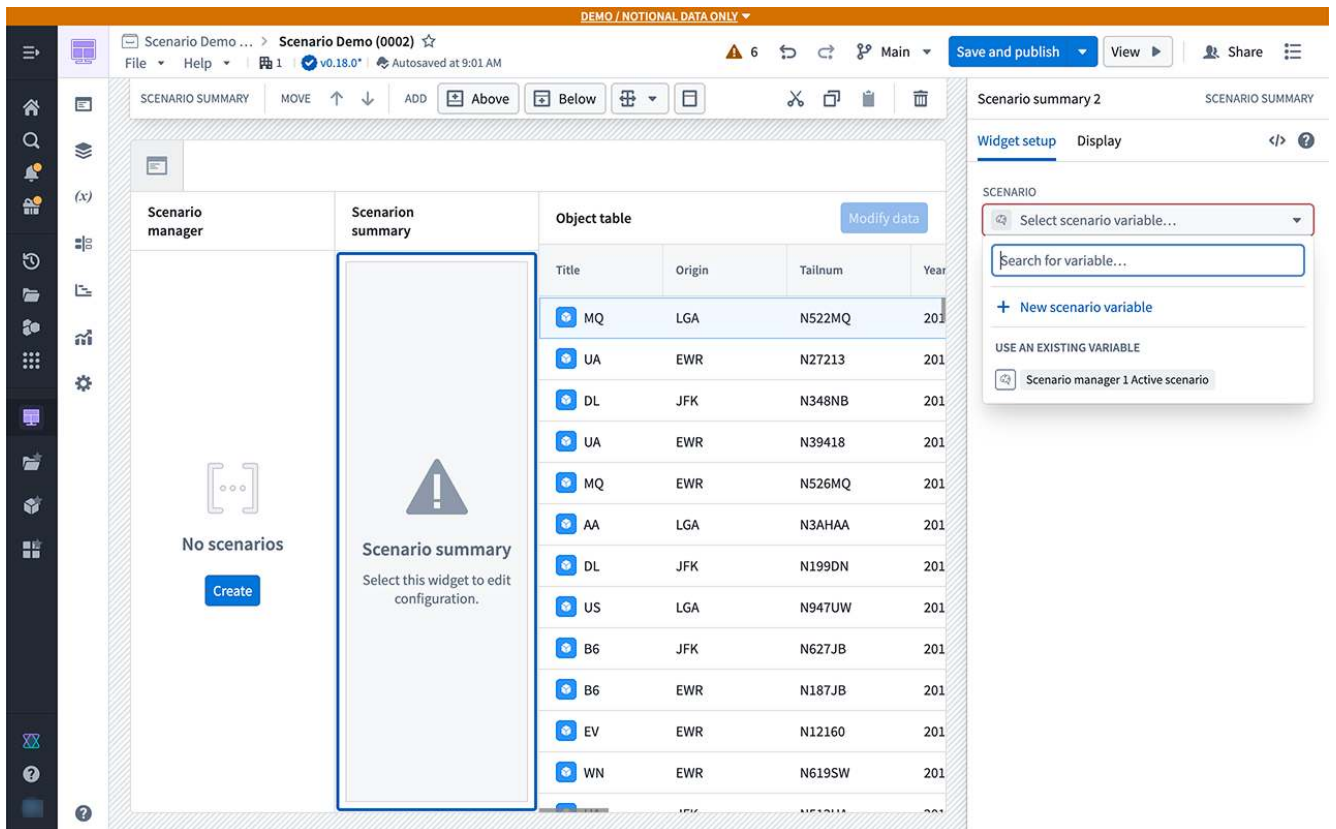


Part III: Add the Scenario Summary widget

- To preview the changes you will be making on the scenario, add the **Scenario Summary** widget to your module.
- In the widget configuration, connect to the active scenario variable generated by the Scenario Manager widget.

[API Reference ↗](#)

[Send feedback](#)



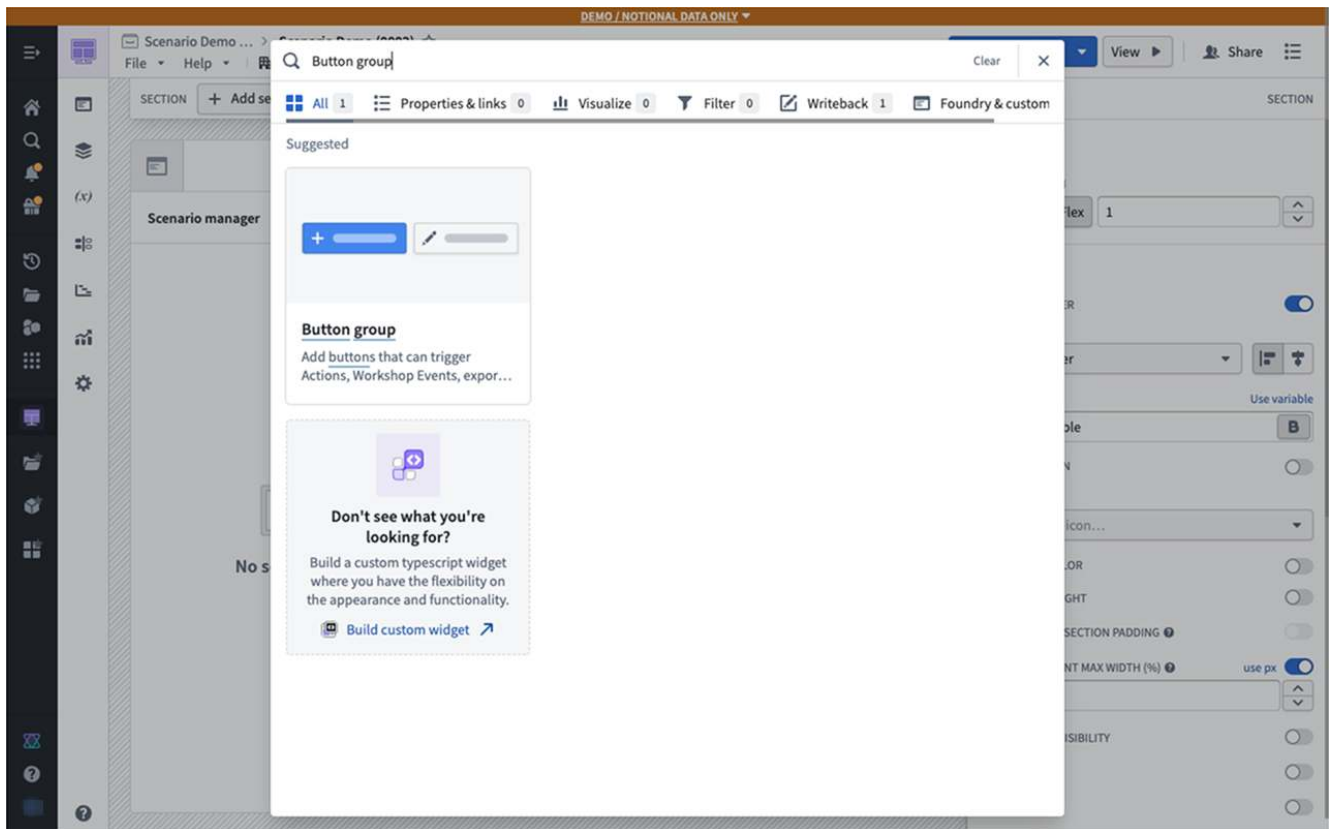
At this point, no modifications have been applied to the scenario; the data should be the same.

Part IV: Configure an action

1. Add a **Button Group** widget to configure an action to apply to your scenarios.

[API Reference ↗](#)

[Send feedback](#)



2. Select an action that modifies the object types in your table.
3. To apply this action to a scenario instead of the Ontology, enable the **Apply to Scenario** option and select the active scenario variable from the menu.

Scenario Demo (0002) ☆

File Help 1 v0.18.0* Autosaved at 9:04 AM

BUTTON GROUP SHIFT ← →

Scenario manager Scenario summary Object table Modify data

Title	Origin	Tailnum	Year
MQ	LGA	N522MQ	203
UA	EWR	N27213	201
DL	JFK	N348NB	201
UA	EWR	N39418	201
MQ	EWR	N526MQ	201
AA	LGA	N3AHAA	201
DL	JFK	N199DN	201
US	LGA	N947UW	201
B6	JFK	N627JB	201
B6	EWR	N187JB	201
EV	EWR	N12160	201
WN	EWR	N619SW	201

No scenarios Create

Uninitialized scenario

Scenario manager 1 Active scenario... BUTTON GROUP

Widget setup Display

ON CLICK

Action

Edit Scenario Demo on Scenario Demo - Flights

Default layout

Form Table

End-user features

SWITCH LAYOUT

PARAMETER DEFAULTS

Local default values for parameters

Select parameter to configure

Add all parameters

ADDITIONAL OPTIONS

HIDE FORM AND APPLY IMMEDIATELY IF VALID

CUSTOM ACTION TITLE

BUTTON STATE IF INVALID

Disabled Hidden

APPLY TO SCENARIO

Scenario manager 1 Active scenario

Send feedback

API Reference ↗

- 4. Using the newly configured action, try changing the property of any object in the table to a new value.
- 5. Before applying the action, create a new scenario in the Scenario Manager widget by selecting **Create**.

DEMO / NOTIONAL DATA ONLY

Scenario manager

Scenario 1

Create

Scenario summary

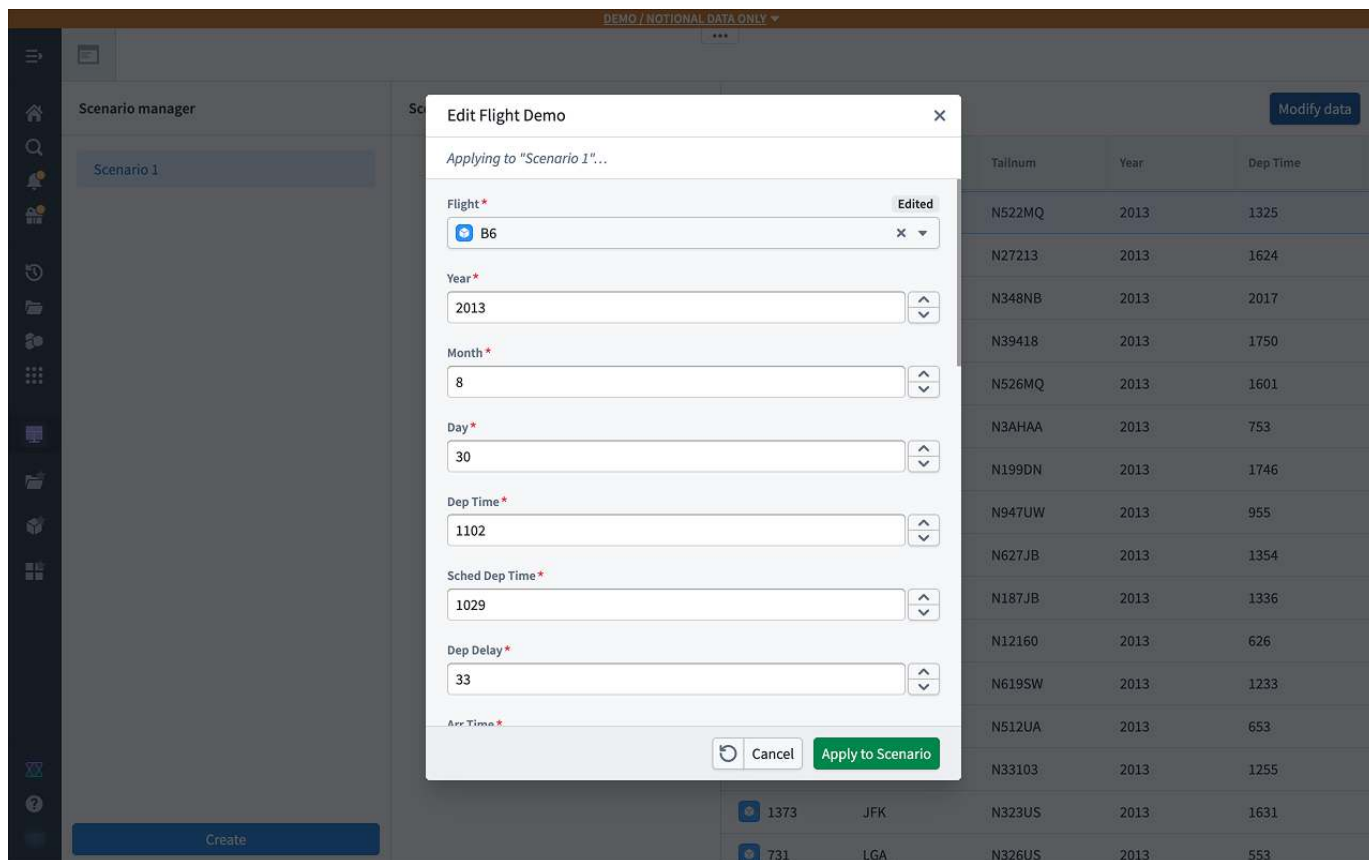
No changes in this scenario

Object table

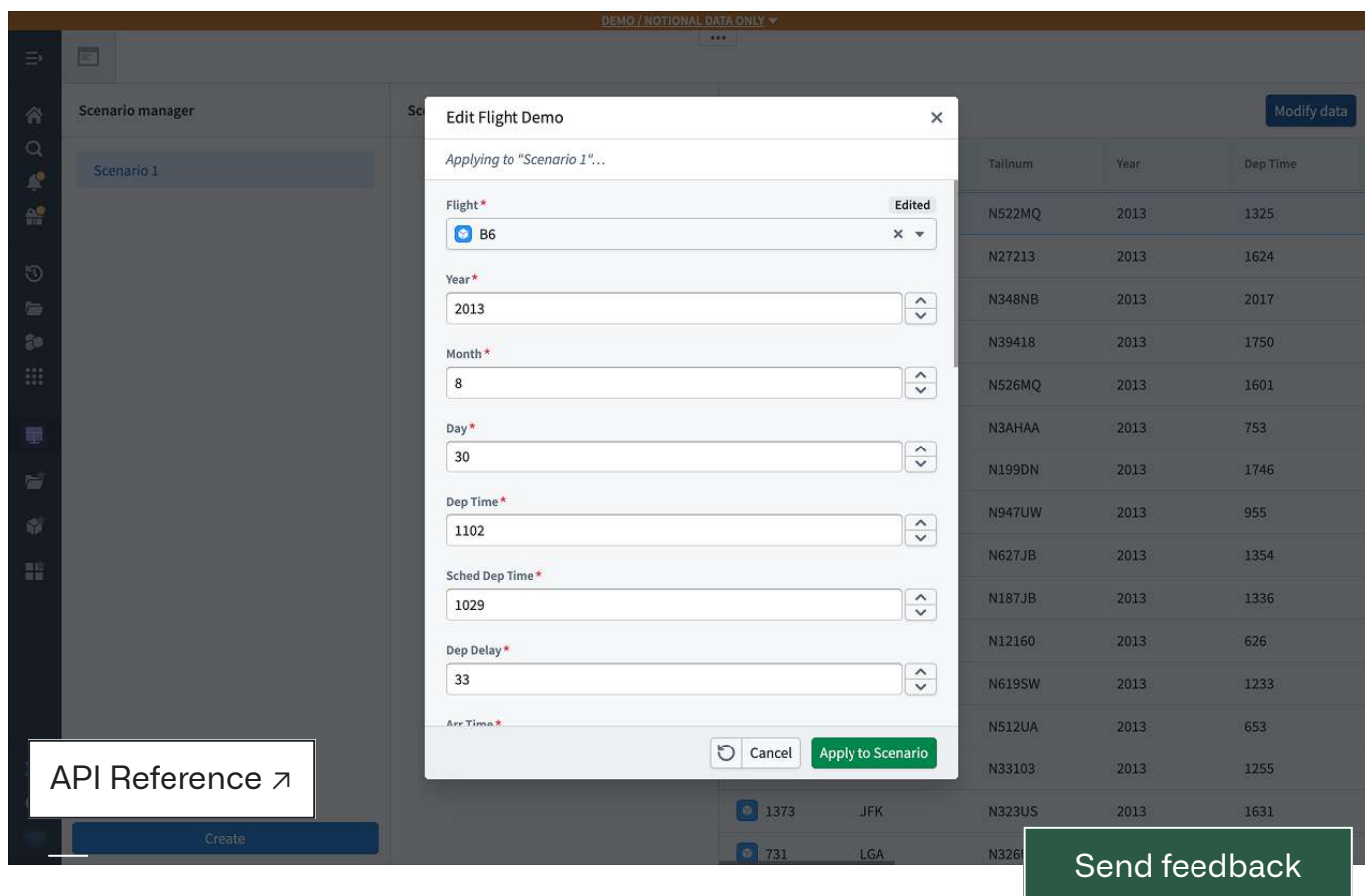
Title	Origin	Tailnum	Year	Dep Time
MQ	LGA	N522MQ	2013	1325
UA	EWR	N27213	2013	1624
DL	JFK	N348NB	2013	2017
UA	EWR	N39418	2013	1750
MQ	EWR	N526MQ	2013	1601
AA	LGA	N3AHAA	2013	753
DL	JFK	N199DN	2013	1746
US	LGA	N947UW	2013	955
B6	JFK	N627JB	2013	1354
B6	EWR	N187JB	2013	1336
EV	EWR	N12160	2013	626
WN	EWR	N619SW	2013	1233
UA	JFK	N512UA	2013	653
UA	EWR	N33103	2013	1255
DL	JFK	N323US	2013	1631
DL	LGA	N326US	2013	553

Modify data

In the example below, the **Year** for a flight in the table is being updated. You should see the Object Table widget refresh with the new data.



A record will appear in the Scenario Summary widget indicating the change in data. This action has *not* been applied to the Ontology and exists only within the scenario.

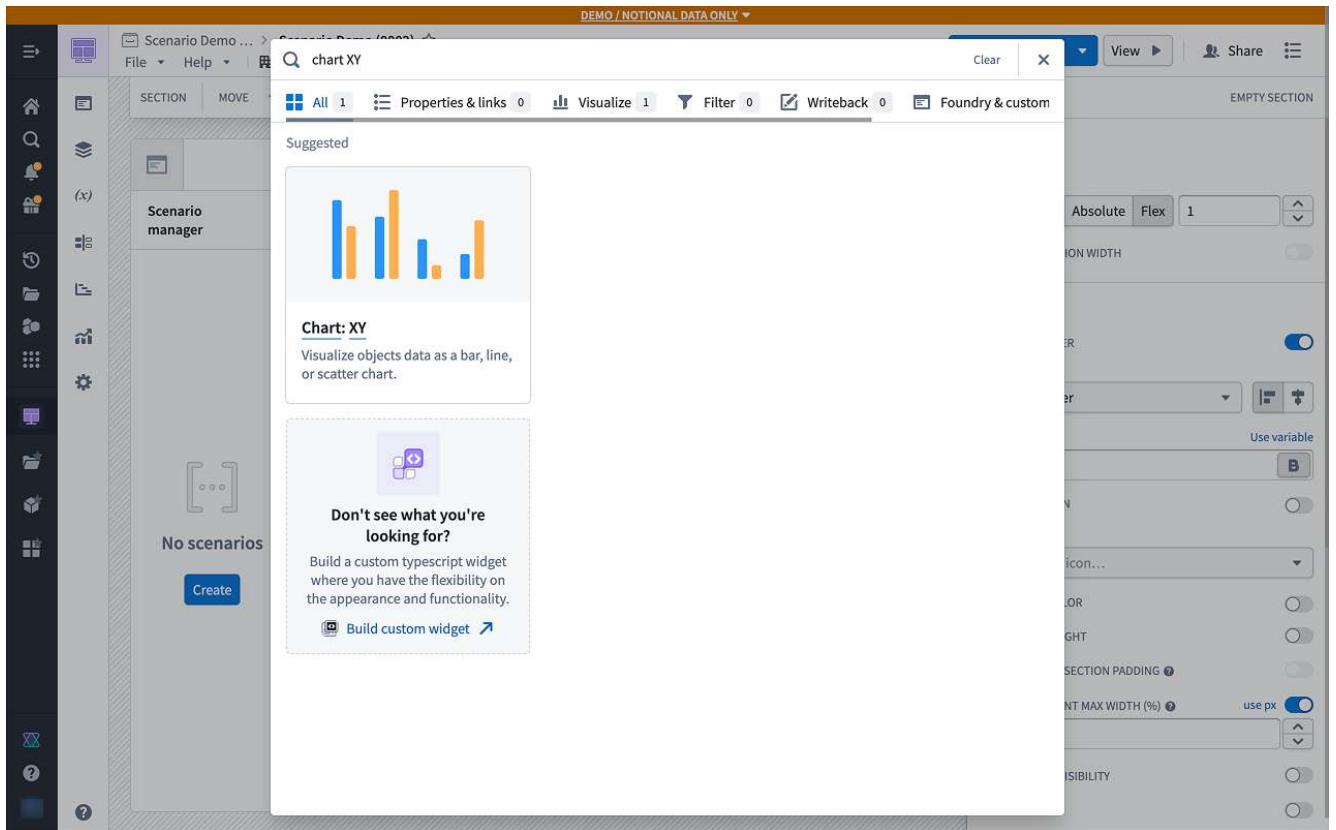


API Reference ↗

Send feedback

Part V: Add a chart widget

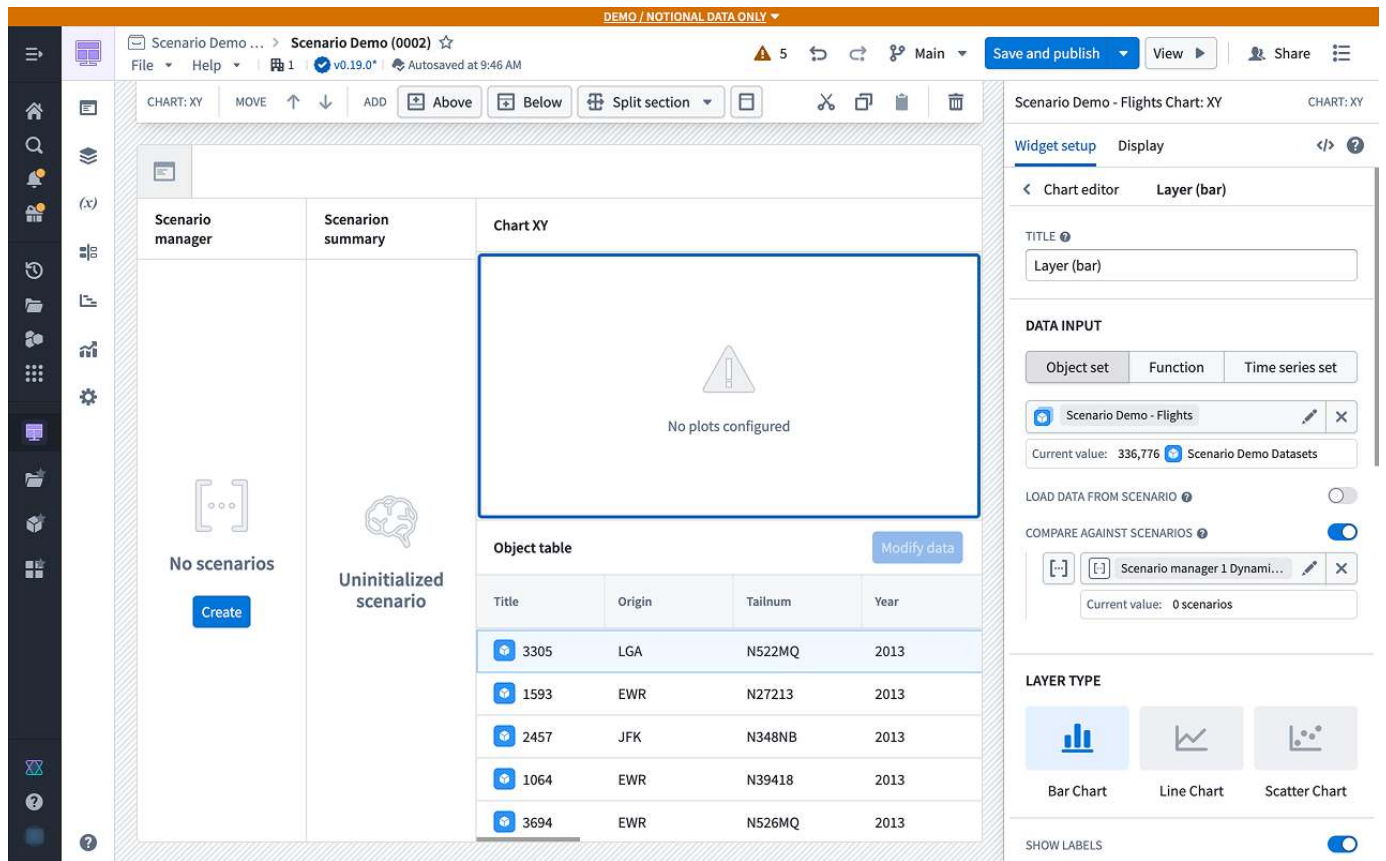
1. To better visualize the results, add a section above the table with a **Chart: XY** widget.



The Chart: XY widget supports an arbitrary number of scenarios, and different scenarios can be configured in different layers.

API Reference ↗

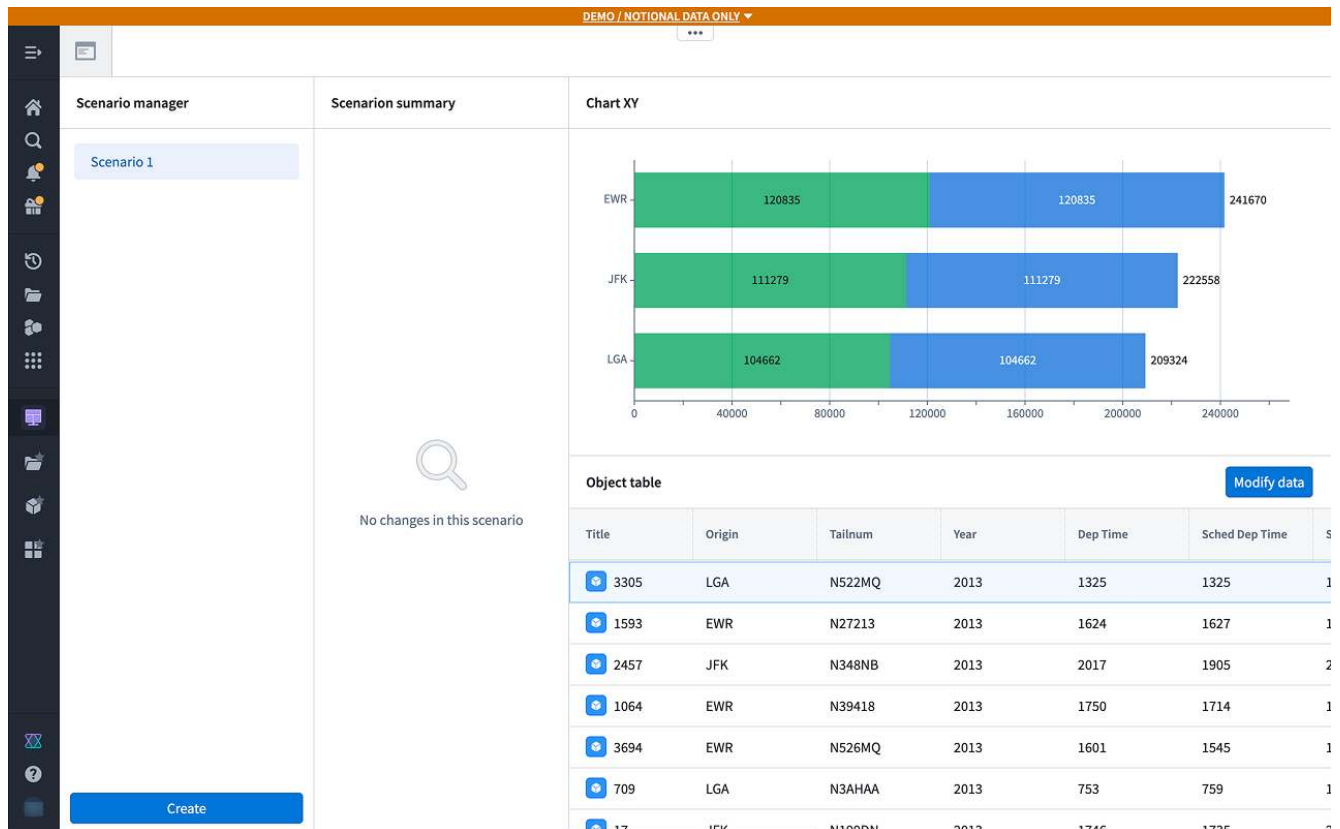
Send feedback



2. Try exploring the various layer types to see how multiple scenarios are visualized in them.
3. You can also configure **Group By** and other aggregates, which will properly respect scenario values.

[API Reference ↗](#)

[Send feedback](#)

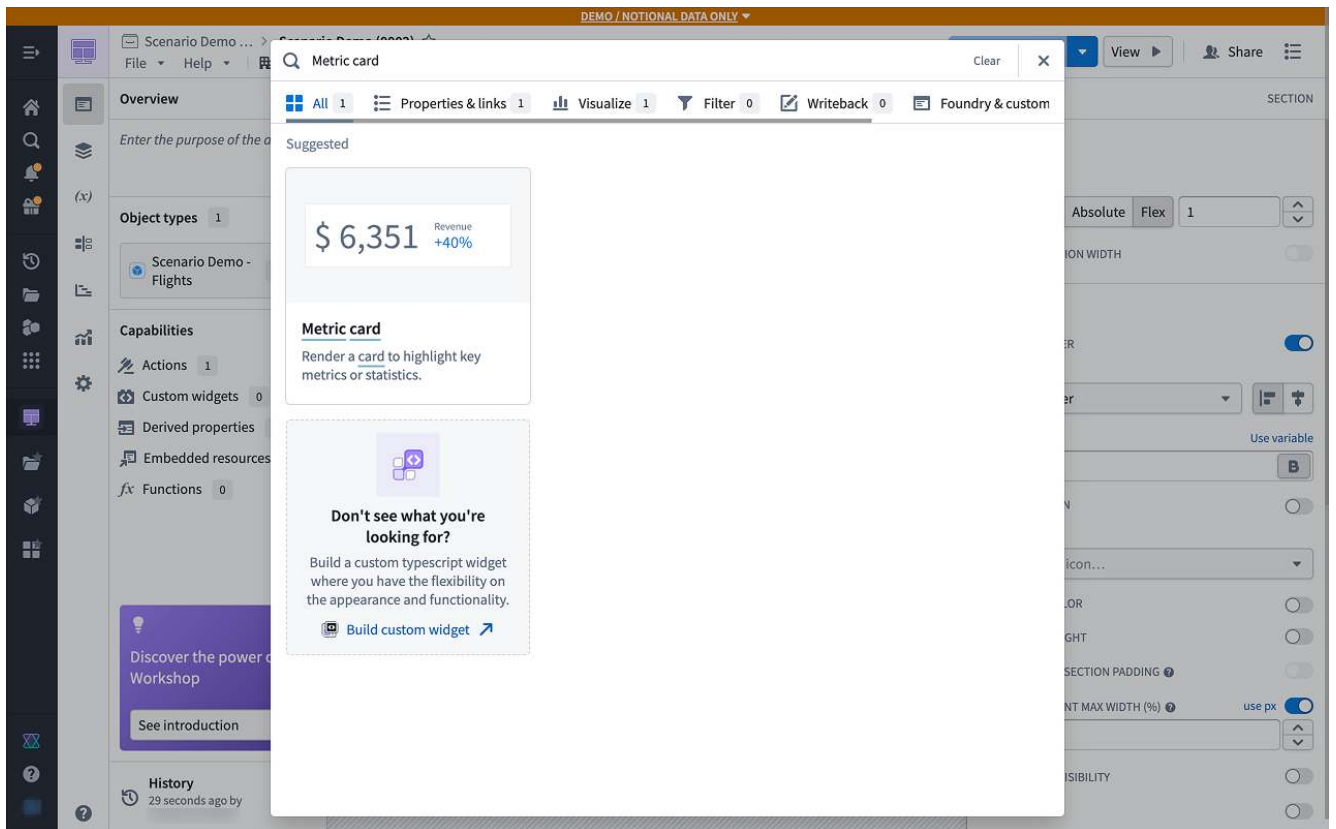


Part VI: Add a Metric Card widget

1. You can also populate values in metric cards from scenarios. Add a new **Metric Card** widget.

[API Reference ↗](#)

[Send feedback](#)



2. You can configure variable values from scenarios to use in widgets that are not inherently scenario-aware. The Metric Card widget, for example, does not have an explicit scenario configuration section. Instead, in the Metric Card configuration, create a new numeric metric with a value defined by a new object set aggregation variable.

The screenshot shows the 'Metric card 1' configuration window. The 'Widget setup' tab is active, and a dropdown menu is open, listing various variable types: Static, Function, Object set aggregation, Object property, Time series, Variable transformation, and Struct field. The background interface includes a sidebar with 'Overview', 'Object types' (Scenario Demo - Flights), 'Capabilities' (Actions, Custom widgets, Derived properties, Embedded resources, Functions), and a 'History' section. The main area shows the 'Metric card 1' configuration with a 'Display' tab and a 'Metric 1' section.

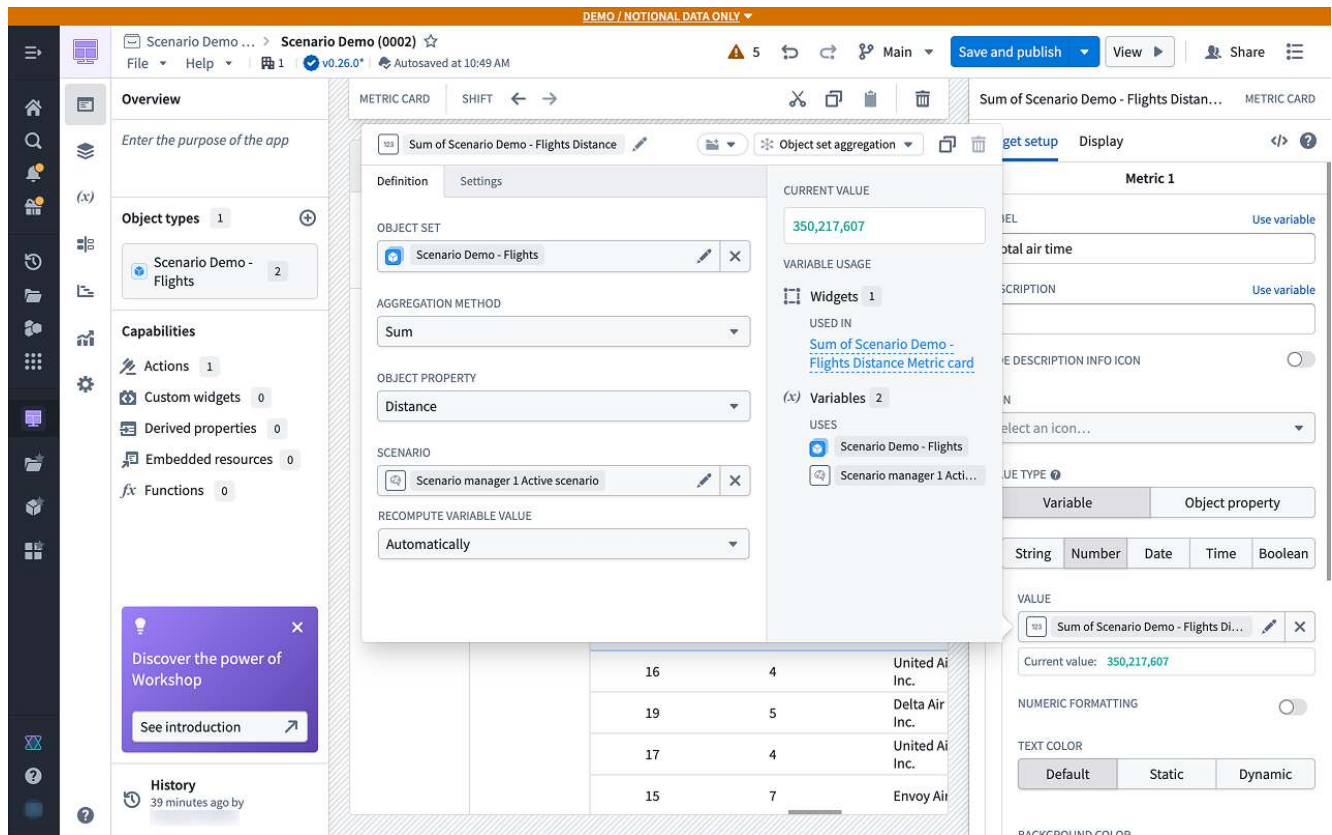
Origin	Count
EWR	120835
JFK	111279
LGA	104662

Hour	Month
13	7
16	4
19	5
17	4
15	7

Delta Air Inc.
United Ai Inc.
Envoy Air

Send feedback

In the variable configuration pane, a **Scenario** section lets you add a scenario variable. If selected, the aggregation runs with the scenario's modifications applied. Similarly, object property variable configurations also support scenarios.



Next steps

Now that you have created your first scenario, we recommend experimenting with various configurations and layouts of the scenario-specific Workshop widgets used in our tutorial.

Persist temporary scenario metadata

If you determine that you do want to persist this scenario's metadata, follow the guidance in our [persisted scenarios](#) documentation.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Store scenario metadata as objects

Store scenario metadata as objects

Beta

Ontology scenarios are in the [beta](#) phase of development and may not be available on your enrollment. Contact Palantir Support for access to the Ontology scenario interface.

You can define and store metadata of a scenario in the Ontology, where each object represents a scenario, to create a **persisted** scenario.

When stored in the Ontology, scenarios are shareable and collaborative. Teams can compare multiple persisted scenarios side-by-side, attach rich metadata including descriptions and authorship, and link scenarios to related objects for easy organization. Persisted scenarios inherit Foundry's permissions and governance model, ensuring proper access control, and create a durable audit trail of the decision-making process.

Configure scenario object type

To configure the scenario object type in Ontology Manager, follow the steps below:

1. In Ontology Manager, navigate to the **Object types** tab and select **+ New object type**.
2. Follow the configuration wizard. At step 3 (**Properties**), choose to **Add** an interface, then search for the **Ontology Scenario** interface. Ensure that **Scenario Reference** is mapped to the primary key of your object type, then complete the remaining steps.

API Reference ↗



Send feedback

Create a new object type

1 Datasource

2 Metadata

3 Properties

4 Actions

5 Save location

STEP 3

Properties

Interfaces (optional) ?

+ Implement interface

Ontology Scenario

Scenario Reference

→

Fulfilled by ScenarioReference

✓

Source

Property

+ Add property

User input / actions

ScenarioReference

Primary key

User input / actions

ScenarioName

Title

User input / actions

CreatedBy

Primary key ?

ScenarioReference

Title ?

ScenarioName

Back

Next

- Once the new object type is created, open the **Properties** tab and confirm that the **Primary Key** property has a value type of **Scenario Reference** assigned to it.
- Optionally, add additional properties to store metadata about the scenario, such as **Created By** or **Description**. Save the configuration and wait for the initial sync to complete.

Configure create/save action

- In Ontology Manager, locate your Scenario Object Type. If you did not configure a **Create Object** action during the initial setup, select **+ New** in the **Action types** card to create one.
- Within the **Create Object** action configuration, add an additional parameter of type **String**. In the **Constraints** card, set the value type to **Scenario Reference Value**. Then disable the **Visibility** toggle to hide the parameter from end users.

API Reference ↗

Send feedback

DEMO / NOTIONAL DATA ONLY

Ontology Manager

View all related resources

Main New

Back to Form

Form content

- Name
- Date created
- Scenario reference

Scenario reference

No description

Type: String

ID: scenarioChanges

RID: ...e7-4ef0-ac5d-8bce2dee76d0

Value Display Overrides Details

General

Visible ☐ Add override Disabled ☐ Required ☒ Add override

Constraints

Allow multiple values ☐ Users will be able to select more than one string

User input Multiple choice User Group Value type SIDC Rich text (Markdown)

Scenario Reference Value

3. In the **Rules** tab of the Action configuration page, ensure the **Unique ID** property is mapped to the new **Scenario Reference** parameter.

DEMO / NOTIONAL DATA ONLY

Ontology Manager

View all related resources

Main New

Scenario Demo - Scenario Object Type

Scenario Demo - Create Scenario

Overview

Rules

Parameters

User Interface

Capabilities

Security & Submission Criteria

Automations

History

Observability

1. Create object Scenario Demo - Scenario Object Type

Scenario Demo - Scenario Object Type Experimental

PROPERTY	MAP TO
Name	Name
Date created	Date created
Unique ID	Scenario reference

+ Add property

+ Add new rule

Convert to Function Code Experimental

If you want to convert this action to a function backed action, the code below can be used as a starting point.

TypeScript v1 Python TypeScript v2

```

1 import { Edits, LocalDate, OntologyEditFunction } from "@foundry/functions-api";
2 import { Objects, ScenarioDemoScenarioObjectType } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5
6   @Edits(ScenarioDemoScenarioObjectType)
7   @OntologyEditFunction()
8   public async execute(edit: OntologyEditFunction, localDate: LocalDate, name: string, scenarioReference: string): Promise<void> {
9
10  }
11 }

```

API Reference

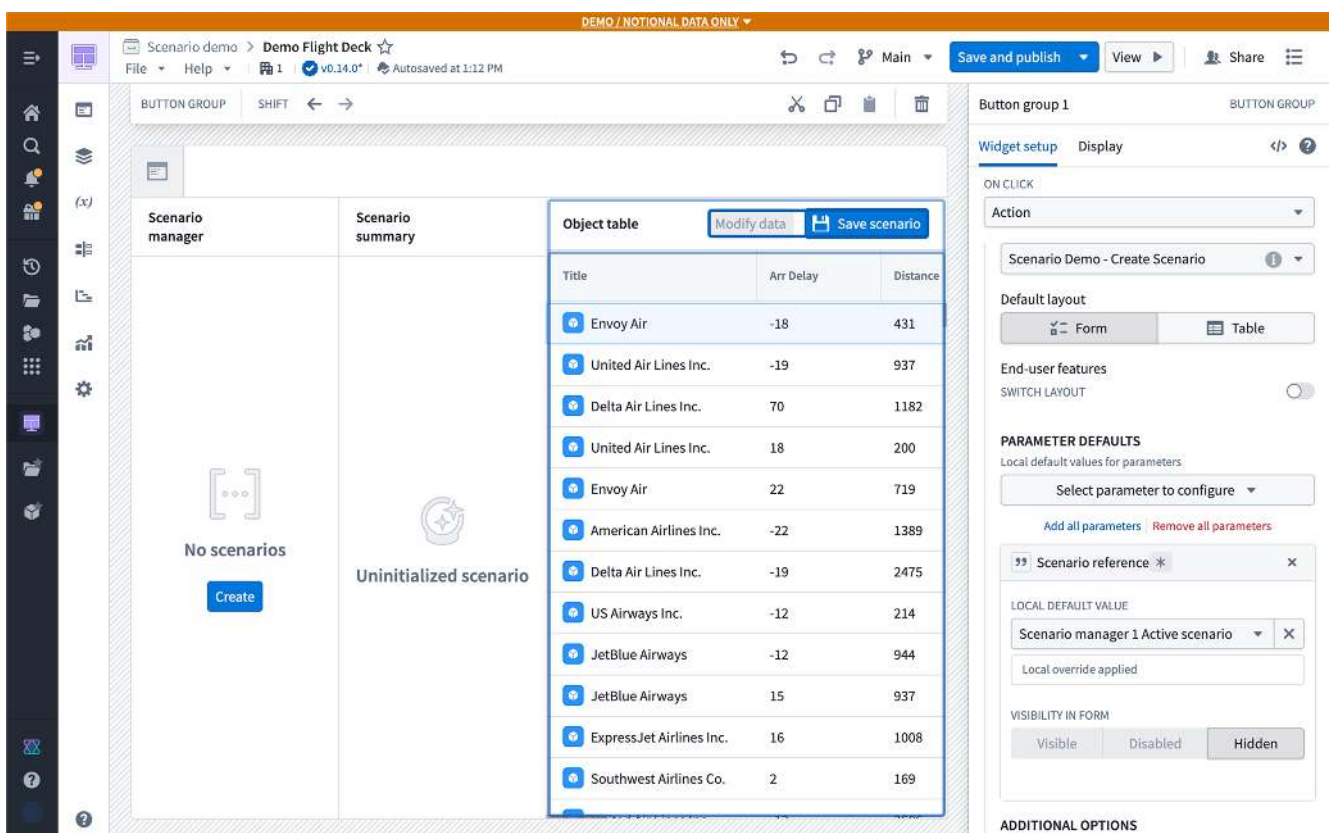
Last edited 1 hour ago by

Send feedback

Persist an existing temporary scenario

Before proceeding, ensure you have completed the [temporary scenario configuration tutorial](#). Once your first temporary scenario is in place, follow these steps in Workshop:

1. Add a **Button Group** widget to your application to handle saving the scenario.
2. Set the button's action to the **Create Object** action configured on your Scenario Object Type.
3. In the **Default Value** configuration, set the **Scenario Reference** parameter to be populated by the Active Scenario variable generated by the **Scenario Manager** widget.



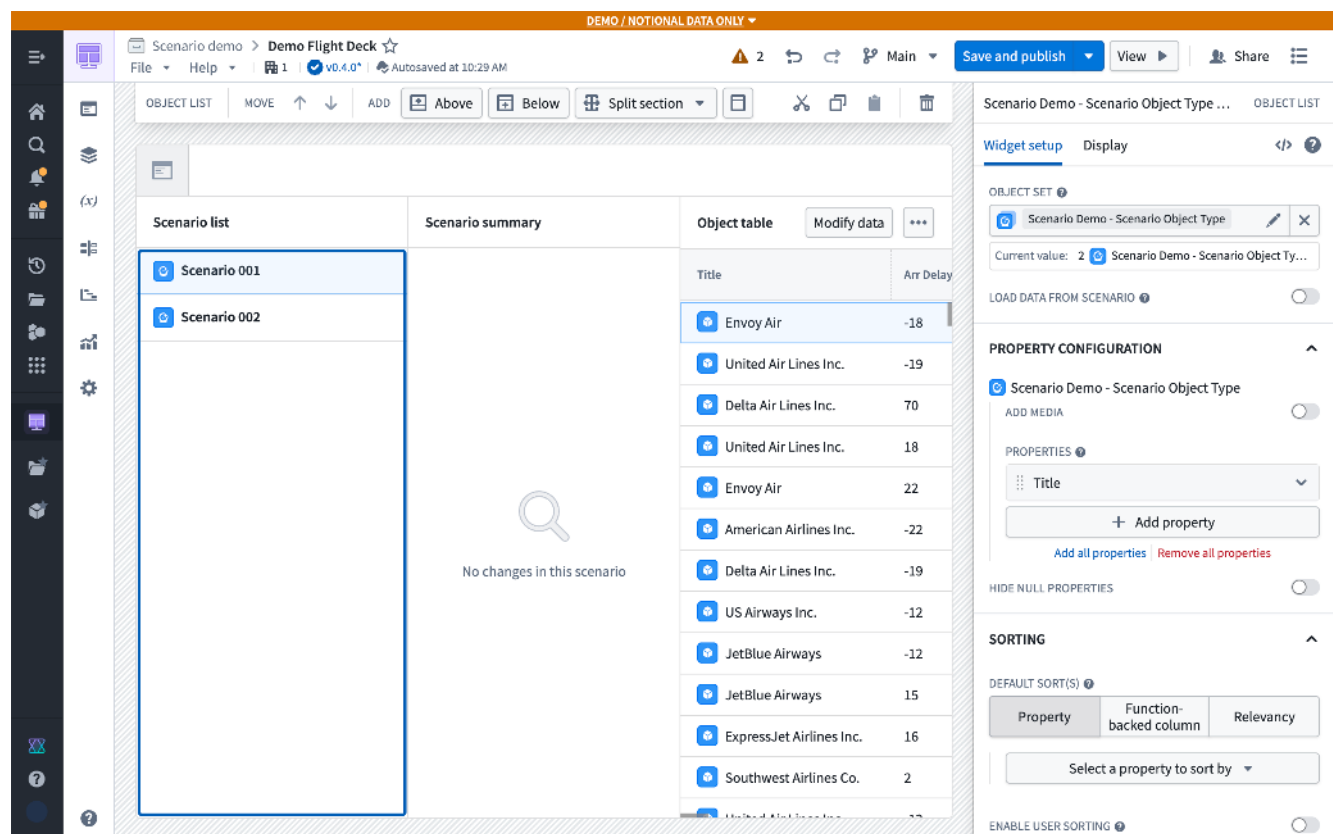
Create new scenario object

Make the following adjustments in Workshop to display persistent scenarios:

1. In the **Save Scenario** action, the **Scenario reference** parameter does not require a default value; it will be auto-generated upon action execution.
2. Replace the **Scenario Manager** widget with an **Object List** widget (display widget) configured to list your persisted Scenario objects.

Send feedback

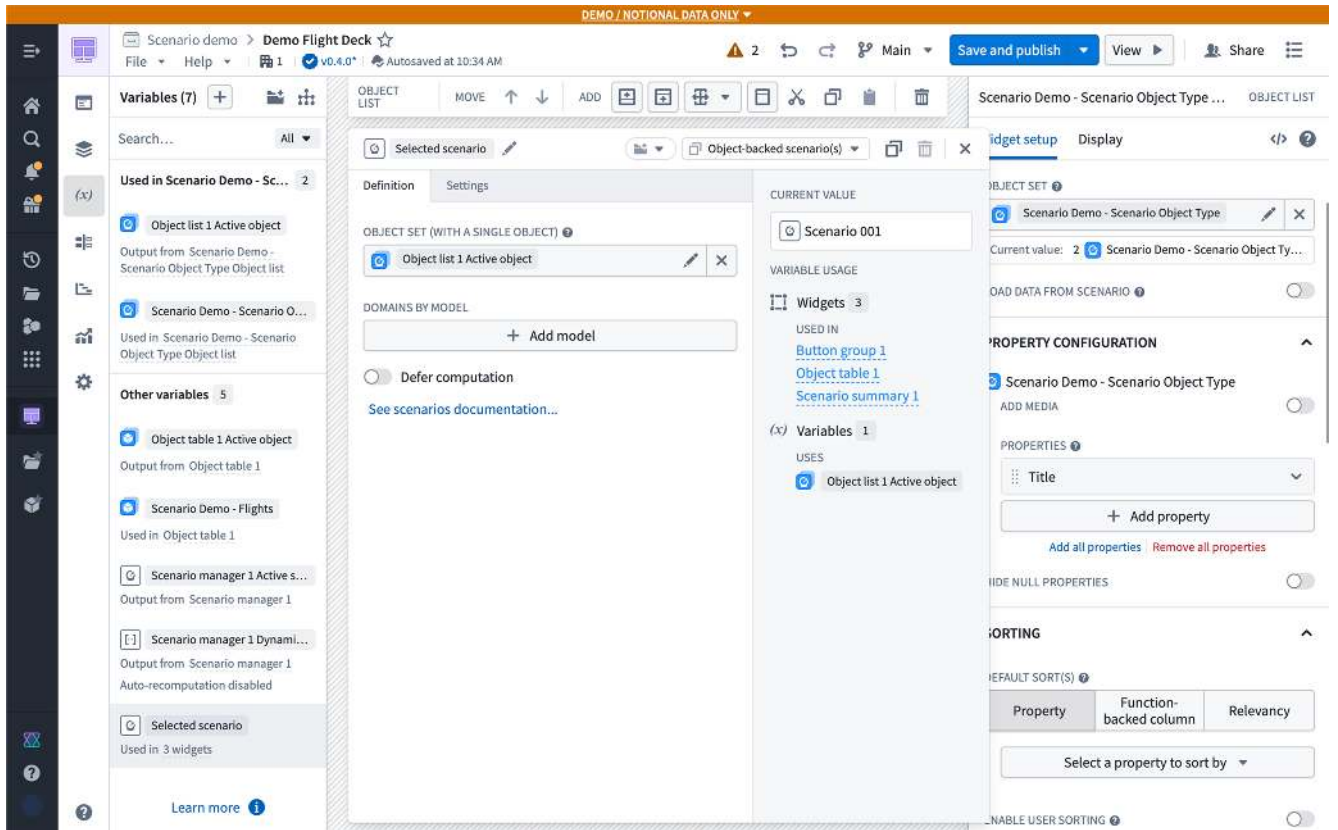
3. Set the **Scenario Object Type** as the data source for the newly added widget.



4. Replace the **Active Scenario** variable with an **Object-backed scenario** variable, and set its value to the **Active Object** from the **Object List** widget. You should update the variable selection in other widgets.

API Reference ↗

Send feedback



PREVIOUS
← Temporary scenarios

NEXT
Merge scenarios →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Merge scenarios

Scenarios are merged through action types that provide granular control over the scenario edits and enforce fine-grained execution permissions. At minimum, the merge action requires the single **Scenario** parameter, which is the RID of the scenario and includes all actions executed on the scenario. The parameter value can be passed from a Workshop variable or OSDK application.

Apply edits on a scenario

When working within a scenario, all edits (creating objects, modifying properties, deleting objects, and adding or removing links) exist only within that scenario's isolated sandbox. These edits do not affect the main Ontology. Applying the scenario commits all those staged edits to the Ontology as a single transaction via the merge action.

Configure the merge action

1. In **Ontology Manager**, navigate to the **Action types** tab and select **+ New action type**. In the creation wizard, select the **Scenario** tab.
2. Within the **Scenario** tab, define the scope of your scenario edits by adding the relevant **Object types** and **Link types**.

[API Reference ↗](#)[Send feedback](#)

New action
Action type

Create a new action type

1 Action type
2 Metadata
3 Submission criteria
4 Save location

STEP 1
Select an action type you want to configure
Enable users to make changes to the ontology by configuring actions they can execute.

Object Link Function Webhook Interface Notification Scenario Beta

Configure an action type that creates or edits object instances and links of the types specified below, with the changes sourced from a Scenario.

Object Type(s)
Select object types

Link Type(s)
Select link types

Skip Next

3. After completing the wizard, open the **Security & Submission Criteria** tab of the newly created action to define which users are permitted to execute the merge action.

i If you have an existing action that you want to use as a merge action, you can add an **Apply Scenario rule** to it from the **Rules** tab in the action configuration in Ontology Manager, rather than creating a new action type.

Discover

New action

Overview

Rules

Parameters

User Interface

Capabilities

API Reference

1. Apply scenario Beta

Configure an action type that creates or edits object instances and links of the types specified below, with the changes sourced from a Scenario.

Object Type(s)
Select object types

Link Type(s)
Select link types

Scenario parameter
Scenario

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Use scenarios with OSDK

⚠ Experimental

Scenario support in the [Ontology SDK \(OSDK\)](#) is experimental and may not be available on your enrollment; be aware that the API surface described on this page is under active development and subject to change.

The [Ontology SDK \(OSDK\)](#) allows you to create and interact with [Ontology scenarios](#) directly from your application code. A scenario-scoped client behaves like a regular OSDK client: every object you read, every aggregation you compute, and every action you apply is automatically evaluated against the scenario's isolated sandbox rather than the main Ontology.

Scenario support is available in both the TypeScript and Python OSDK. The sections below describe how to perform the following operations in each language:

- Create a new scenario or attach to an existing one.
- Read and query objects in the context of a scenario.
- Apply actions and edits within a scenario.
- Merge a scenario into the main Ontology.
- List the entities edited within a scenario.

TypeScript OSDK

See [API Reference](#) available in `@osdk/client` version `2.27.0` and later; generate a new version of your SDK in [Developer Console](#).



Send feedback

i [Custom widgets](#) use the OSDK and can also create and interact with scenarios. Review [Use Ontology SDK \(OSDK\) in a widget set](#) for more information.

Create or attach to a scenario

Use `createScenario` to create a new scenario, or `withScenario` to attach to an existing scenario by its RID. Both functions return a scenario-scoped client.

```
1 import { createScenario, withScenario } from "@osdk/client/unstable-do-nothing";
2
3 // Create a new scenario. This is asynchronous because it calls the platform.
4 const scenario = await createScenario(client);
5
6 // Or attach to an existing scenario by RID. This is synchronous and makes a REST call.
7 const existingScenario = withScenario(client, "ri.actions..scenario.0000-0000-0000-0000-0000-0000-0000-0000");
8
9 // Retrieve the scenario reference (RID) to persist or share.
10 const scenarioRid: string = scenario.getScenarioReference();
```

If the base `client` is configured with a branch, the new scenario created by `createScenario` uses that branch as its base.

You can also attach to a scenario that is stored as a [persisted scenario object](#). The primary key of a persisted scenario object is its scenario reference, so pass the object's `$primaryKey` to `withScenario`.

```
1 // `scenarioObject` is a persisted scenario object you already have, from the platform.
2 // The scenario object's primary key is the scenario reference (RID).
3 const scenario = withScenario(client, scenarioObject.$primaryKey);
```

API Reference ↗ within a scenario

The scenario client is callable exactly like a regular OSDK client. All requests are scoped to the scenario.

Send feedback

```

1 import { Restaurant } from "@my-app/sdk";
2
3 // Load a page of objects as they appear within the scenario.
4 const page = await scenario(Restaurant).fetchPage();
5
6 // Filters and aggregations are also evaluated against the scenario.
7 const filtered = await scenario(Restaurant)
8   .where({ numberOfReviews: { $gte: 100 } })
9   .fetchPage();
10
11 const counts = await scenario(Restaurant).aggregate({
12   $select: { $count: "unordered" },
13 });

```

Apply actions within a scenario

Apply an action through the scenario client to stage edits inside the scenario's sandbox. These edits do not affect the main Ontology until the scenario is merged.

```

1 import { addReview } from "@my-app/sdk";
2
3 const result = await scenario(addReview).applyAction({
4   restaurantId: "restaurantId",
5   reviewRating: 5,
6   reviewSummary: "It was great!",
7 });

```

⚠ Batch actions are not supported when applying actions within a scenario. Scenarios also cannot be nested within another scenario or combined with a transaction.

Next

[API Reference ↗](#)

Scenario

To merge a scenario, apply any action type configured with an **Apply** rule. Applying this rule to an action automatically exposes a scenario parameter of type

[Send feedback](#)

`scenarioReference`. Pass the scenario client directly as the value of that parameter; the OSDK resolves it to the scenario reference automatically.

```
1 import { applyScenario } from "@my-app/sdk";
2
3 // `applyScenario` is an action configured with an Apply Scenario rule, wh
4 // exposes a scenario parameter named `scenario`.
5 await client(applyScenario).applyAction({ scenario });
```

The action and parameter names are specific to your Ontology. Review [Merge scenarios](#) for guidance on configuring the **Apply Scenario** rule on an action type.

List edited entities within a scenario

The scenario client exposes methods to discover which objects and links were edited within a scenario. These results return sparse identifiers: only the `$apiName` and `$primaryKey` fields are populated. To load full property values, pass the primary keys back through the scenario client.

Edited objects

Use `getEditedEntityTypes` to discover which object and link types changed, then `getEditedEntities` to page through the edited objects of a given type, or `editedEntitiesAsyncIter` to stream them.

```
1 import { Restaurant } from "@my-app/sdk";
2
3 // Discover which object types and link types were edited in the scenario
4 const { objectTypes, linkTypes } = await scenario.getEditedEntityTypes();
5
6 // Page through edited objects of a given object type.
7 const editedPage = await scenario.getEditedEntities(Restaurant, { pageSiz
8 for (const obj of editedPage.data) {
9   log(obj.$apiName, obj.$primaryKey);
```

API Reference ↗

```
11
12 // Or stream all edited objects of a given object type
13 for await (
```

Send feedback

```

14   const obj of scenario.editedEntitiesAsyncIter(Restaurant, { pageSize: 5
15   } {
16     console.log(obj.$primaryKey);
17   }

```

To load full property values for the edited objects, re-fetch them through the scenario client.

```

1   const keys = editedPage.data.map((obj) => obj.$primaryKey);
2   const full = await scenario(Restaurant)
3     .where({ $primaryKey: { $in: keys } })
4     .fetchPage();

```

Edited links

Use `getEditedLinkTypes`, `getEditedLinks`, and `editedLinksAsyncIter` to inspect edited links. Each edited link is returned as a directed triple of `source`, `target`, and `linkType`. Only many-to-many links are returned; edited one-to-many links surface through `getEditedEntities` on the object type that owns the foreign key.

```

1   import { Restaurant } from "@my-app/sdk";
2
3   // Discover which many-to-many link types for a given object type were edited.
4   const editedLinkTypes = await scenario.getEditedLinkTypes(Restaurant);
5
6   // Page through edited links for a given link type.
7   const linksPage = await scenario.getEditedLinks(Restaurant, "sisterRestaurant",
8     pageSize: 200,
9   });
10  for (const { source, target, linkType } of linksPage.data) {
11    console.log(source.$primaryKey, "->", target.$primaryKey, `(${linkType})`);
12  }
13
14  // Or stream all edited links of a given link type.
15  for await (
16    const { source, target } of scenario.editedLinksAsyncIter(
17      Restaurant,
18      "sisterRestaurant",

```

API Reference ↗

```

19   )
20   ) {

```

Send feedback

```
21 console.log(source.$primaryKey, "->", target.$primaryKey);
22 }
```

Python OSDK

Scenario support is available in Python OSDK version `2.209.0` and later. Listing edited objects and links requires version `2.221.0` or later. Scenario support is a beta feature: you must run scenario code within an `AllowBetaFeatures` context.

Create or attach to a scenario

Use `ScenarioClient.create` to mint a new scenario, or the `ScenarioClient` constructor to attach to an existing scenario by its RID. Both return a scenario-scoped client.

```
1 from foundry_sdk_runtime import AllowBetaFeatures
2 from osdk import FoundryClient, ScenarioClient, UserTokenAuth
3
4 with AllowBetaFeatures():
5     # Create a new scenario. This calls the platform to mint the scenario
6     scenario_client = ScenarioClient.create(client)
7
8     # Or attach to an existing scenario by RID.
9     scenario_client = ScenarioClient(
10         client, scenario_rid="ri.actions..scenario.0000-..."
11     )
12
13     # Retrieve the scenario reference (RID) to persist or share.
14     scenario_rid = scenario_client.scenario_rid
```

The new scenario created by `ScenarioClient.create` inherits the ontology and branch of the base `client`. The base client passed to `ScenarioClient` must not itself be a scenario client.

Y API Reference ↗ to a scenario that is stored as a [persisted scenario object](#). The primary key of a persisted scenario object is its scenario reference, so pass the `scenario_rid` to the `ScenarioClient` constructor.

Send feedback


```

1 with AllowBetaFeatures():
2     # `scenario_object` is a persisted scenario object you already have, f
3     # The scenario object's primary key is the scenario reference (RID).
4     scenario_client = ScenarioClient(
5         client, scenario_rid=scenario_object.get_primary_key()
6     )

```

Query objects within a scenario

Read and query objects through `scenario_client.ontology.objects`. All reads are automatically scoped to the scenario.

```

1 with AllowBetaFeatures():
2     # Load a single object as it appears within the scenario.
3     restaurant = scenario_client.ontology.objects.ExampleRestaurant.get("p
4
5     # Load a page of objects from within the scenario.
6     page = scenario_client.ontology.objects.ExampleRestaurant.page(page_si

```

Apply actions within a scenario

Apply an action through the scenario client to stage edits inside the scenario's sandbox. These edits do not affect the main Ontology until the scenario is merged.

```

1 from foundry_sdk_runtime.types import ActionConfig, ActionMode
2
3 with AllowBetaFeatures():
4     scenario_client.ontology.actions.add_review(
5         action_config=ActionConfig(mode=ActionMode.VALIDATE_AND_EXECUTE),
6         restaurant_id="restaurantId",
7         review_rating=5,
8         review_summary="It was great!",

```

API Reference ↗

Send feedback

Merge a scenario

To merge a scenario, apply any action type configured with an **Apply Scenario** rule. Adding this rule to an action automatically exposes a scenario parameter of type `scenarioReference`. Pass the scenario client (or its `scenario_rid` string) as the value of that parameter.

```
1 from foundry_sdk_runtime.types import ActionConfig, ActionMode
2
3 with AllowBetaFeatures():
4     # `apply_scenario` is an action configured with an Apply Scenario rule
5     # exposes a scenario parameter named `scenario`. The action and parameter
6     # are specific to your Ontology.
7     client.ontology.actions.apply_scenario(
8         action_config=ActionConfig(mode=ActionMode.VALIDATE_AND_EXECUTE),
9         scenario=scenario_client, # or scenario=scenario_client.scenario_rid
10    )
```

Review [Merge scenarios](#) for guidance on configuring the **Apply Scenario** rule on an action type.

List edited entities within a scenario

The scenario client exposes methods to discover which objects and links were edited within a scenario. These results return sparse identifiers: only the `object_api_name` and `primary_key` fields are populated. To load full property values, pass the primary keys back through the scenario client.

Edited objects

Use `get_edited_entity_types` to discover which object and link types changed, then `get_edited_objects` to page through the edited objects of a given type, or `iterate_edited_objects` to stream them.

API Reference ↗

```
1 from osdk.ontology.objects import ExampleRestaurant
2
3 with AllowBetaFeatures():
```

Send feedback

```

4      # Discover which object types and link types were edited in the scenario
5      edited_types = scenario_client.get_edited_entity_types()
6      print(edited_types.object_types)
7      print(edited_types.link_types)
8
9      # Load a page of edited objects of a given object type.
10     edited_page = scenario_client.get_edited_objects(
11         ExampleRestaurant, page_size=500
12     )
13     for obj in edited_page.data:
14         print(obj.object_api_name, obj.primary_key)
15
16     # Or stream all edited objects of a given object type. The iterator
17     # automatically paginates and deduplicates by primary key.
18     for obj in scenario_client.iterate_edited_objects(
19         ExampleRestaurant, page_size=500
20     ):
21         print(obj.primary_key)
22
23     # Or asynchronously stream all edited objects of a given object type.
24     async for obj in scenario_client.async_iterate_edited_objects(
25         ExampleRestaurant, page_size=500
26     ):
27         print(obj.primary_key)

```

To load full property values for the edited objects, re-fetch them through the scenario client. Deleted objects may return `None`.

```

1  with AllowBetaFeatures():
2      full = [
3          scenario_client.ontology.objects.ExampleRestaurant.get(obj.primary_key)
4          for obj in edited_page.data
5      ]

```

Edited links

Use `get_edited_link_types`, `get_edited_links`, and `iterate_edited_links` to inspect edited links. An edited link is returned as a directed triple of `source`, `target`, and `link_type`. Many-to-many links are returned; edited one-to-many links surface through `get_edited_objects` on the object type that owns the foreign

API Reference ↗

Send feedback

```

1  from osdk.ontology.objects import ExampleRestaurant
2
3  with AllowBetaFeatures():
4      # Discover which many-to-many link types for a given object type were
5      edited_link_types = scenario_client.get_edited_link_types(ExampleRest
6
7      # Load a page of edited links for a given link type.
8      links_page = scenario_client.get_edited_links(
9          ExampleRestaurant, "sisterRestaurants", page_size=200
10     )
11     for link in links_page.data:
12         print(
13             link.source.primary_key,
14             "->",
15             link.target.primary_key,
16             f"({link.link_type})",
17         )
18
19     # Or stream all edited links of a given link type.
20     for link in scenario_client.iterate_edited_links(
21         ExampleRestaurant, "sisterRestaurants"
22     ):
23         print(link.source.primary_key, "->", link.target.primary_key)
24
25     # Or asynchronously stream all edited links of a given link type.
26     async for link in scenario_client.async_iterate_edited_links(
27         ExampleRestaurant, "sisterRestaurants"
28     ):
29         print(link.source.primary_key, "->", link.target.primary_key)

```

PREVIOUS

← Merge scenarios

NEXT

Applications / Object Explorer / Overview →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[API Reference ↗](#)

[Terms of Use ↗](#)

Send feedback

